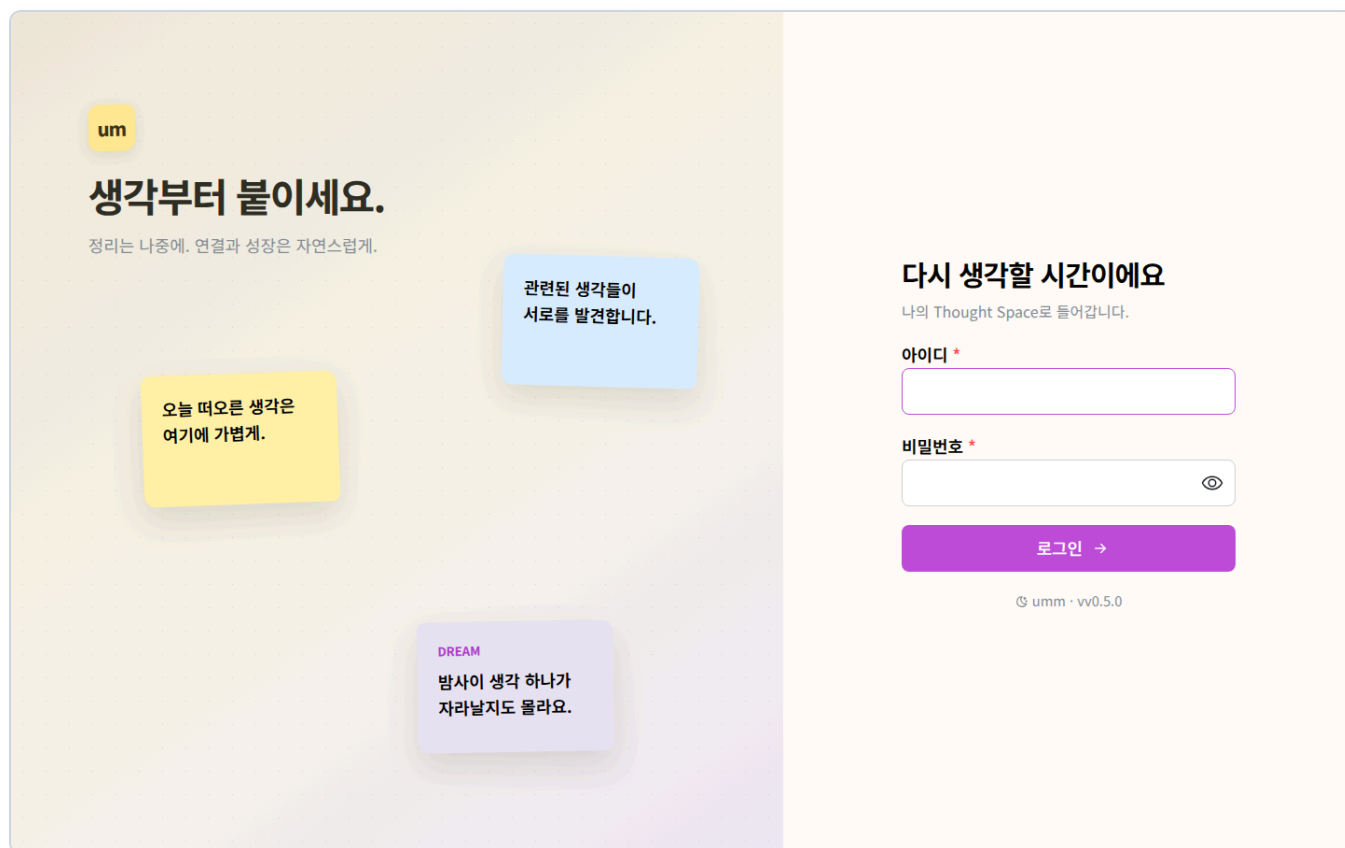


umm 기능 및 화면 가이드 (Features & UI Guide)

umm 은 생각을 문서나 폴더로 정리하기 전에 공간에 먼저 붙이고, 연결하고, 밤사이 **Dream**으로 다시 발견하는 **Spatial Thought Memory** 플랫폼입니다.

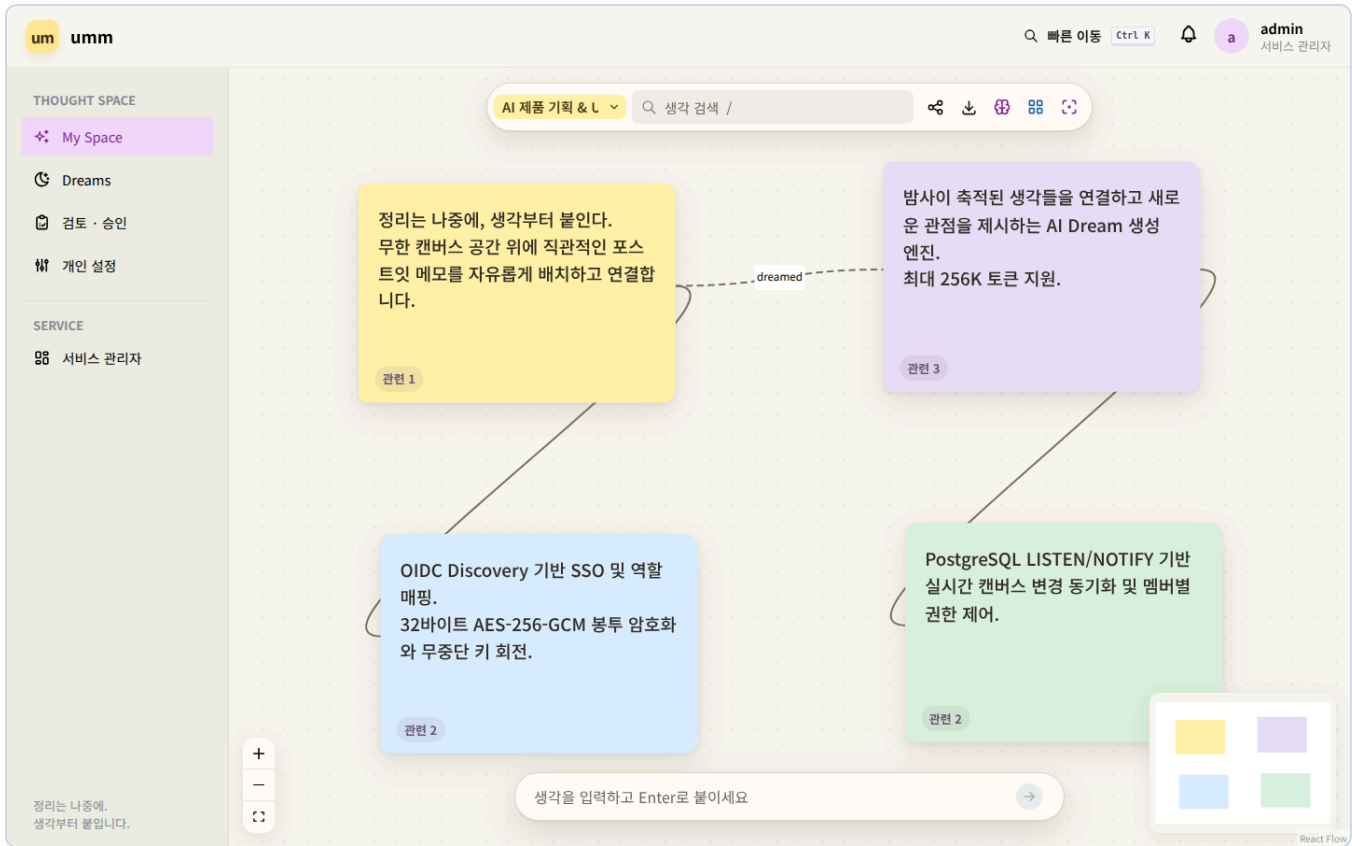
📸 전체 메뉴별 주요 화면 및 기능 명세

1. 인증 및 로그인 (/login)



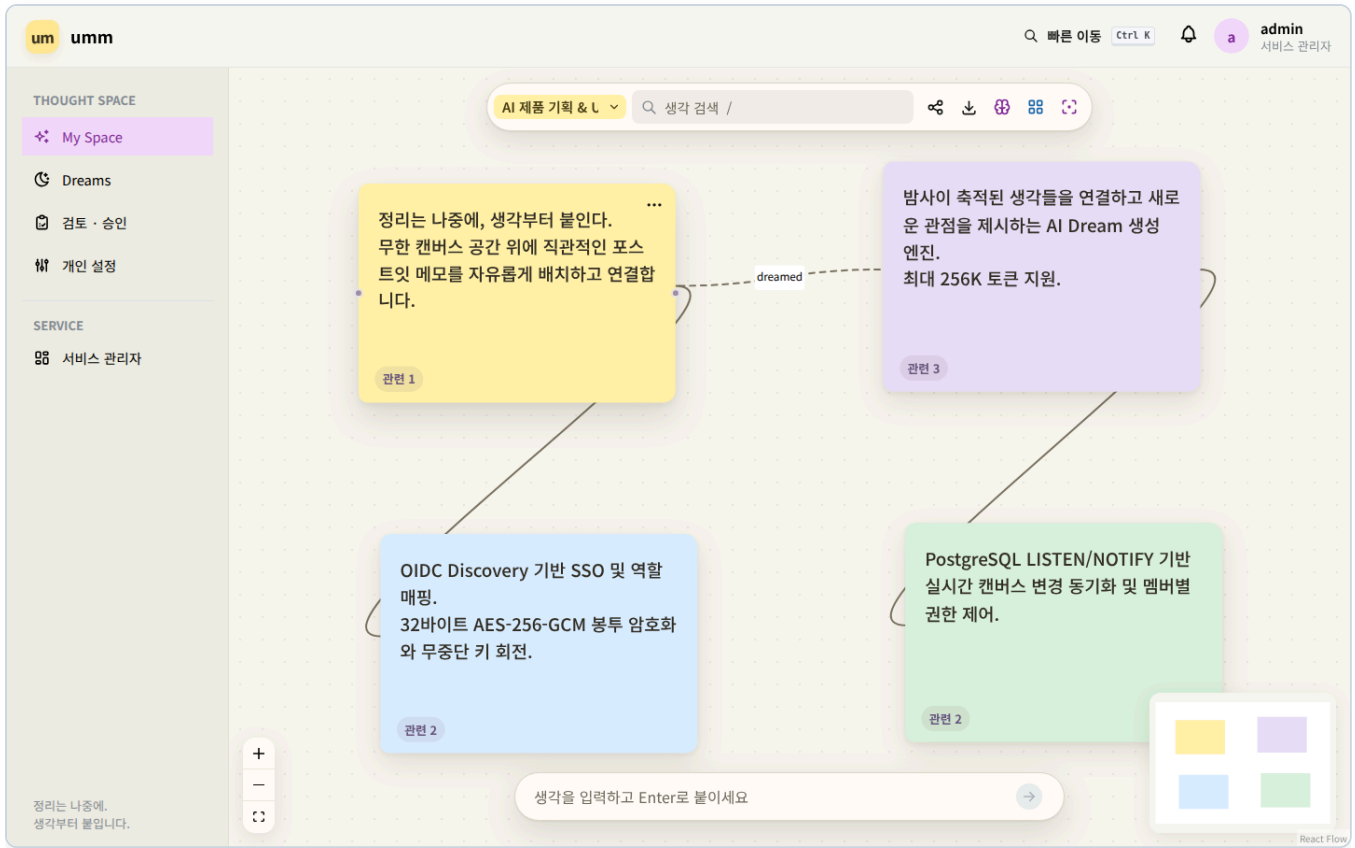
- 로컬 부트스트랩 관리자: 초기 환경변수(`BOOTSTRAP_ADMIN` , `BOOTSTRAP_ADMIN_PASSWORD`) 기반 보안 로그인
- Keycloak OIDC SSO: 설정 시 `Keycloak SSO로 계속` 버튼을 통한 엔터프라이즈 통합 로그인 지원
- 버전 및 서비스명: 하단에 현재 릴리스 버전(`v0.8.1`)과 서비스명 실시간 표시

2. 무한 생각 캔버스 (/space/:spaceId)



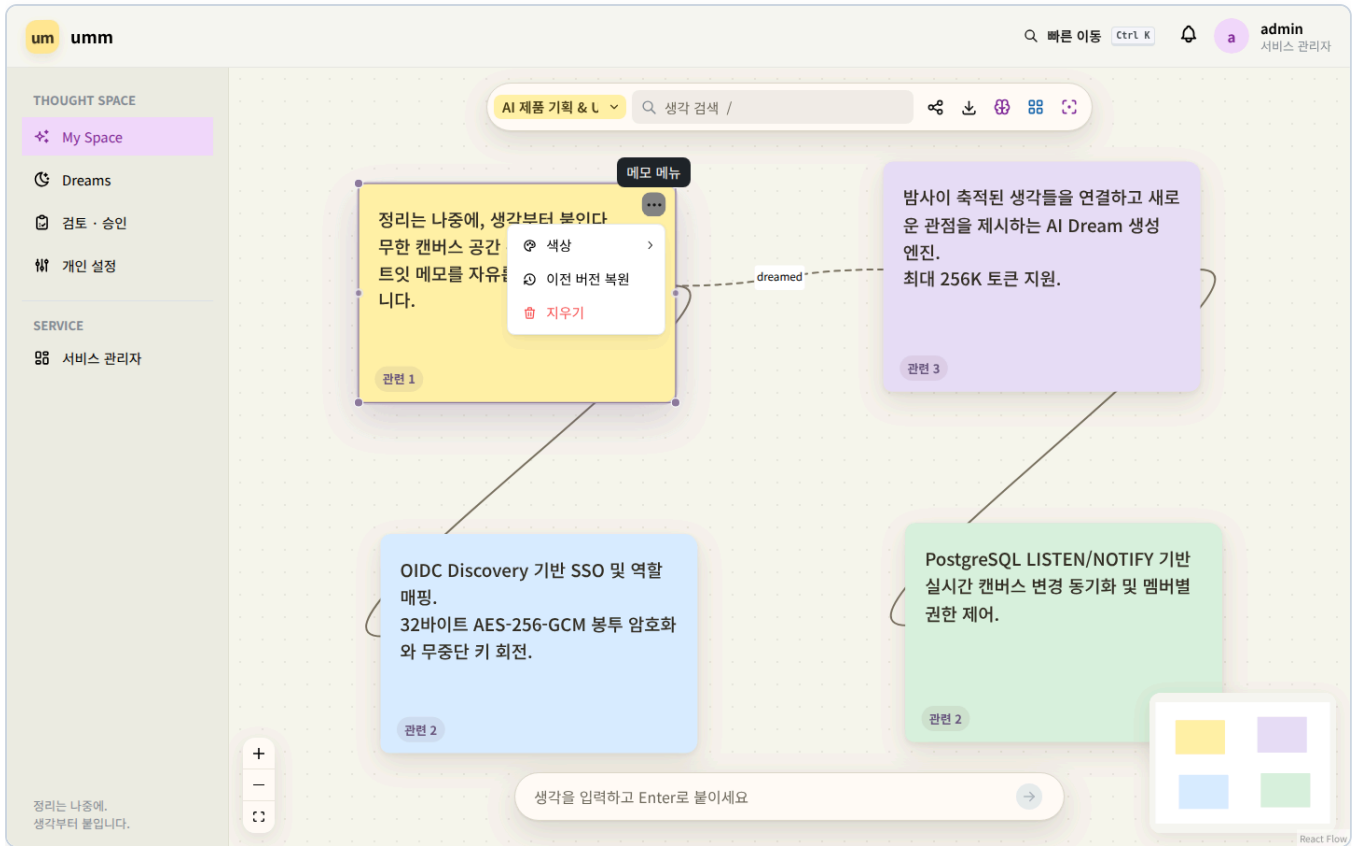
- **Spatial Infinite Canvas**: React Flow 기반의 자유로운 줌(Zoom), 팬(Pan), 미니맵 및 도트 그리드 배경
- **포스트잇 노트 배치**: 더블클릭, **N**, **/** 또는 하단 Quick Capture 바를 통해 공간 어디든 즉시 메모 생성
- **생각 연결선 (Edges)**: 노트 핸들을 드래그하여 생각 간의 관계(관련, Dreamed 등)를 직관적으로 연결
- **실시간 자동 저장**: 저장 버튼 없이 모든 입력, 위치 이동, 크기 변경이 PostgreSQL에 실시간 커밋
- **메모별 AI 제외**: 메모 메뉴에서 민감한 생각을 AI Assist와 Dream 분석 대상에서 제외하고, 외부 호출 직전의 source lease에서도 다시 확인. 외부 임베딩 Gateway에도 본문을 보내지 않고 로컬 vector만 사용

3. 포스트잇 상세 편집 및 컬러 테마



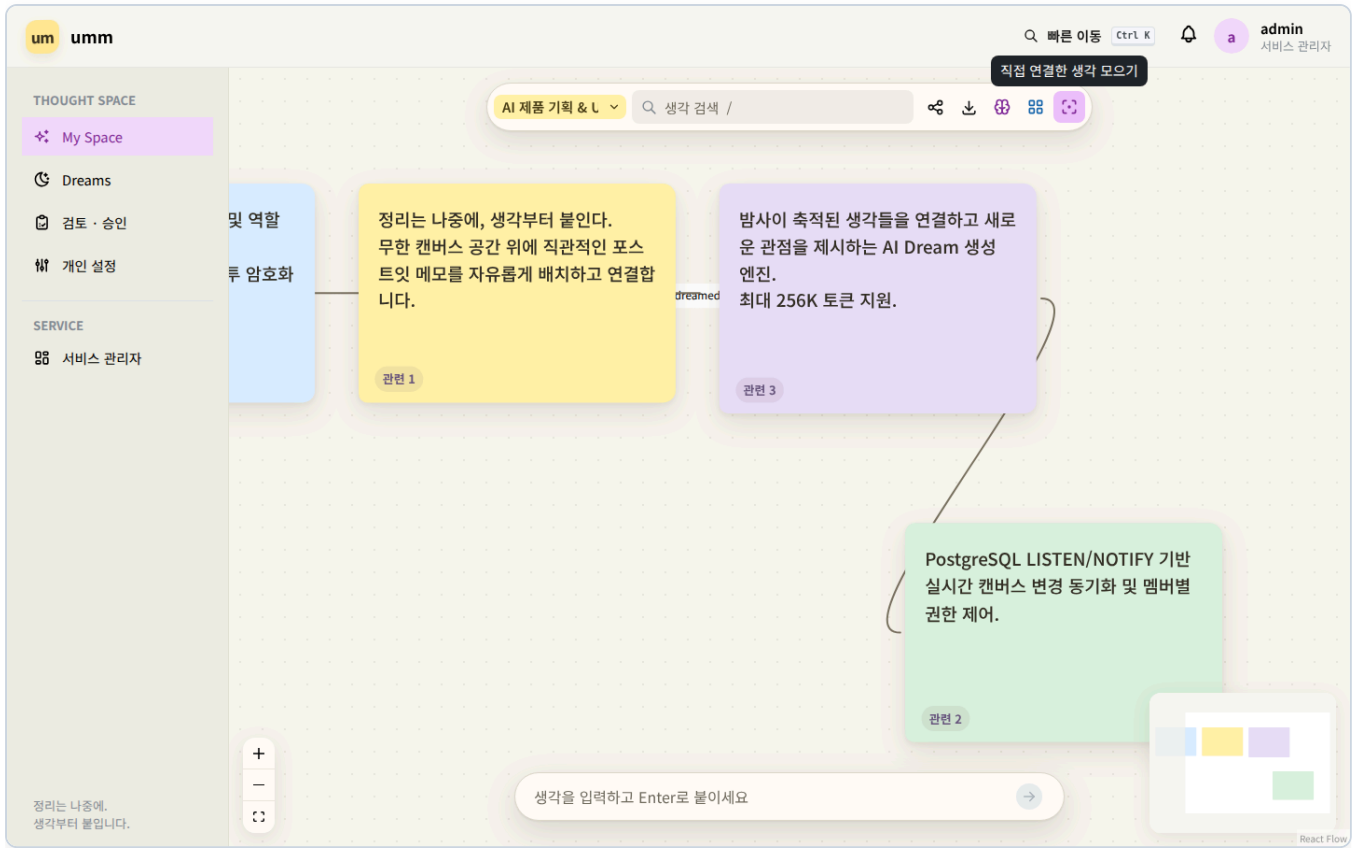
- **7종 Semantic Pastel Palette:** 노랑(Yellow), 파랑(Blue), 보라(Purple), 라벤더(Lavender), 초록(Green), 빨강(Red), 회색(Gray)
- **리사이즈 & 드래그:** 노드 경계면 리사이저로 크기를 자유롭게 조정하고 드래그하여 배치
- **위치 Undo / Redo:** **Cmd+Z** / **Shift+Cmd+Z** 단축키로 이전 위치 및 이동 이력 복원

4. 포스트잇 버전 이력 및 복원 (Revisions & Restore)



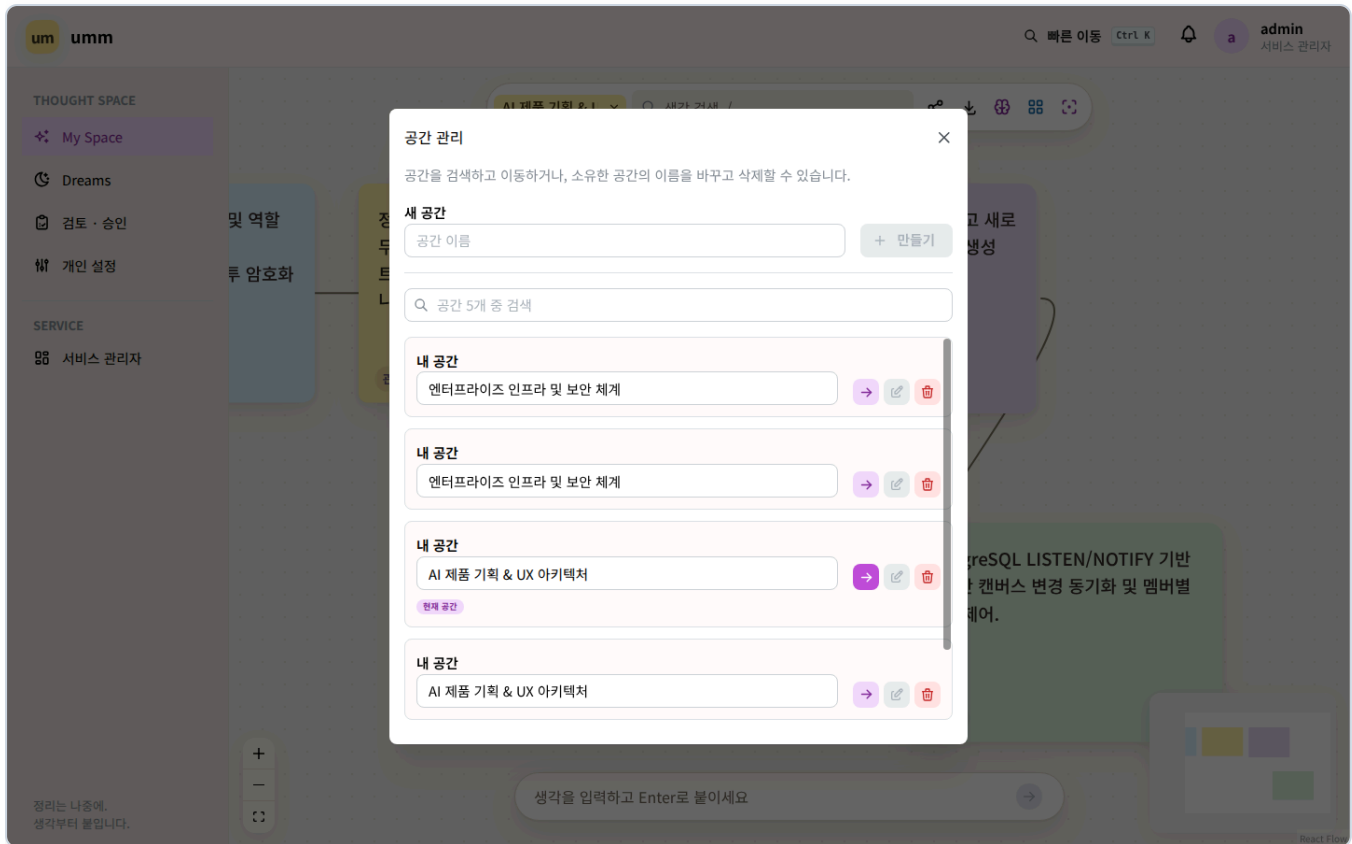
- 버전 스냅샷 자동 생성: 메모 내용 변경 시마다 백그라운드에서 버전 스냅샷 기록
- 이전 버전 롤백: 실수로 변경하거나 삭제된 내용을 특정 시점의 버전으로 안전하게 복원

5. Thought Gravity (생각 인력)



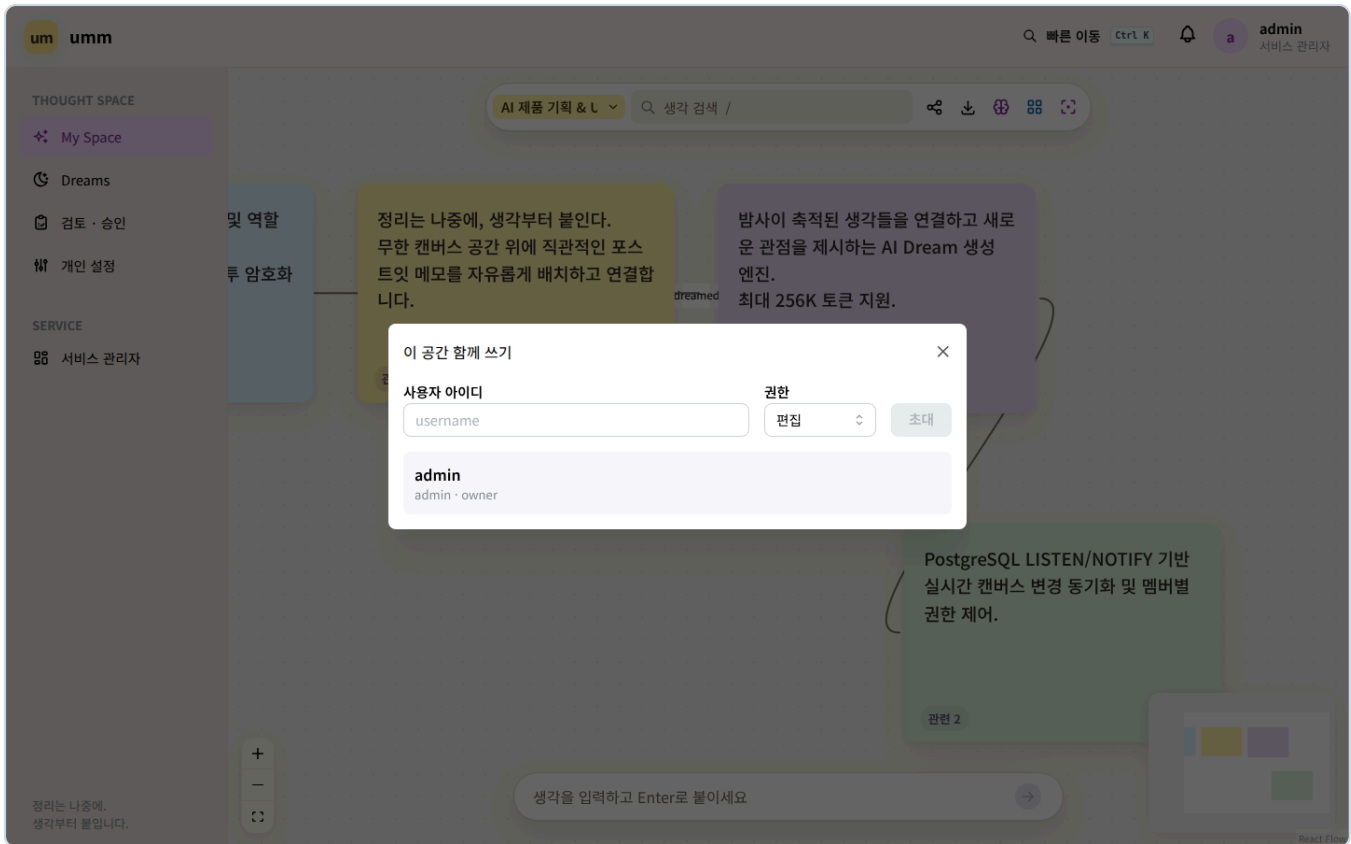
- 원형 궤도 자동 정렬: 중심 생각을 선택하고 **Thought Gravity** 버튼을 누르면 연결된 모든 관련 생각들이 중심 노드 주위 궤도로 자연스럽게 모임
- 복잡한 캔버스 정리: 흩어진 아이디어들을 한 번의 클릭으로 주제별로 집결

6. 공간 관리자 (Space Manager)



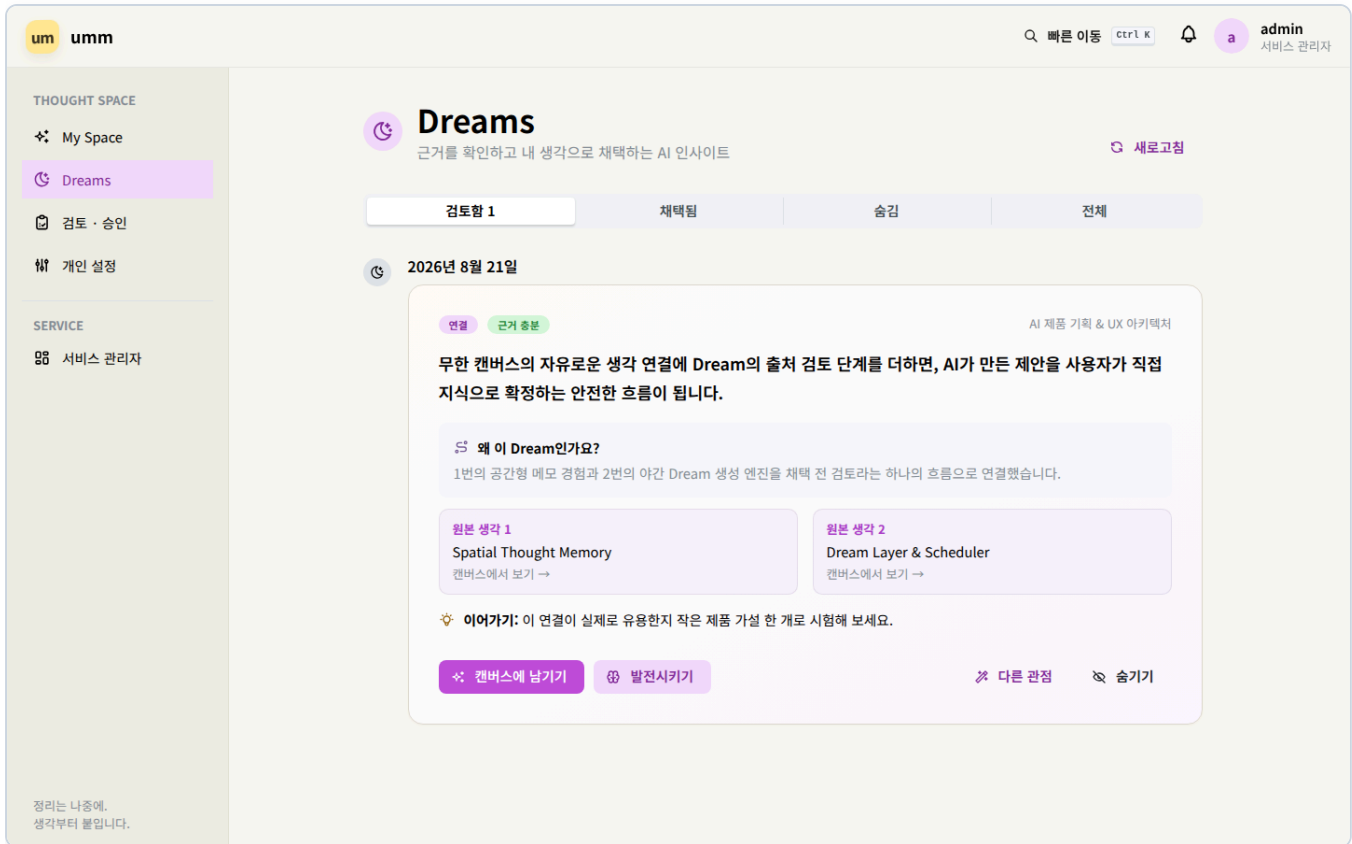
- **공간 생성 (Create Space):** 새로운 프로젝트 및 생각 주제별 독립 캔버스 생성
- **공간 검색 및 전환:** 등록된 모든 공간을 실시간 검색하고 빠르게 이동
- **공간 이름 변경 및 삭제:** 소유한 공간의 이름을 인라인으로 변경하거나 안전 확인 후 영구 삭제
- **공간별 AI 제외:** 공간 관리에서 전체 공간의 AI Dream 분석을 즉시 켜거나 끄며, 제외 공간의 본문과 해당 공간으로 범위를 제한한 검색어는 외부 임베딩 Gateway에도 전송하지 않음

7. 공간 협업 및 공유 (Space Collaboration)



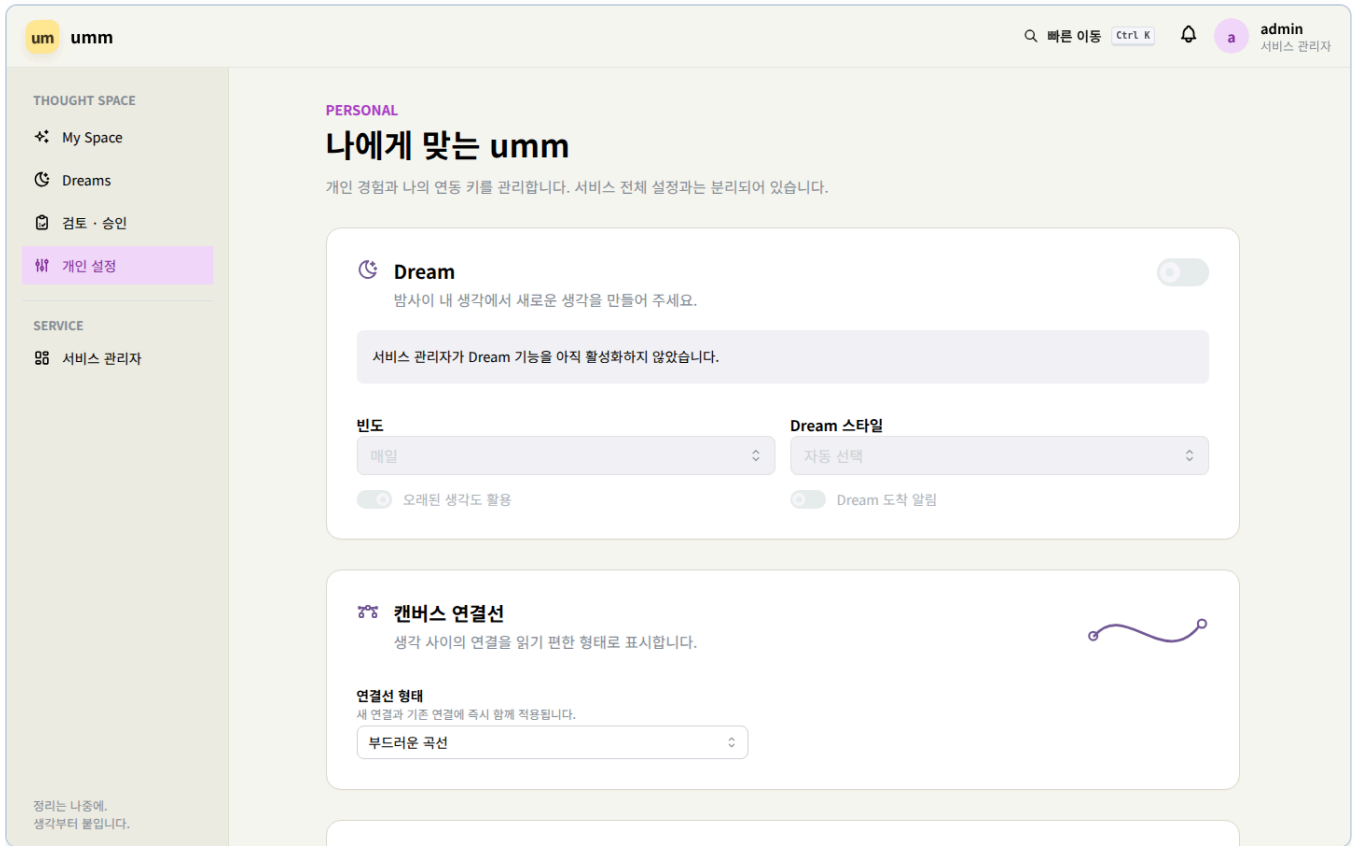
- 팀원 초대: 사내 사용자를 검색하여 공동 작업 공간으로 초대
- 세분화된 권한 제어:
 - 보기 (View) : 메모 조회 및 내보내기만 가능
 - 편집 (Edit) : 메모 생성, 수정, 연결선 조작 가능
 - 관리 (Manage) : 멤버 관리 및 공간 설정 제어
- 팀장 승인 연동: 관리자 검토 정책이 켜져 있을 경우 공간 공유 시 팀장 승인 단계 자동 삽입

8. Dream 검토함 (/dreams)



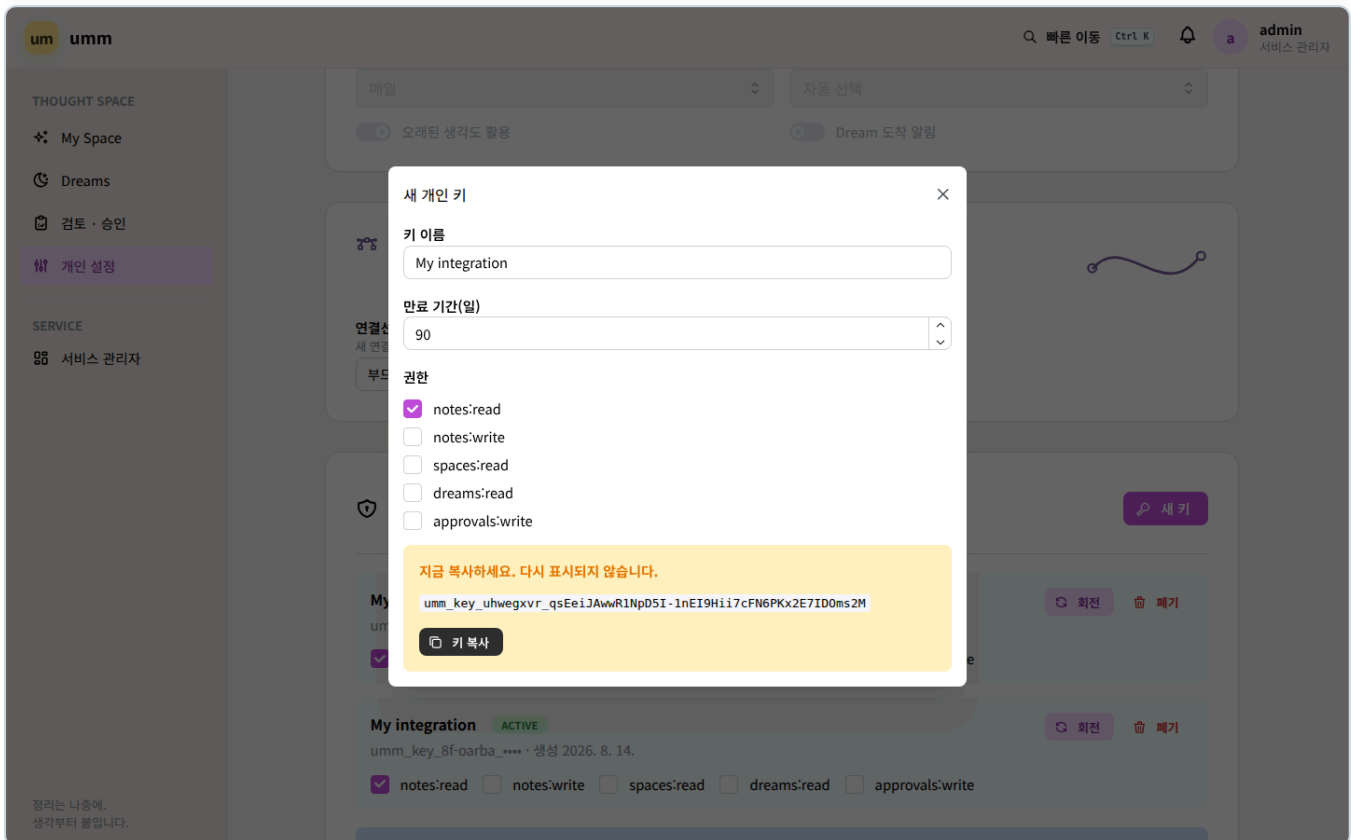
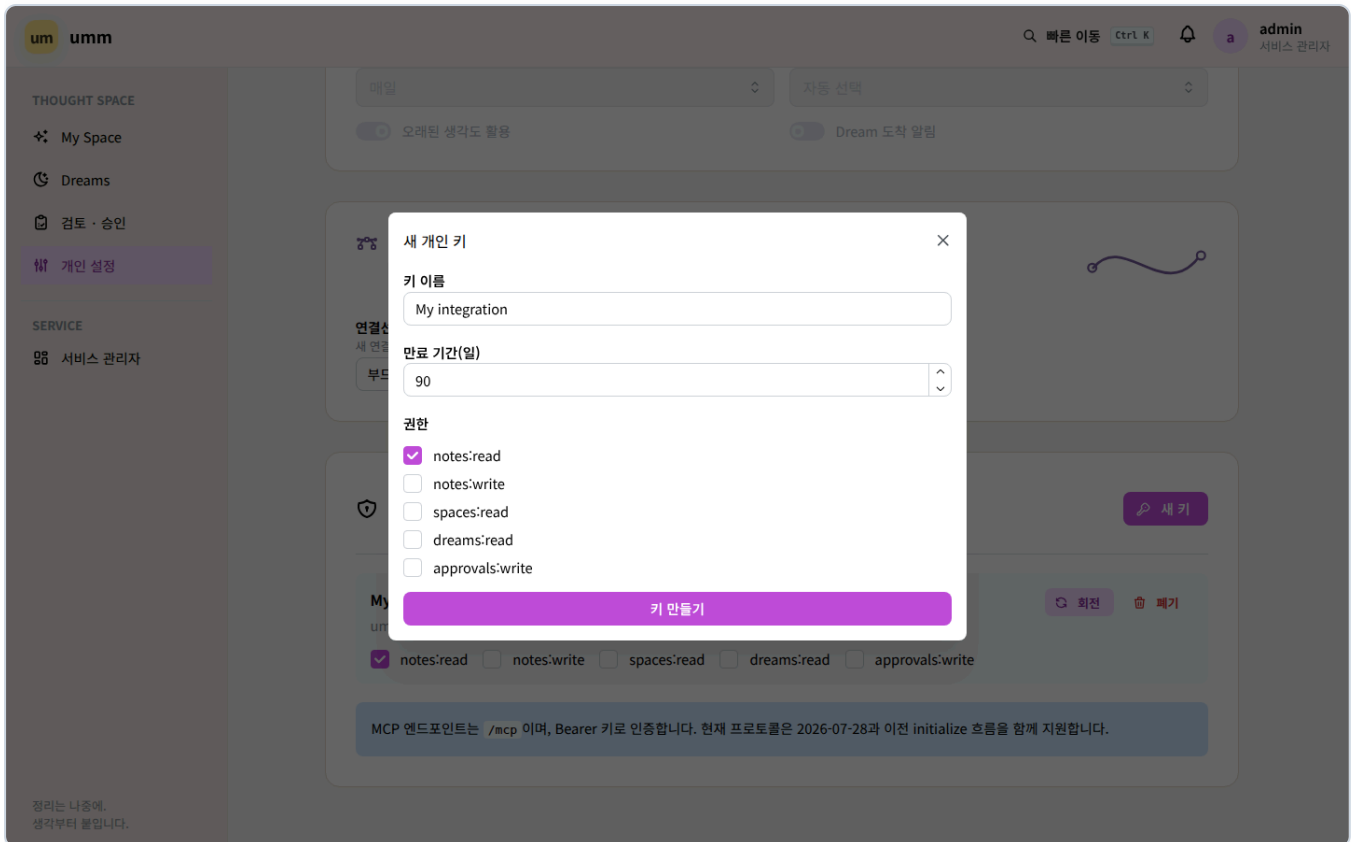
- **채택 전 후보 보관:** Dream Scheduler가 만든 관점은 검토함에 먼저 도착하며, 사용자가 채택하기 전에는 캔버스와 연결선을 변경하지 않음
- **근거와 설명:** 연결/질문/확장 등의 유형, 설명형 품질 배지, 생성 이유, 실제 원본 메모를 함께 표시
- **캔버스 안착:** **캔버스에 남기기** 를 누를 때만 라벤더 포스트잇과 원본 생각 연결선을 원자적으로 생성
- **다른 관점과 발전:** 같은 근거로 중복되지 않는 후보를 다시 만들거나 확장/반대 관점/실행 항목으로 발전하며, AI Assist와 자동 생성까지 외부 AI 호출 동안 원본·현재 공유권한을 하나의 접근 lease로 고정. 채택 후 발전 결과도 메모와 연결선을 한 트랜잭션으로 저장
- **정확한 노출·알림 이동:** 카드가 화면에 50% 이상 실제 보일 때만 노출로 집계하고, Dream 도착 알림을 누르면 해당 카드로 바로 이동
- **현재 공유 범위만 표시:** 공유 공간에서 만든 개인 Dream도 현재 owner/member 권한이 있을 때만 Today·검토함에 본문·공간명·원본을 표시하며, 공유가 회수되면 기존 Dream ID의 피드백·채택·재생성·발전을 함께 닫음
- **구체적 피드백:** 뻔함, 무관함, 사실 오류, 반복은 스타일 선호에 반영하고, 과도한 빈도 피드백은 생성 주기를 자동 완화

9. 개인화 설정 (/settings)



- **Dream 개인화 옵션:**
 - Dream 생성 빈도 (매일, 주 3회, 주 1회)
 - 스타일 선호 (자동, 연결 중심, 질문 중심, 확장 중심, 자유)
 - 오래된 생각 활용 여부 및 Dream 일시 중지 (오늘 / 3일 / 일주일)
- **연결선 형태 선택:** 부드러운 곡선(Bezier), 둥근 꺾은선(Smoothstep), 직선(Straight) 실시간 미리보기 및 변경

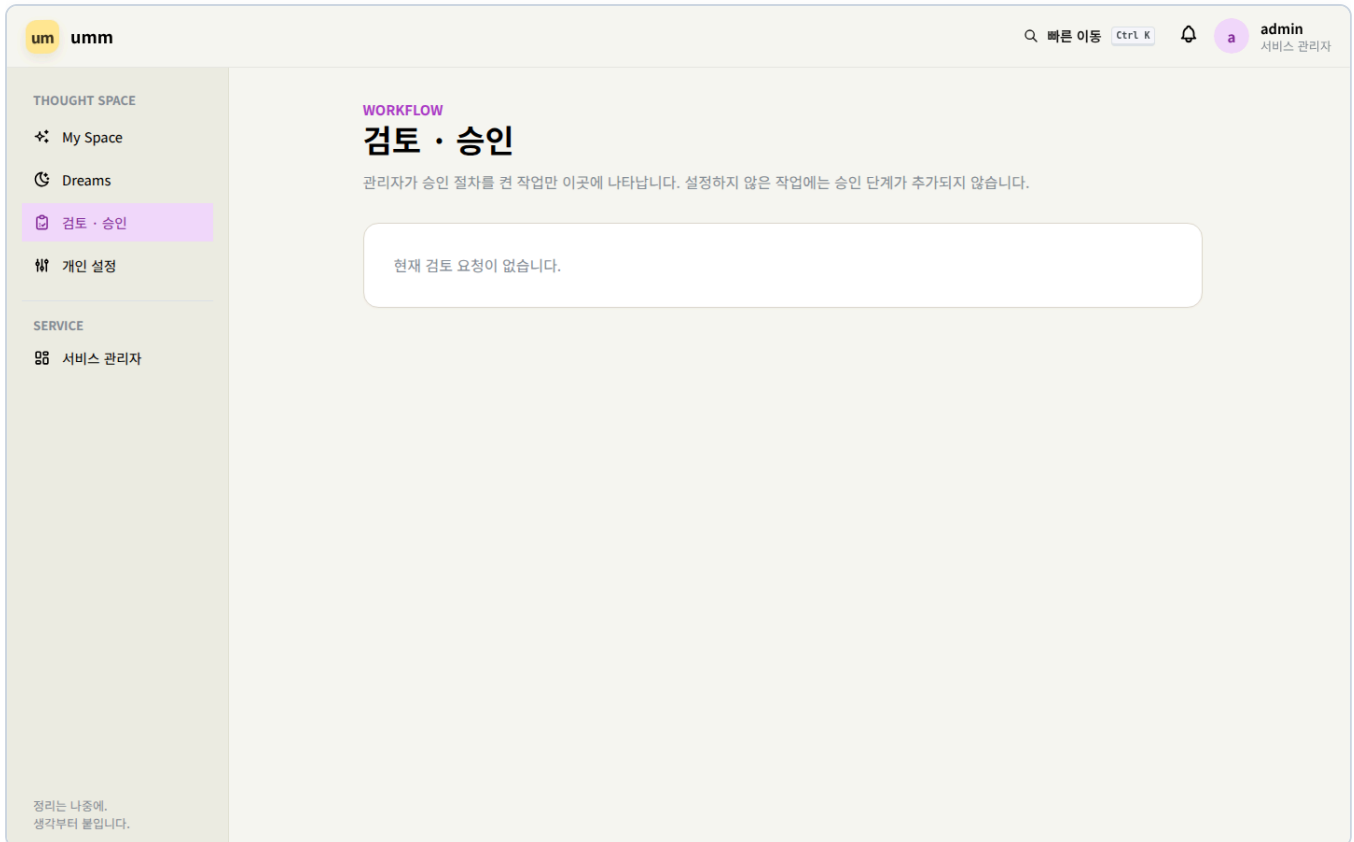
10. 개인 API · MCP 키 관리



- 최소 권한 **Scoped API Key**: 필요한 권한(`spaces:read` , `notes:read` , `notes:write` , `ai:assist` 등)만 선택하여 발급. 외부 Gateway와 AI 쿼터를 쓰는 AI Assist는 `notes:read` 와 분리된 `ai:assist` 만 허용
- 일회성 **Secret** 표시: 발급 시 1회만 노출되는 보안 토큰

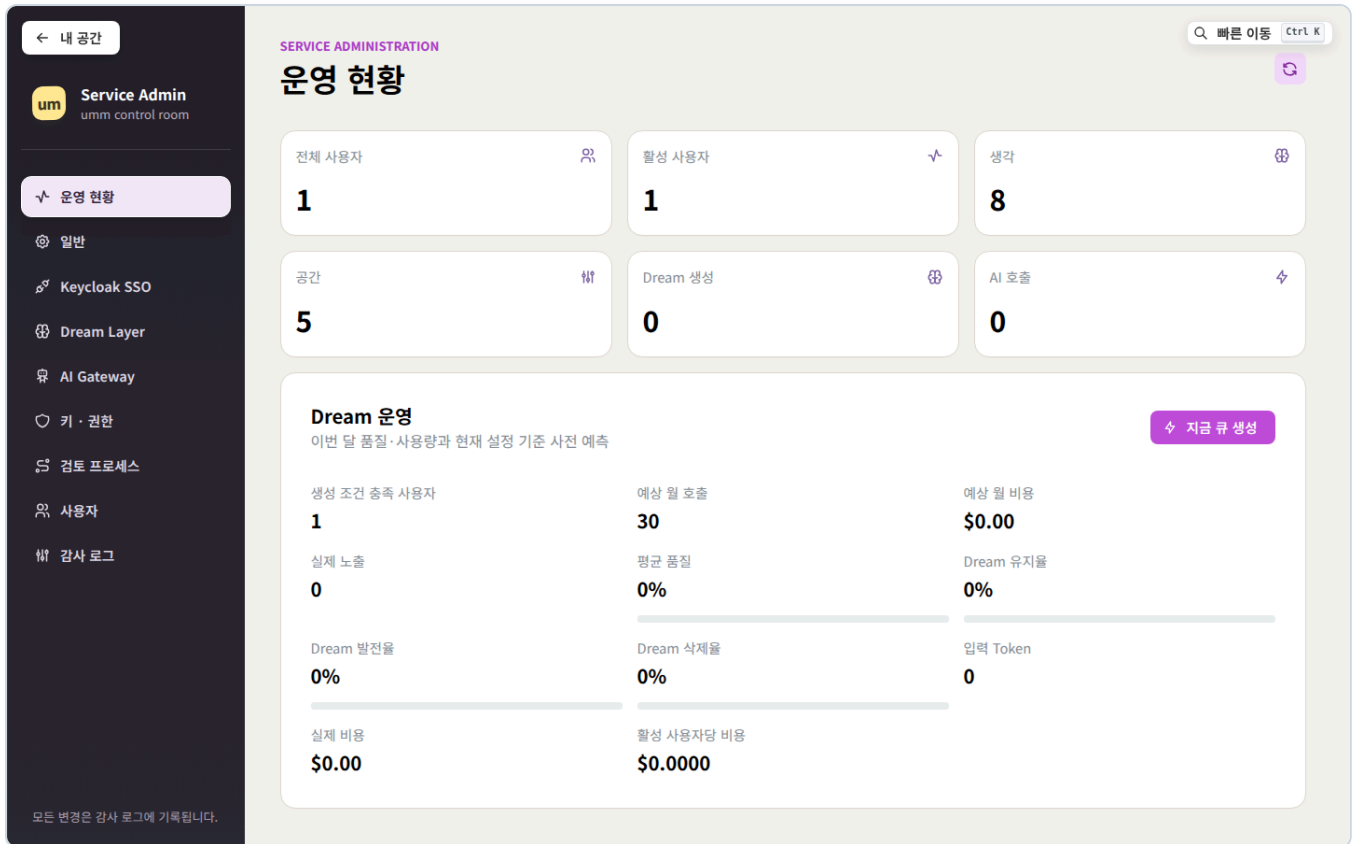
- **무중단 키 회전 (Zero-Downtime Rotation):** 설정된 중첩 시간(예: 24시간) 동안 기존 키와 신규 키를 동시 지원하여 서비스 중단 없는 안전한 키 교체. 보안 난수 생성이 실패하면 새 키나 짧은 대체 키를 저장하지 않고 기존 상태를 그대로 유지

11. 검토 및 승인 대기열 (/approvals)



- **작업별 조건부 승인:** 공간 공유 및 외부 내보내기 요청 목록 조회
- **팀장/관리자 결정:** 요청 사유 확인 후 의견(Comment)과 함께 승인(Approve) 또는 반려(Reject) 처리

12. 서비스 관리자: 운영 현황 (/admin/overview)



- **KPI 지표 요약:** 전체 사용자, 활성 사용자, 생각 수, 공간 수, Dream 생성 수, AI 호출 수
- **Dream 운영 예측 및 비용 분석:** 예상 월 호출량과 비용, 검토 수, 채택률, 유의미 활용률, 발전/숨김률, 채택 Dream당 비용
- **즉시 큐 생성 (Manual Trigger):** 야간 스케줄러 대기 없이 즉시 Dream 분석 작업 큐 실행

13. 서비스 관리자: 일반 설정 (/admin/general)

← 내 공간

um

Service Admin

umm control room

운영 현황

일반

Keycloak SSO

Dream Layer

AI Gateway

키 · 권한

검토 프로세스

사용자

감사 로그

모든 변경은 감사 로그에 기록됩니다.

SERVICE ADMINISTRATION

빠른 이동 Ctrl K

일반

서비스 기본 정보

재시작 없이 적용되며 로그인 화면과 서비스 전반에 반영됩니다.

서비스 이름

umm

공개 URL

OIDC Callback과 Origin 검증에 사용됩니다.

http://localhost:8080

세션 유지 시간

24 시간

서비스 시간대

Asia/Seoul

모든 변경사항이 저장되었습니다.

저장

- 서비스 기본 정보: 서비스 명칭, 공개 URL(Public URL), 세션 유지 시간(1~720시간), 서비스 시간대(IANA Timezone) 구성
- 무중단 즉시 반영: 서버 재시작 없이 저장 즉시 전체 서비스에 적용

14. 서비스 관리자: Keycloak SSO (/admin/oidc)

← 내 공간

um Service Admin
umm control room

운영 현황

일반

Keycloak SSO

Dream Layer

AI Gateway

키 · 권한

검토 프로세스

사용자

감사 로그

모든 변경은 감사 로그에 기록됩니다.

SERVICE ADMINISTRATION

Keycloak SSO

빠른 이동 Ctrl K

↺

Keycloak SSO · OIDC

연결 시험

Issuer URL, Client ID, Client Secret만으로 Discovery를 통해 자동 연결합니다.

Keycloak SSO 활성화

Issuer URL
https://keycloak.internal/realms/umm

Client ID

Client Secret

관리자 그룹/역할
umm-admin

팀장 그룹/역할
umm-team-lead

Keycloak Confidential Client에서 Standard Flow를 켜고 Callback을 <http://localhost:8080/api/v1/auth/oidc/callback>으로 정확히 등록하세요.

모든 변경사항이 저장되었습니다.

저장

- **OIDC Discovery 자동 연동**: Issuer URL 입력만으로 엔드포인트 자동 탐색
- **암호화 자격증명 보관**: Client Secret을 AES-256-GCM으로 암호화 저장
- **그룹 및 역할 매핑**: 사내 Keycloak 그룹/역할을 umm의 `admin`, `team_lead`, `user` 로 자동 매핑
- **원클릭 연결 시험**: 실시간으로 Keycloak 서버와의 OIDC 핸드셰이크 검증

15. 서비스 관리자: Dream Layer (/admin/dream)

- 스케줄 및 빈도 제어: 자동 생성 시간(기본 02:00), 주기(매일/평일/주말/요일별/N일 간격)
- 256K Context Window 토큰 한도 제어: 모델 사양에 맞춰 4K부터 256K까지 토큰 슬라이더 설정
- 품질 기준선(Quality Threshold) & Quiet Mode: 가치가 높은 경우에만 선별적으로 Dream을 생성하는 Quiet Mode 및 노출 최소 점수 제어

16. 서비스 관리자: AI Gateway (/admin/ai_gateway)

← 내 공간

um

Service Admin
umim control room

운영 현황

일반

Keycloak SSO

Dream Layer

AI Gateway

키 · 권한

검토 프로세스

사용자

감사 로그

모든 변경은 감사 로그에 기록됩니다.

SERVICE ADMINISTRATION

AI Gateway

빠른 이동 Ctrl K

내부 AI Gateway

OpenAI 호환 Chat Completions 엔드포인트를 사용합니다. 외부 연결 없이 내부 모델 서버를 지정할 수 있습니다.

Base URL

http://llm-gateway.internal:8000

API Key

Timeout

긴 추론 모델은 충분한 시간을 지정하세요.

45 초

재시도

2

입력 \$ / 1M token

0

출력 \$ / 1M token

0

AI 로그 보존

90 일

Prompt Version

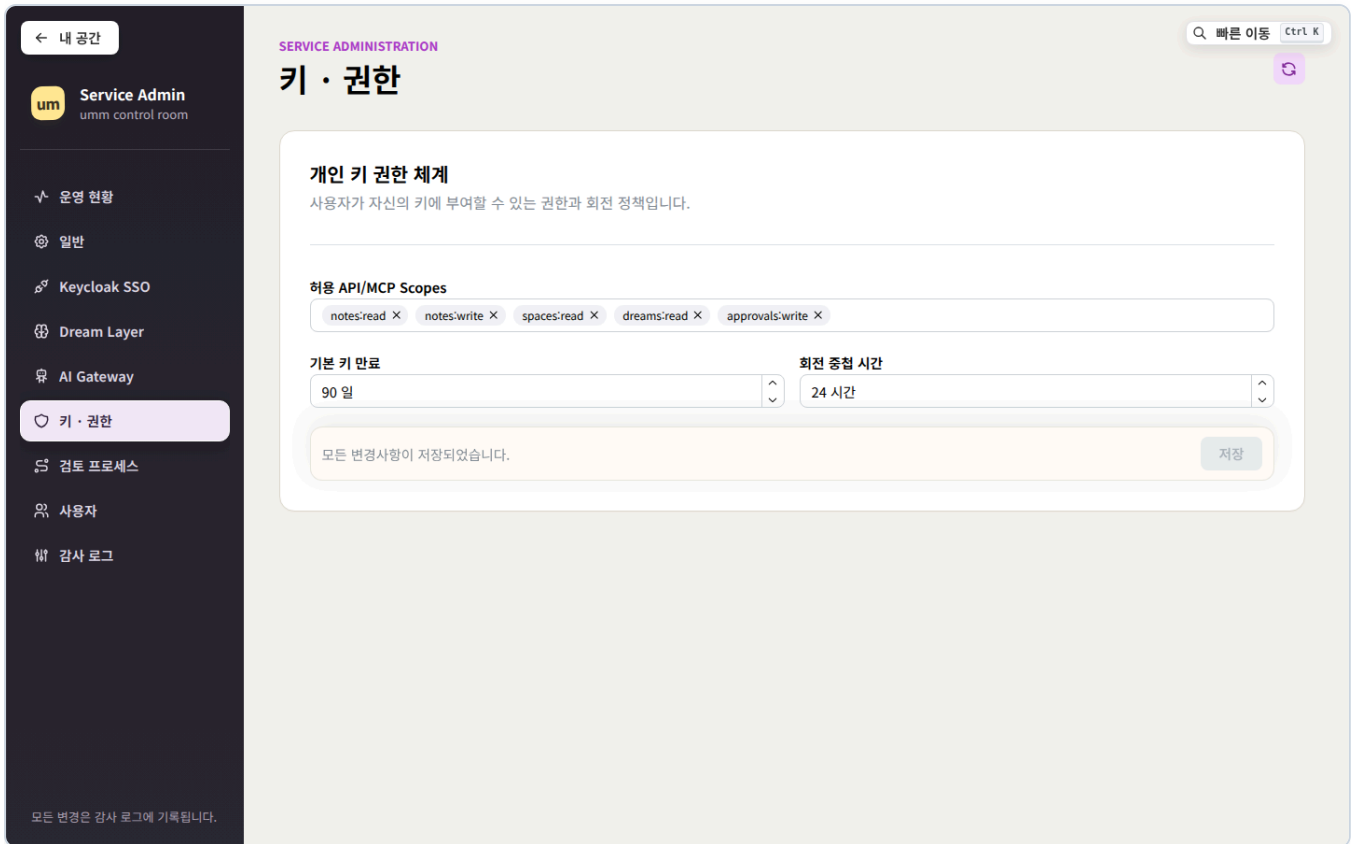
dream-v1

모든 변경사항이 저장되었습니다.

저장

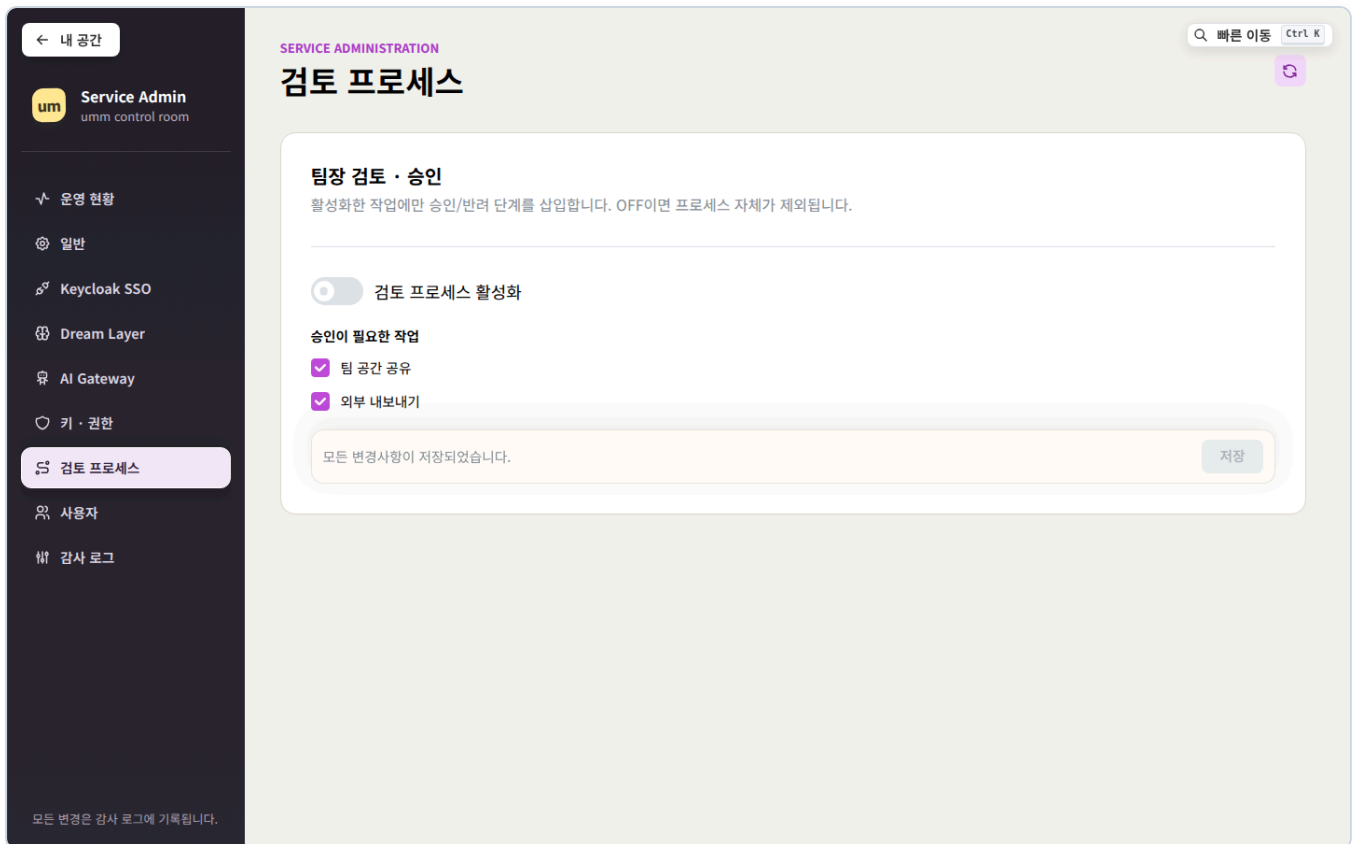
- 내부 OpenAI 호환 Gateway 연동: Base URL, API Key, Timeout(5~1800초), 재시도 횟수 지정
- 비용 산정 및 데이터 보존: 토큰당 단가(\$/1M tokens), AI 로그 보존 주기(기본 90일), 원문 프롬프트 암호화 저장 옵션

17. 서비스 관리자: 키 · 권한 체계 (/admin/security)



- **허용 API/MCP Scopes 제어**: 일반 사용자가 발급 가능한 권한 태그 화이트리스트 관리
- **키 수명 주기 정책**: 기본 키 유효기간(일) 및 회전 시 중첩 시간(시간) 설정
- **롤링 업그레이드 설정 보존**: 구버전 관리자 화면이 새 로그인·API·AI 한도 필드를 생략해도 최신 저장값에서 생략 항목만 원자적으로 병합하며, 명시한 `0` 은 유지하고 `null` 은 거부

18. 서비스 관리자: 검토 프로세스 (</admin/workflow>)



- 작업별 승인 프로세스 On/Off: 팀 공간 공유, 외부 내보내기 중 승인이 필요한 작업 선택

19. 서비스 관리자: 사용자 관리 (/admin/users)

← 내 공간

um Service Admin
umm control room

운영 현황

일반

Keycloak SSO

Dream Layer

AI Gateway

키 · 권한

검토 프로세스

사용자

감사 로그

모든 변경은 감사 로그에 기록됩니다.

SERVICE ADMINISTRATION

사용자

Keycloak 그룹 매핑 또는 여기에서 역할과 팀을 변경할 수 있습니다.

사용자	역할	팀	상태
admin admin ·	관리자	팀 없음	☒

- 사내 계정 통합 관리: 사용자별 역할(`user` , `team_lead` , `admin`), 소속 팀명, 계정 활성화/비활성 스위치 제어

20. 서비스 관리자: 불변 감사 로그 (/admin/audit)

← 내 공간

um Service Admin
umim control room

운영 현황

일반

Keycloak SSO

Dream Layer

AI Gateway

키 · 권한

검토 프로세스

사용자

감사 로그

모든 변경은 감사 로그에 기록됩니다.

SERVICE ADMINISTRATION

감사 로그

빠른 이동

Ctrl K

시각	행위자	작업	대상
2026. 8. 14. 오후 7:44:29	admin	API_KEY.CREATE	api_key · 04408cc1-0fd1-4d9b-8af9-551bad889c72
2026. 8. 14. 오후 7:44:13	admin	NOTE.CREATE	note · fdec7208-81b4-4c9e-b72c-a093ff5eb735
2026. 8. 14. 오후 7:44:13	admin	NOTE.CREATE	note · fdd2a826-122c-4a5f-ab3b-fc8953b50992
2026. 8. 14. 오후 7:44:13	admin	NOTE.CREATE	note · 8ff0db4e-dc4a-4b3c-807a-d3ce262823a7
2026. 8. 14. 오후 7:44:13	admin	NOTE.CREATE	note · 0daf2b40-3273-4c43-9c42-4617ca3ef6e4
2026. 8. 14. 오후 7:44:13	admin	SPACE.CREATE	space · 292fed54-8eb5-4116-8cb5-de47c4c1df7e
2026. 8. 14. 오후 7:44:13	admin	SPACE.CREATE	space · 394ba99f-5488-488d-b4a5-6f0590779d90
2026. 8. 14. 오후 7:44:12	admin	AUTH.LOCAL.LOGIN	user · 4dcba2b5-f79b-4f3d-9cf4-f775a624a74f
2026. 8. 14. 오후 7:43:43	admin	API_KEY.CREATE	api_key · 78d66fdd-cdd9-469e-b76e-9df12c4605b2
2026. 8. 14. 오후 7:43:35	admin	NOTE.CREATE	note · 13ef50f1-c223-448e-ac25-3552aab24efd
2026. 8. 14. 오후 7:43:35	admin	NOTE.CREATE	note · 0be07091-9fa6-4a4d-ae44-acedef347b29
2026. 8. 14. 오후 7:43:35	admin	NOTE.CREATE	note · 18eea206-8f57-45be-832a-273a02e3e316
2026. 8. 14. 오후 7:43:35	admin	NOTE.CREATE	note · c86028b5-f369-415e-9154-3a16e0b40295
2026. 8. 14. 오후 7:43:35	admin	SPACE.CREATE	space · 49311534-7d0b-44bc-9f67-11be6ee6fd8f

- 추가 전용(Append-only) 감사 추적: 시각, 행위자, 작업 구분(Action), 대상 리소스(Resource ID)를 위변조 없이 기록

21. 오늘의 리뷰 (/today)

- 집중 검토함: 다시 볼 날짜가 된 생각, 오래 검토하지 않은 생각, 연결되지 않은 생각, 대기 중인 Dream, 최근 댓글을 한 화면에 모읍니다.
- 맥락형 온보딩: 공간 생성·생각 작성·연결·Dream 확인 단계가 실제 사용 상태에 따라 자동 완료됩니다.
- 사용자별 다시 보기·고정: 공유 생각에서도 서로의 상태를 덮어쓰지 않고 검토 완료, 1~365일 미루기, 중요 항목 고정을 관리합니다.
- 독립적인 활동 요약 설정: 개인 설정에서 "리뷰 요약에 활동 포함"을 꺼도 최근 협업 활동만 숨기며, 고정·기한 도래·오래된 생각의 검토 카드는 그대로 유지합니다.
- 권한·삭제 경계: 현재 접근 가능한 공간의 삭제되지 않은 생각에 속한 활동만 표시합니다.

21-1. 결정 기록 (/decisions)

- 표시한 것만 시간순으로: 갈래 채택·접어 둠, 질문에 답 표시, 상충 표시 네 가지. 활동량을 읽고 전환점이라 부르지 않습니다.
- 이유가 결정 옆에: 여섯 달 뒤의 질문은 "무엇을 적었나"가 아니라 "어떻게 하기로 했고 왜인가"이고, 왜가 가장 먼저 사라집니다.
- 진행 중인 갈래는 없습니다: 정해진 것이 없고, 넣으면 결정 기록이 할 일 목록이 됩니다.
- 비어 있을 때: "표시된 것이 없다는 뜻이지, 아무 일도 없었다는 뜻은 아닙니다."

21-2. 생각의 갈래 (캔버스 메모 메뉴 → 연결과 갈래)

- 진행 중 · 채택 · 접어 둔 세 상태. 정리할 때 이유가 필수이며 스키마 CHECK와 API 양쪽에서 막습니다.
- 접어 둔 갈래는 어디서나 표시됩니다: 캔버스 배지, 검색 결과, `/ai/ask` 근거, 조력자 조회, 마크다운 내보내기, MCP `note_lines`.
- 거절한 결정을 다시 쓰면 알려 줍니다: 근접 중복 한쪽이 접어 둔 갈래면 오늘의 리뷰가 이유와 함께 먼저 보여 주고, 이 쌍에는 합치기 버튼을 주지 않습니다 — 합치면 먼저 적은 쪽(접어 둔 쪽)이 남기 때문입니다.
- 추론하지 않습니다: 조용한 갈래를 접힌 것으로 처리하지 않습니다. 침묵은 결정이 아닙니다.

21-3. 초점 렌즈 (캔버스 툴바)

- 갈래별로, 또는 선택한 생각에서 연결 1~3걸음으로 캔버스를 줍힙니다.
- 흐리게 두되 숨기지 않고, 몇 개를 흐리게 뒀는지 항상 말합니다. 사라진 생각은 지워진 생각으로 읽힙니다.
- 기준이 사람이 그은 구조(갈래·연결)라서 임베딩 성능에 좌우되지 않습니다.

21-4. 살펴보고 답하는 조력자 (오늘의 리뷰)

- 한 번 검색해 답하는 대신 찾아보고 → 읽고 → 다시 찾아본 뒤 답하고, 조회 과정을 순서대로 보여 줍니다.
- 읽기 전용이 구조로 보장됩니다: 쓰기 도구가 애초에 바인딩되지 않습니다.
- 한 번 실행은 조회 6회까지. 한도에 닿으면 도구 없이 한 번 더 물어 지금까지 확인한 것을 남기고 잘렸다고 표시합니다.
- 한 턴은 요청한 사람의 일일 AI 할당량에서 차감됩니다.

22. 협업·검색·오프라인 신뢰성

- 하이브리드 검색: 접근 가능한 전체 메모와 긴 본문 뒤쪽의 키워드 일치를 보존하면서 로컬 의미 유사도를 결합하고 공간·종류·수정 시각 필터, 점수와 추천 이유를 제공합니다.
- 백링크·댓글·멘션: 실제 연결선 방향을 확인하고 포스트잇 안에서 댓글을 남기며 `@username` 으로 공간 멤버를 알립니다. 댓글 작성 transaction은 현재 멤버십을 끝까지 잠급니다. 해결·재개·삭제도 댓글·생각·공간과 현재 membership 권한을 commit까지 잠가 멤버 제거나 편집 권한 강등이 동시에 시작돼도 이전 권한으로 상태·이벤트를 확정하지 않습니다. 작성자도 현재 공간 접근권한이 있을 때만 자신의 댓글을 삭제할 수 있습니다. 생각이 삭제 되면 보관한 댓글 ID로 해결·재개·삭제 이벤트를 만들 수 없고 `comment-mutation-forbidden` 으로 종료하며 기존 댓글·멘션 알림도 목록과 unread count에서 숨깁니다.
- 오프라인 큐: 연결이 끊기면 변경을 장치에 보관하고 다시 연결될 때 먹등 키로 재전송하며, 영구 거부된 요청은 사유를 알리고 그 항목만 제거한 뒤 다음 변경을 계속 처리합니다. 권한 회수 뒤의 댓글 해결·삭제도 terminal type으로 구분해 독립적인 후속 변경을 막지 않습니다. 업그레이드 도중 구버전 탭이 레거시 큐에 늦게 기록한 변경도 기존 계정 큐와 잠금 아래 병합한 뒤 이전 키를 정리하므로 사라지지 않습니다. 저장 공간·사이트 권한 오류가 발생 하면 durable queue 성공으로 오인하지 않고 즉시 복구 안내를 표시하며, 저장되지 않은 Canvas 상태를 마지막 durable snapshot으로 되돌립니다. 저장소 접근이 거부되어도 앱 초기화·Canvas 기본 설정과 queue 상태 렌더링을 유지하고 읽을 수 없는 기존 항목을 덮어쓰지 않습니다.
- 충돌 병합: 다른 위치에서 먼저 저장한 버전이 있으면 내 내용과 서버 내용을 나란히 비교해 선택·병합합니다.
- 접근 가능한 대체 보기: 키보드 화살표 이동, 명확한 focus ring, 최소 터치 영역, reduced-motion, 선형 생각 목록을 지원합니다.

23. 개인 서명 웹훅 (/settings)

- `note.*`, `comment.*`, `member.*`, `dream.accepted` 등 원하는 이벤트를 HTTPS endpoint로 전달합니다.
- HMAC-SHA256 signing secret은 생성·회전 직후 한 번만 표시합니다.
- 도메인 변경과 PostgreSQL outbox를 원자적으로 커밋해 재시작 뒤에도 이어 보내며, 사설·예약 IP SSRF 차단, 세 번 재시도, 연속 10회 실패 시 자동 중지를 적용합니다. 다국어 오류는 유효한 UTF-8 rune 경계에서 제한해 실패 상태와 자동 중지 횟수를 빠짐없이 기록합니다.
- at-least-once 전달 시도에 대비해 수신 시스템은 delivery UUID를 멍등 처리합니다.
- terminal payload는 즉시 제거하고 delivery metadata는 30일 뒤 정리합니다.

24. 관리자 AI 평가·키 회전·관측성

- **AI 평가 세트**: 입력 생각, 기대 단어, 금지 단어로 Dream grounding·품질 회귀를 실행하고 점수·모델·prompt version을 기록합니다.
- **Master-key 회전**: primary/fallback keyring 상태와 회전 대기·읽기 실패를 확인하고 저장된 암호문을 트랜잭션으로 다시 암호화합니다. `enc:` wrapper 이전에 저장된 raw v1 AI prompt도 현재 primary key의 wrapper·v2 형식으로 회전한 뒤에만 pending이 0이 됩니다. OIDC·AI Gateway의 마스킹 설정 저장은 회전과 같은 설정 lock 뒤 최신 secret을 병합해 이전 화면의 동시 저장에도 새 암호문을 보존합니다.
- **운영 지표**: 댓글·온보딩·웹훅 실패·AI eval 결과와 HTTP request/latency를 관리 화면 및 Prometheus에서 확인합니다. Prometheus endpoint는 관리자 브라우저 세션 또는 명시적인 `metrics:read` API 키만 허용하며 일반 사용자 세션과 다른 scope 키는 거부합니다.

25. 언어 · 테마 · 가져오기 (v0.8.0)

- **한국어 / English**: 프로필 메뉴에서 언어를 바꾸면 화면 전체가 즉시 전환됩니다. 선택은 계정에 저장되어 다른 브라우저에서도 이어집니다. 언어·테마 부분 저장이 동시에 겹쳐도 각 선택을 원자적으로 합쳐 서로 되돌리지 않습니다. 번역이 없는 문구는 한국어 원문으로 표시되므로 화면이 비어 보이는 일이 없습니다.
- **밝게 / 어둡게 / 시스템 설정**: 로그인하면 계정의 테마를 현재 탭의 활성 theme provider에 즉시 적용하고, 시스템 설정은 자동 모드로 복원합니다. 첫 페인트 전에 테마가 결정되어 어두운 화면으로 들어올 때 흰 화면이 번쩍이지 않으며, 포스트잇은 두 테마에서 모두 종이색을 유지합니다.
- **마크다운 가져오기**: 내보내기 메뉴에서 마크다운 파일이나 붙여넣은 텍스트를 `#` 제목·`---` 구분선·빈 줄 기준으로 나눠 각각 하나의 생각으로 캔버스에 붙입니다(남의 마크다운은 한 번에 최대 200개). 200개를 넘으면 실제 개수를 안내하고 가져오기를 시작하지 않으므로 201번째 이후 원문도 편집기에 그대로 남습니다. 부분 실패 시 성공 항목은 유지하고 실패 항목만 편집기에 남겨 다시 시도합니다.
- **자기 백업으로 공간 되돌리기**: umm이 내보낸 마크다운은 글자만이 아니라 **공간 그대로** 돌아옵니다 — 캔버스에서의 자리, 생각 사이의 연결, 갈래와 그 갈래가 끝난 이유, 질문·아이디어 구분, 메모 색까지. 파일 첫머리의 `Exported from umm at ...` 줄로 자기 파일임을 알아보므로 **내보낸 파일은 나누지 마세요**: 그 줄이 없는 조각은 평범한 마크다운으로 읽혀 자리와 연결이 사라집니다. 200개 상한도 자기 파일에는 적용하지 않습니다. 일부가 실패해 다시 시도할 때도 초안을 내보내기 형태로 되돌려 두므로, 두 번째 시도에서 자리와 연결을 잃지 않습니다.
- **모바일 공유 타킷과 안전한 앱 셀**: 설치형 PWA로 추가하면 다른 앱의 공유 시트에 umm이 나타나고, 공유한 내용이 빠른 입력창에 채워집니다. 서비스 워커는 HTML 탐색 응답만 오프라인 앱 셀로 갱신해 manifest·asset JSON·아이콘을 직접 열어도 다음 오프라인 화면이 해당 정적 파일로 바뀌지 않습니다. Content hash가 붙은 bundle만 장기 immutable로 캐시하고 고정 manifest·service worker·아이콘은 재검증해 새 릴리스의 바로가기와 앱 메타데이터를 오래된 값으로 고정하지 않습니다.

- **키보드 편집:** 캔버스에서 **Tab** 으로 생각 사이를 이동하고 **Enter** 로 편집을 시작하며 **Esc** 로 빠져나옵니다.

26. 규모와 남용 방지 (v0.8.0)

- **푸시 기반 실시간 협업:** 공간을 열어 둔 사람마다 초당 폴링하던 구조를 PostgreSQL **LISTEN/NOTIFY** 팬아웃으로 바꿨습니다. 유휴 상태의 데이터베이스 비용이 0이고, 변경은 즉시 도착합니다. 상한 1~2의 작은 pool에서는 모든 연결을 request에 남기도록 전용 listener를 끄고 1초 안전 폴링으로 자동 전환하며, 상한 3부터 두 request 연결을 남긴 채 listener를 시작합니다. 실행 중 listener가 끊기거나 복구되면 상태 전환이 열린 SSE를 즉시 깨워 긴 safety deadline을 기다리지 않고 폴링 주기를 다시 맞춥니다.
- **인덱스를 타는 검색:** 키워드 조건을 **pg_trgm** GIN 인덱스가 사용될 수 있는 형태로 다시 작성했습니다.
- **선택적 게이트웨이 임베딩:** 관리자가 임베딩 모델을 지정하면 의미 검색 품질이 올라가고, 비워 두면 지금까지처럼 완전 오프라인으로 동작합니다. 모델명과 Gateway endpoint fingerprint를 함께 벡터 공간으로 식별해 같은 모델명을 쓰는 제공자로 주소를 바꿔도 기존 벡터를 점진적으로 다시 만듭니다. 설정 변경 전에 시작한 원격 응답과 장애 뒤의 전체 로컬 정규화 pass는 새 Gateway 벡터를 덮어쓰지 못합니다. 외부 임베딩 직전 대상 note.space와 Gateway 설정을 다시 잠그고 응답 저장까지 유지하며, 공간 범위 검색은 membership과 활성 note도 검색 완료까지 고정합니다. AI 제외·접근 회수가 먼저 적용되면 본문과 검색어는 로컬에서만 처리하고, 호출이 먼저 시작되면 정책 변경이 종료까지 기다립니다. 메모 단위 제외가 섞인 공간도 일반 메모를 포함해 Canvas와 검색을 하나의 로컬 비교 공간으로 유지합니다. 암호화된 API key를 복호화할 수 없으면 ciphertext를 Gateway에 보내지 않고 로컬로 fail closed 합니다. 원격 장애나 한 batch 안의 vector 차원 불일치 시에는 비교 집합 전체를 한 로컬 벡터 공간으로 맞추고 5분 회로 차단으로 반복 timeout을 피합니다. 임베딩과 Dream Gateway의 오류 본문 일부는 다국어 rune를 쪼개지 않고 제한해 관리자 오류 응답과 로그가 항상 유효한 UTF-8을 유지합니다.
- **로그인 잠금과 요청 한도:** 인스턴스 간에 공유되는 로그인 잠금, 호출자별 API 한도, AI 생성의 분당·하루 한도를 관리자 화면에서 조정합니다. 같은 주소·계정의 비밀번호 검증과 실패 기록을 하나의 PostgreSQL 임계 구역에서 처리해 병렬 요청도 설정한 추측 횟수를 넘지 않습니다.
- **로그인한 기기:** 개인 설정에서 다른 로그인을 확인하고 즉시 종료할 수 있습니다.
- **응답별 CSP nonce:** 스크립트 실행을 응답마다 새로 만드는 nonce로 고정해, 주입된 스크립트가 같은 출처를 가리켜도 실행되지 않습니다.

umm 사용자 실무 가이드 (User Guide)

umm은 복잡한 폴더 트리나 문서 서식에 얽매이지 않고, 머릿속 생각을 자유로운 2차원 공간 위에 직관적으로 펼치고 발전시킬 수 있도록 돕는 서비스입니다.

1. 공간(Space) 관리

- **공간 개념:** 프로젝트, 아이디어 주제, 업무 영역별로 독립된 무한 캔버스입니다.
- **공간 전환 & 검색:**
 - 상단 툴바의 공간 이름 버튼(예: AI 제품 기획 & UX 아키텍처)을 클릭하여 공간 목록을 열람합니다.
 - 검색창에 키워드를 입력하여 원하는 공간으로 빠르게 이동합니다.
- **새 공간 만들기:**
 - 공간 드롭다운 하단의 새 공간 만들기 또는 공간 관리 를 선택합니다.
 - 공간 이름을 입력하고 Enter 또는 만들기 버튼을 누릅니다.
- **공간 팀원 공유:**
 - 상단 툴바의 공간 공유 (IconShare) 버튼을 클릭합니다.
 - 팀원의 아이디를 입력하고 권한(보기, 편집, 관리)을 선택한 후 초대 합니다.

2. 생각 포스트잇 작성 및 편집

포스트잇 생성 4가지 방법

1. **캔버스 빈 공간 더블클릭:** 마우스 커서가 위치한 지점에 빈 포스트잇이 즉시 생성됩니다.
2. **단축키 N:** 포스트잇 입력 필드에 포커스되어 타이핑 즉시 새 생각이 붙습니다.
3. **단축키 /:** 상단 생각 검색 필드로 즉시 이동합니다.
4. **하단 Quick Capture 바:** 화면 하단 캡처 바에 생각을 입력하고 Enter를 누르면 화면 중앙에 즉시 배치됩니다.

포스트잇 조작 및 색상

- **내용 입력 & 자동 저장:** 메모장을 클릭하여 자유롭게 내용을 작성하면, 별도의 저장 버튼 없이 수백 밀리초 내에 자동 저장됩니다.
- **7가지 시맨틱 색상 변경:**
 - 메모 우측 상단의 점 세 개 (IconDots) 메뉴 → 색상 선택
 - 노랑 (Yellow), 파랑 (Blue), 보라 (Purple), 라벤더 (Lavender), 초록 (Green), 빨강 (Red), 회색 (Gray) 중 선택
- **크기 조절 & 회전:** 메모를 클릭하면 모서리에 핸들이 표시되며, 드래그하여 자유롭게 크기를 늘리거나 줄일 수 있습니다.

- **메모 삭제:** 메모 메뉴에서 **지우기**를 선택하거나, 노드를 선택한 상태에서 **Delete** / **Backspace**를 누릅니다.

🕒 메모 버전 이력 및 복원

- 메모 메뉴에서 **이전 버전 복원**을 클릭하면, 변경되기 전 과거 시점의 스냅샷으로 안전하게 롤백할 수 있습니다.

3. 생각 연결 및 지능형 배치

🔗 생각 연결선 (Edges)

- 메모 좌우의 동그란 연결 핸들을 마우스로 드래그하여 다른 메모의 핸들에 연결합니다.
- 연결선은 부드러운 곡선(Bezier), 둥근 꺾은선(Smoothstep), 직선(Straight) 형태로 표시됩니다.

🌌 Thought Gravity (생각 인력)

- 중심이 되는 메모를 하나 선택한 뒤 상단 툴바의 **Thought Gravity (IconFocusCentered)**를 클릭합니다.
- 선택한 메모와 직간접적으로 연결된 모든 생각들이 중심 메모를 기준으로 원형 궤도를 그리며 정렬됩니다.

💡 의미 기반 연관 생각 (Related Thoughts)

- 포스트잇 하단에 **관련 N** 배지가 표시되면 이를 클릭합니다.
- AI Gateway 연결 없이도 내부 의미 유사도 엔진이 연관된 생각들을 찾아 유사도 점수(%) 및 추천 사유와 함께 주변에 배치합니다.

4. AI 생각 발전 도구 (AI Assist)

캔버스 상단 툴바의 **AI 생각 도구 (IconBrain)**를 클릭하면 선택된 메모들을 기반으로 5가지 발전 기능을 실행할 수 있습니다:

도구	설명	활용 예시
요약	흩어진 메모들의 핵심 결론을 명확하게 한 문장으로 압축	긴 브레인스토밍 메모 정리
질문 만들기	현재 생각에서 놓치고 있는 핵심 검토 질문 도출	기획 누락 사항 점검
확장	생각의 범위를 넓혀 새로운 적용 분야와 아이디어 제안	비즈니스 모델 확장
반대 관점	반론과 리스크, 비판적 시각을 제시	보안 및 규제 리스크 사전 분석
실행 항목	아이디어를 당장 실행 가능한 Action Item(To-Do)으로 전환	스프린트 태스크 도출

결과가 마음에 들면 **새 생각으로 붙이기**를 눌러 캔버스에 바로 포스트잇으로 추가할 수 있습니다. 서버는 선택한 생각과 공간의 현재 접근권한·AI 제외 상태를 외부 호출 직전에 다시 확인하고 응답이 끝날 때까지 고정합니다. 공유 회수나 제외 설정이 먼저 적용되면 해당 본문은 AI Gateway에 전달되지 않습니다.

3-1. 메모 정렬

캔버스 도구 모음에 세 가지가 있습니다. 셋 다 **Ctrl+Z** 한 번으로 되돌아갑니다.

도구	하는 일	무엇을 건드리지 않나
겹침 정리	겹친 것만 최소 거리로 떼어 놓습니다	여유가 있는 생각은 한 개도 옮기지 않습니다
생각 군집	묶음별로 나란히 정렬합니다	어느 묶음에도 없는 생각은 그대로 둡니다
Thought Gravity	고른 생각의 직접 연결된 것들을 둘레에 모읍니다	나머지 생각

겹침 정리가 기본으로 쓰기 좋은 쪽입니다. 나머지 둘이 배치를 다시 만든다면, 이건 배치를 **지킵니다** — 자리가 넉넉한 생각은 손대지 않으므로, 한쪽 구석이 뭉친 걸 풀어도 나머지 공간은 그대로입니다. 생각을 여러 개 고른 뒤 누르면 **고른 것들 안에서만 정리합니다**.

세 도구 모두 메모의 **실제 크기**를 봅니다. 크게 늘려 둔 메모도 이웃을 덮지 않습니다.

이 캔버스에서 위치는 장식이 아니라 **그 생각을 기억하는 방식**입니다. 그래서 정렬은 되돌릴 수 있어야 하고, 필요 이상으로 움직이지 않아야 합니다.

4-1. 내 기억에 물어보기 · 살펴보고 답하기

오늘의 리뷰 화면 위쪽에 있습니다. 두 가지 모드가 있고, 둘 다 **직접 적어 둔 생각만** 근거로 씁니다.

모드	하는 일
한 번에 답하기	질문으로 한 번 검색하고, 근거를 [번호] 로 인용해 답합니다
살펴보고 답하기	조력자가 찾아보고, 읽고, 필요하면 다시 찾아본 뒤 답합니다. 어디를 봤는지 순서대로 보여 줍니다

- 근거가 없으면 **모델을 부르지 않습니다**. 그때가 근거 없는 답이 가장 그럴듯하고 가장 틀리는 경우입니다.
- 조력자는 읽기만 합니다. 만들기·고치기·연결하기 도구가 아예 붙어 있지 않아, 부르고 싶어도 부를 것이 없습니다. 제안은 글로 돌아오고 실행은 사람이 합니다.
- **AI 제외로 표시한 생각은 연결을 통해서도 새지 않습니다**. 빠진 개수만 알려 줍니다.
- 채팅 모델이 설정되어 있어야 하며, 한 번 살펴볼 때마다 일일 AI 사용량에서 차감됩니다.

4-2. 생각의 갈래와 결정 기록

시도해 봤다가 접어 둔 생각은, 검색해서 다시 만났을 때 **현재 방침과 똑같이 생겼습니다**. 그래서 이미 버린 선택지를 다시 집어들게 됩니다. 갈래는 그걸 막습니다.

만들기: 메모 메뉴(:) → 연결과 갈래 → 갈래 이름을 적고 만들기. 넣기: 같은 패널에서 갈래 배지를 눌러 이 생각을 넣거나 뺍니다. 정리하기: 갈래 메뉴 → 이 방향으로 정함 또는 접어 두기.

이유는 필수입니다. 이유 없이 접어 둔 갈래는 나중에 결정만 남고 까닭은 사라집니다. 그게 이 기능이 막으려는 망각입니다.

접어 둔 갈래의 생각에는 어디에 나타나도 표시가 붙습니다 — 캔버스, 검색, 내 기억에 물어보기, 조력자, 내보내기, MCP. 그리고 거의 같은 생각을 새로 적으면 오늘의 리뷰가 "접어 둔 갈래를 다시 쓰고 있습니다"라고 이유와 함께 알려 줍니다.

왼쪽 메뉴의 결정 기록은 표시한 것만 시간순으로 모읍니다: 갈래 채택·접어 둔, 질문에 답 표시, 상충 표시. 활동이 많았다는 이유로 전환점을 지어내지 않습니다. 비어 있으면 "표시된 것이 없다는 뜻이지, 아무 일도 없었다는 뜻은 아닙니다"라고 말합니다.

4-3. 한 갈래만 보기 (초점)

캔버스 톨바의 초점 버튼에서 갈래를 고르면 그 갈래만 선명해지고 나머지는 흐려집니다. 생각을 하나 고른 상태라면 연결 1~3걸음으로도 좁힐 수 있습니다.

흐리게 둘 뿐 숨기지 않습니다. 사라진 생각은 지워진 생각으로 읽힙니다. 그리고 몇 개를 흐리게 뒀는지 항상 함께 말합니다.

좁히는 기준은 갈래와 연결 — 사람이 직접 그은 구조입니다. 그래서 임베딩 성능과 무관하게 일정합니다.

5. Dream 검토 및 채택

- 검토함: 왼쪽 Dreams 메뉴에서 밤사이 생성된 후보를 확인합니다. 자동 생성은 현재 접근 가능한 사용자 작성 생각만 원본으로 삼고, 후보는 아직 캔버스에 추가되지 않은 상태입니다.
- 근거 확인: 왜 이 Dream인가요? 설명과 원본 생각 카드를 확인하고, 원본 카드를 누르면 해당 메모로 바로 이동합니다.
- 채택: 캔버스에 남기기를 누르면 현재 공간 편집권한을 transaction 끝까지 확인한 뒤 Dream 메모와 원본 생각 연결선이 함께 만들어집니다.
- 다른 관점: 같은 원본 생각을 유지하면서 최근 Dream과 겹치지 않는 다른 결론을 요청합니다.
- 발전시키기: Dream을 더 구체적으로 확장하거나, 반대 관점에서 검토하거나, 실행 항목으로 바꿉니다. 서버는 외부 AI 호출이 끝날 때까지 Dream과 원본 공간의 현재 접근권한을 고정하므로 공유 회수와 요청이 겹치면 둘 중 먼저 시작된 작업 순서가 보장됩니다. 채택한 Dream의 발전 결과를 남기면 원본 Dream 옆에 새 메모와 연결선이 함께 저장되며, 네트워크 재시도에도 중복되지 않습니다.
- 도착 알림: 상단 중 모양 알림에서 Dream을 누르면 읽음 처리 후 해당 검토 카드로 바로 이동합니다. 카드는 화면에 실제 보였을 때만 노출로 집계됩니다.
- 공유 회수: 공유 공간의 Dream은 현재 그 공간을 볼 수 있을 때만 Today와 검토함에 표시됩니다. 공간에서 제거되면 파생 내용과 원본 카드가 숨겨지고 이전 주소나 ID로 피드백·채택·재생성·발전할 수 없습니다.

- **숨김 피드백:** 뻔함, 관련 없음, 사실 오류, 반복은 이후 자동 스타일 선호에 반영됩니다. **너무 자주 와요** 를 고르면 생성 빈도가 **매일 → 주 3회 → 주 1회** 순서로 낮아지고, 이미 주 1회라면 7일 동안 쉽니다.
- **민감 정보 제외:** 메모의 점 세 개 메뉴에서 **Dream 분석에서 제외** 를 선택하거나, 공간 관리에서 **AI Dream 분석** 을 끌 수 있습니다. 제외한 내용은 AI Assist에도 전달되지 않습니다.

6. 캔버스 단축키 일람표

단축키	기능	설명
더블클릭	메모 생성	캔버스 빈 곳에 즉시 포스트잇 생성
N	Quick Capture	하단 새 생각 입력창으로 포커스
/	생각 검색	상단 캔버스 메모 검색창으로 이동
Cmd + F	생각 검색	캔버스 메모 검색창 열기
F	캔버스 맞춤	모든 메모가 한눈에 들어오도록 줌 조정 (Fit View)
Cmd + A	전체 선택	캔버스 안의 모든 메모 선택
Delete / Backspace	메모 삭제	선택된 메모 일괄 삭제
Cmd + Z	위치 Undo	메모 이동 이전 위치로 되돌리기
Shift + Cmd + Z	위치 Redo	되돌린 메모 위치 다시 적용
Tab	생각 사이 이동	포스트잇 사이를 키보드로 이동
Enter	편집 시작	선택한 포스트잇의 내용 편집으로 들어가기
방향키	위치 이동	선택한 포스트잇을 5px씩 이동 (Shift 와 함께 20px)
Esc	선택 해제 / 편집 종료	편집 중이면 카드로 빠져나오고, 아니면 검색과 선택을 해제

마우스 없이도 캔버스를 쓸 수 있습니다. **Tab** 으로 포스트잇을 옮겨 다니고, **Enter** 로 편집에 들어가고, **Esc** 로 나온 뒤 방향키로 위치를 잡습니다.

7. 내보내기 (Export)

상단 툴바의 **내보내기 (IconDownload)** 메뉴를 통해 3가지 형식으로 산출물을 추출할 수 있습니다:

1. **Markdown (.md)**: 메모 제목, 본문, 연결 관계를 정리된 마크다운 문서로 다운로드
2. **Image (.png)**: 현재 캔버스의 전체 전경을 2000px급 고해상도 투명 PNG 이미지로 다운로드
3. **PDF (.pdf)**: A4 가로/세로 비율에 맞춰 고품질 PDF 문서로 즉시 생성 및 인쇄

같은 메뉴의 마크다운 가져오기로 반대 방향도 가능합니다. 마크다운 파일을 고르거나 텍스트를 붙여넣으면 # 제목, --- 구분선, 빈 줄을 기준으로 나뉘어 각 항목이 하나의 생각으로 캔버스에 붙습니다. 일부 항목이 실패하면 성공한 항목은 그대로 두고 실패한 항목만 입력란에 남습니다.

umm이 내보낸 파일은 공간 그대로 되 돌아옵니다. 생각과 제목뿐 아니라 캔버스에서의 자리, 생각 사이의 연결, 생각의 갈래와 그 갈래가 어떻게 끝났는지, 질문·아이디어 구분, 메모 색까지 함께 복원됩니다. 파일을 두 개 이상 고르거나 이미 적어 둔 내용 뒤에 붙여도 됩니다.

내보낸 파일은 나누지 마세요. umm은 파일 첫머리의 Exported from umm at ... 줄을 보고 자기 파일임을 알아봅니다. 파일을 잘라 여러 번에 나눠 넣으면 그 줄이 없는 조각은 평범한 마크다운으로 읽혀, 자리·연결·갈래가 사라지고 - id: 같은 줄이 본문에 남습니다.

남의 마크다운을 가져올 때는 한 번에 최대 200개까지입니다. 200개를 넘으면 실제 개수를 알려 주고 가져오기 버튼을 비활성화하며, 201번째 이후 내용도 입력란에 그대로 남습니다. 이 상한은 umm이 내보낸 파일에는 적용되지 않습니다 — 공간 하나를 통째로 되돌리는 일이라 개수로 막지 않습니다.

모바일에서는 다른 앱의 공유 시트에 umm이 나타납니다. 기사나 메모를 공유하면 캔버스의 빠른 입력창에 내용이 채워진 채로 열립니다.

8. 개인 설정 및 API/MCP 키 발급

- /settings 메뉴 이동: 우측 상단 프로필 메뉴 또는 네비게이션에서 개인 설정으로 이동합니다.
- Dream 설정: Dream 기능 켜기/끄기, 도착 알림, 생성 주기, 스타일 선호도를 지정합니다.
- 연결선 형태 선택: 베zier 곡선 / 둥근 꺾은선 / 직선 중 선호하는 스타일을 선택합니다.
- 로그인한 기기: 이 계정에 로그인된 다른 브라우저를 확인하고 개별 종료하거나 한 번에 모두 로그아웃할 수 있습니다. 모르는 기기가 보이면 즉시 종료하고 비밀번호를 바꾸세요.
- 언어와 테마: 우측 상단 프로필 메뉴에서 한국어/English와 밝게/어둡게/시스템 설정을 고릅니다. 선택은 계정에 저장되므로 다른 브라우저에서도 그대로 따라오고 로그인 직후 현재 탭에도 적용됩니다. 두 선택을 빠르게 연달아 바꿔도 서로 되돌리지 않습니다.
- 개인 API · MCP 키 발급:
 1. 새 키 버튼을 클릭합니다.
 2. 키 이름과 만료 기간을 입력하고 필요한 권한 스코프(Scopes)를 체크합니다.
 - 생각 조회만 필요하면 notes:read, AI Assist로 선택한 생각을 발전시켜야 할 때만 ai:assist를 추가합니다. notes:read 만으로는 외부 AI 호출을 시작할 수 없습니다.
 3. 키 만들기 를 누르고 생성된 umm_key_... Bearer 토큰을 안전한 곳에 복사합니다.
 4. 키 만료 전 회전 버튼을 누르면 설정된 중첩 시간(예: 24시간) 동안 구버전/신버전 키가 공존하여 무중단 전환이 가능합니다. 서버의 보안 난수 생성에 문제가 생기면 교체는 실패하고 기존 키 상태가 유지되므로 잠시 뒤 다시 시도하세요.

9. 오늘의 리뷰와 다시 보기

로그인 후 기본 화면은 `/today` 입니다. 검토 카드에서 `완료` 를 누르면 현재 시각이 기록되고, `다시 보기` 를 선택하면 지정한 날짜까지 목록에서 미뤄집니다. 검토·고정 상태는 사용자별로 독립되어 공유 생각에서도 다른 참여자의 목록에 영향을 주지 않습니다. 개인 설정의 `리뷰 요약에 활동 포함` 을 끄면 최근 댓글 활동만 숨고 고정된 생각, 다시 볼 날짜가 된 생각, 오래 검토하지 않은 생각은 계속 표시됩니다. 연결되지 않은 생각은 캔버스로 열어 다른 생각과 연결할 수 있고, 대기 Dream은 검토함으로 이어집니다. 온보딩 단계는 실제 공간·생각·연결 상태를 기준으로 자동 갱신됩니다.

10. 댓글, 멘션과 백링크

포스트잇 메뉴의 `댓글` 을 열어 최대 4,000자의 의견을 남깁니다. 같은 공간에 참여한 사용자를 `@username` 으로 언급하면 알림이 해당 포스트잇 댓글로 바로 연결됩니다. 작성 중 공간 공유가 회수되면 서버가 현재 멤버십을 transaction에서 다시 확인하므로 이전 권한으로 댓글이나 알림을 만들지 않습니다. 기존 알림도 생각이 삭제되거나 공간 접근권한이 사라지면 목록과 읽지 않은 개수에서 숨겨집니다. 편집 권한이 있는 사용자는 논의를 해결 또는 다시 열 수 있고 작성자·공간 관리자는 삭제할 수 있습니다. 원본 생각이 삭제되거나 공간에서 제거된 뒤에는 이전에 보관한 댓글 주소나 ID로 해결·재개·삭제할 수 없으며, 오프라인에 저장해 둔 해당 변경도 재연결 시 사유를 알리고 제거됩니다. 오른쪽 맥락 패널의 백링크는 들어오는 연결과 나가는 연결을 구분합니다.

11. 오프라인 작업과 충돌 해결

네트워크가 끊긴 동안의 생성·수정은 브라우저 queue에 저장되고 상단 상태 표시에서 대기 수를 확인할 수 있습니다. 재연결하면 자동 동기화되며 같은 메모의 연속 수정은 마지막 상태로 합쳐집니다. 브라우저 저장 공간이 가득 찼거나 사이트 저장 권한이 차단되면 “오프라인 저장 실패”를 표시하며 저장되지 않은 변경을 대기 중이라고 잘못 안내하지 않습니다. 저장할 수 없었던 Canvas 편집은 마지막으로 서버나 오프라인 queue에 실제 보존된 내용으로 돌아가므로 화면에만 남은 내용을 저장 완료로 오해하지 않습니다. 실패한 요청이 진행되는 동안 다시 편집했다면 새 내용은 자신의 저장 시도가 끝날 때까지 먼저 되돌리지 않습니다. 이 경우 브라우저의 사이트 데이터 권한과 남은 저장 공간을 확인하고 연결을 복구한 뒤 다시 저장하세요. 저장소 접근이 차단되어도 로그인 화면, Canvas 기본 설정과 오프라인 상태 표시는 계속 열립니다. 서버에 더 최신 버전이 있으면 자동 덮어쓰기하지 않고 비교 창을 엽니다. 내 버전, 서버 버전 또는 두 본문을 합친 버전을 선택해 새 버전으로 저장하세요. 삭제된 대상이나 권한 회수 뒤의 댓글 변경처럼 다시 시도해도 적용할 수 없는 변경은 오류 사유를 알린 뒤 해당 항목만 queue에서 제거하고, 뒤의 독립적인 변경은 계속 동기화합니다. 서비스 업그레이드 중 이전 버전 탭을 함께 열어 두었더라도 그 탭이 늦게 기록한 대기 변경은 새 계정 queue와 자동 병합됩니다.

설치형 PWA는 마지막 HTML 앱 셸을 보존합니다. manifest나 아이콘 같은 정적 파일을 주소창에서 직접 열었더라도 앱 셸 캐시를 덮어쓰지 않으므로, 이후 네트워크가 끊겨도 umm 화면으로 다시 진입할 수 있습니다. 새 버전을 배포하면 고정 주소의 manifest·service worker·아이콘은 서버에 재검증되고 content hash가 붙은 bundle만 장기 캐시되어 설치 정보와 바로가기가 이전 릴리스에 고정되지 않습니다.

공용 PC에서는 브라우저 저장소에 오프라인 변경이 남을 수 있으므로 로그아웃 전에 동기화 완료를 확인하고 브라우저 프로필을 종료하세요.

12. 접근성 보기와 키보드

캔버스 상단에서 선형 목록 보기를 선택하면 공간 위치에 의존하지 않고 생각을 순서대로 읽을 수 있습니다. 포스트잇에 focus한 뒤 화살표 키로 위치를 미세 조정할 수 있으며, 운영체제에서 동작 줄이기를 켜면 불필요한 animation이 비활성화됩니다.

umm 관리자 운영 가이드 (Admin Guide)

umm은 사내 폐쇄망 및 오프라인 엔터프라이즈 환경에서 외부 의존성 없이 안정적으로 동작하도록 설계되었습니다.

1. 런타임 환경변수 4종

서비스 기동 시 필요한 환경변수는 정확히 다음 4개뿐이며, 그 외 모든 정책은 관리자 웹 콘솔에서 런타임으로 관리됩니다:

환경변수명	필수 여부	설명	예시
POSTGRES_DSN	필수	PostgreSQL 데이터베이스 연결 문자열	postgres://umm:password@postgres:5432/umm?sslmode=disable
BOOTSTRAP_ADMIN	필수	최초 생성할 시스템 관리자 로그인 아이디	admin
BOOTSTRAP_ADMIN_PASSWORD	필수	부트스트랩 관리자의 초기 비밀번호	ComplexPassword123!
ENCRYPTION_KEY	필수	비밀값 암호화를 위한 32바이트 AES 키	01234567890123456789012345678901

[!NOTE] DB에 이미 bootstrap 관리자가 존재할 경우, 서버를 재시작해도 비밀번호가 임의로 덮어써지지 않습니다.

선택 환경변수 `ENCRYPTION_KEY_PREVIOUS`는 master-key 회전 기간, `UMM_HTTP_ADDR`는 listen 주소 변경, `UMM_TRUSTED_PROXY_CIDRS`는 신뢰할 reverse proxy 주소 지정, 표준 `OTEL_EXPORTER_OTLP_*`는 trace 전송에만 사용합니다. 필수 입력은 네 개로 유지됩니다.

TLS 종료 proxy 뒤에서 실행할 때만 직접 연결되는 proxy의 IP 또는 CIDR을 심표로 구분해 지정하세요(예: `10.42.0.0/16,fd00:42::/64`). 설정하지 않으면 `X-Forwarded-For`, `X-Real-IP`, `X-Forwarded-Proto`를 모두 무시하며, `0.0.0.0/0` 또는 `::/0`처럼 인터넷 전체를 신뢰 대상으로 지정해서는 안 됩니다. Proxy는 외부에서 들어온 forwarding header를 제거하고 자신이 확인한 값을 기록하도록 구성하세요.

브라우저 변경 요청의 `Origin`은 공개 URL 또는 신뢰 proxy로 확인한 현재 요청과 scheme·host·port가 모두 같아야 합니다. HTTPS 서비스와 같은 hostname이더라도 `http://` Origin은 거부됩니다. PostgreSQL request pool 자동 상한은 host CPU 수와 무관하게 인스턴스당 16이며 compose도 이를 명시합니다. `POSTGRES_DSN`의

`pool_max_conns` 는 replica 수와 DB의 전역 연결 한도를 기준으로 조정하세요. 외부 호출 동안 권한을 고정하는 lease는 request pool을 점유하지 않는 별도 연결을 사용하며 AI 호출 최대 2개와 웹훅 전달 최대 3개로 각각 제한됩니다. 전역 연결 예산에는 replica마다 최악의 경우 `+5` 를 더하고, 각 상한을 넘는 lease는 연결 없이 대기합니다. request pool 상한 1~2에서는 전용 LISTEN을 끄고 1초 안전 폴링으로 모든 연결을 request에 남기며, 상한 3부터 listener를 시작하면서 request용 두 자리를 보존합니다. 실행 중 listener 상태가 바뀌면 열린 SSE가 즉시 깨어나 단절 시 1초 폴링, 복구 시 30초 safety net으로 전환합니다. 알림 목록과 짧은 Dream transaction도 작은 pool에서 연결을 중첩 점유하지 않지만 운영에서는 동시 요청과 worker를 위해 `pool_max_conns` 4 이상을 권장합니다.

로그인 세션 목록에 표시하는 User-Agent는 공백과 잘못된 UTF-8을 정리한 뒤 최대 300 byte의 완전한 rune 경계에서 저장합니다. 긴 한국어·다국어 브라우저 식별자가 중간 byte에서 잘려 올바른 자격 증명의 세션 생성이 실패하거나 주소별 로그인 실패 횟수로 잘못 누적되지 않습니다.

2. Keycloak OIDC SSO 연동

`umm` 은 Keycloak OpenID Connect Discovery를 통해 복잡한 설정 없이 엔터프라이즈 SSO를 즉시 연동합니다.

1. Keycloak Client 생성:

- Client type: `OpenID Connect`
- Client authentication: `ON` (Confidential)
- Standard Flow: `ON`
- Valid redirect URIs: `https://<umm-domain>/api/v1/auth/oidc/callback`

2. 관리자 콘솔 설정 (`/admin/oidc`):

- `Keycloak SSO 활성화` 스위치 ON
- Issuer URL: `https://keycloak.company.internal/realms/enterprise`
- Client ID & Client Secret: Keycloak에서 발급받은 자격증명 입력
- 관리자 그룹/역할: `umm-admins`
- 팀장 그룹/역할: `umm-leads`

3. 연결 시험:

- `연결 시험` 버튼을 클릭하여 OIDC Discovery 및 토큰 엔드포인트 도달 가능 여부를 실시간 검증합니다.

3. Dream Layer & 야간 Scheduler 운영

Dream Layer는 사용자가 밤사이 휴식하는 동안 캔버스에 쌓인 생각들의 의미적 연관성을 분석하고 새로운 아이디어를 제안하는 핵심 백그라운드 엔진입니다.

생성 결과는 v0.6부터 캔버스에 즉시 삽입되지 않고 개인 Dream 검토함에 후보로 저장됩니다. 사용자가 채택할 때만 Dream 메모와 원본 연결선이 함께 생성되며, 기존 버전에서 이미 캔버스에 생성된 Dream은 업그레이드 시 채택 상태로 보존됩니다.

- 스케줄 설정:

- **자동 생성** 스위치 ON
- 생성 시간: 기본 **02:00** (사내 심야 시간대)
- 생성 주기: 매일(Daily), 평일(Weekdays), 주말(Weekends), 특정 요일 선택, N일 간격
- **컨텍스트 & 256K 토큰 한도:**
 - **최소 메모 개수:** 2개 이상 메모가 있어야 분석 시작
 - **분석 범위:** 최근 14일
 - **최대 응답 Token:** 4K부터 ****최대 256K (262,144 tokens)****까지 모델 사양에 맞춰 슬라이더로 조절
- **품질 기준선 (Quality Threshold):**
 - 원본 두 개 이상에 대한 근거성, 적정 새로움, 구체성, 출처 범위를 결합한 내부 최소 점수
 - **Quiet Mode:** 가치가 확실하고 영감을 주는 의미 있는 Dream이 없을 경우 불필요한 노이즈 생성을 건너뛰니다.
- **선택 및 개인정보 보호:**
 - 최근 Dream이 적은 적격 공간을 순환하고, 연결되지 않았으나 의미적으로 이어질 수 있는 메모 조합을 우선합니다.
 - 메모 또는 공간에서 AI 제외를 켜면 Scheduler 자격 계산, Dream 생성, AI Assist에서 모두 제외됩니다. 선택적 임베딩 모델을 설정했다라도 제외 콘텐츠는 Gateway로 전송하지 않고 서버 안의 로컬 vector만 사용합니다. 외부 임베딩 직전 note-space와 Gateway 설정을 다시 잠그고 응답 저장까지 유지하므로, 제외 설정이 먼저 적용되면 본문을 보내지 않고 임베딩이 먼저 시작되면 설정 변경이 호출 종료까지 기다립니다. 메모 하나만 제외된 공간도 Canvas와 범위 검색 전체가 같은 로컬 vector 공간을 사용해 일반 메모의 의미 검색 결과가 사라지지 않습니다. 원격 범위 검색은 space-membership-활성 note를 검색 완료까지 잠그며 제외-접근 회수 뒤의 검색어는 Gateway에 보내지 않습니다.
- **운영 지표:**
 - 단순 생성 수 외에 검토 완료, 채택률, 편집·연결·확장 기반 유의미 활용률, 채택 Dream당 비용을 함께 확인합니다. 노출 수는 검토함을 불러온 횟수가 아니라 카드가 화면에 50% 이상 실제 표시된 경우만 반영됩니다.

4. 내부 AI Gateway 연동

사내망 내부의 LLM Gateway (vLLM, Ollama, TGI, SGLang 등 OpenAI 호환 서버)를 연결합니다.

- **Base URL:** **http://llm-gateway.internal:8000** , **.../v1** , 전체 **.../chat/completions** 주소를 모두 사용할 수 있습니다.
- **API Key:** 내부 보안 게이트웨이 인증 토큰
- **Timeout:** 긴 추론 모델을 위해 최대 1800초까지 설정 가능. 이 값은 재시도를 모두 포함한 한 AI 작업의 전체 시간 예산이며, umm HTTP write timeout은 최대값보다 60초 길게 설정됩니다. 앞단 reverse proxy도 설정값보다 최소 60초 길게 응답 timeout을 구성하세요.
- **재시도:** 0~5회. 전체 Timeout 안에서만 수행되므로 재시도마다 1800초가 다시 부여되지 않습니다.
- **vLLM 추론 모델:** 가능하면 서버에 모델별 **--reasoning-parser** 를 설정합니다. 최종 본문 없이 **reasoning** / **reasoning_content** 만 반환하거나 **<think>** 도중 출력 한도에 도달하면, 재시도가 1 이상일 때

umm이 비추론 모드로 다시 요청하고 최종 `content` 만 사용합니다.

- **비용 통계 관리:** 입력/출력 1M 토큰당 비용을 입력하면 관리자 대시보드에서 월간 예상 비용과 사용자당 비용을 실시간으로 추산합니다.
- **임베딩 모델:** 비워 두면 외부 호출 없이 내장 로컬 임베딩(문자 n-gram)을 사용합니다. 내장 알고리즘은 **어휘 중복** 을 재며 동의어를 인식하지 못합니다(라벨 데이터셋 기준 쌍별 정확도 4.2%). 지금 붙어 있는 백엔드가 어느 쪽인 지는 아래 **임베딩 품질 측정**에서 바로 확인할 수 있습니다. 연관 생각·군집·Dream 품질을 실제로 올리려면 모델 설정이 사실상 필수입니다. 유사도 판정 기준은 v0.9.0부터 백엔드 분포에 맞춰 자동 조정되므로, 모델을 바꿔도 임계값을 손볼 필요가 없습니다. 모델 이름을 넣으면 같은 Gateway의 `/v1/embeddings` 를 사용해 연관 생각·군 집·검색의 의미 점수를 계산합니다. 모델이나 Gateway 주소를 바꾸면 endpoint fingerprint가 달라져 기존 생각이 열릴 때마다 점진적으로 다시 임베딩되며, 같은 모델명을 쓰는 서로 다른 제공자의 벡터도 섞여 비교되지 않습니다. 설정 변경 전에 시작한 이전 Gateway 응답은 저장 시점에 현재 설정과 다시 대조되어 폐기되므로 새 벡터를 뒤늦게 덮어쓰지 못합니다. 외부 호출과 vector 저장 동안에는 대상 note·space의 AI 제외 상태와 현재 Gateway 세대를 AI 전용 lease로 고정하므로 관리자가 제외를 저장한 뒤 캡처된 본문이 늦게 전송되는 경합도 없습니다. Gateway가 응답하지 않으면 전체 비교 집합을 로컬 임베딩으로 맞추고 5분간 로컬 회로 차단 상태를 유지해 Canvas를 열 때마다 원격 timeout이 반복되지 않습니다. 정책에 따른 로컬 전환은 장애로 세지 않아 회로 차단기 를 열지 않습니다. 암호화된 API key를 현재 keyring으로 읽을 수 없는 경우에도 저장된 ciphertext를 Gateway 자 격증명으로 시도하지 않고 즉시 로컬 provider로 단습니다. 이 전체 로컬 정규화 pass도 장애가 난 원래 Gateway 설정 세대에 묶여 있으므로, 두 pass 사이에 저장한 새 Gateway의 vector를 덮지 않습니다. 임베딩과 Dream 오류 응답 본문 일부는 다국어 rune 경계를 보존해 관리자 오류와 로그가 유효한 UTF-8을 유지합니다. 설정을 저장하 면 회로 차단 상태가 즉시 해제되어 새 설정을 확인합니다.

임베딩 Gateway를 따로 두기

임베딩과 채팅 모델이 같은 서버에 있을 이유는 없습니다. 큰 모델은 어딘가 강한 장비에 두고, 임베딩은 umm 옆에 작게 띄우는 편이 흔합니다 — `compose.yaml` 의 `embeddings` 프로파일이 정확히 그 모양입니다.

항목	비우면	적으면
<code>embedding_base_url</code>	위의 Base URL 을 씁니다 (기존 동작 그대로)	임베딩만 이 주소로 보냅니다
<code>embedding_api_key</code>	인증 헤더를 보내지 않습니다	이 키로 인증합니다

채팅 API Key는 임베딩 주소로 따라가지 않습니다.

한 호스트에 발급된 키는 그 호스트의 자격증명입니다. 임베딩 키를 비워 뒀다는 이유로 채팅 키를 대신 보내면, 그 서버 를 운영하는 사람에게 키를 건네주는 셈입니다. 인증이 필요 없는 임베딩 서버가 흔하므로 **비용 = 보내지 않음**입니다.

주소가 잘못되면 **저장을 거부**합니다. 조용히 무시하면 임베딩이 내장 알고리즘으로 돌아간 채, 설정 화면에는 저장된 값이 그대로 보이게 됩니다.

```
docker compose --profile embeddings up -d
docker compose exec embeddings ollama pull bge-m3
```

그 다음 임베딩 Gateway 주소에 `http://embeddings:11434`, 임베딩 모델에 `bge-m3` 을 넣으면 됩니다. 채팅 모델 주소는 그대로 두어도 됩니다.

임베딩 품질 측정

관리자 → **AI Gateway** → **임베딩 품질 측정**은 지금 설정된 백엔드로 라벨링된 한/영 문장쌍 22개를 직접 임베딩해 점수를 냅니다. 두 숫자만 보면 됩니다.

- **판별력** — `같은 뜻, 다른 표현` 의 평균에서 `단어만 겹침` 의 평균을 뺀 값. 음수면 뜻보다 어휘를 높게 보고 있다는 뜻입니다.
- **쌍별 정확도** — 패러프레이즈와 어휘 함정을 짝지어 비교했을 때 올바른 순서를 매기는 비율. 임계값과 무관합니다.

세 가지 상태가 나옵니다.

표시	뜻	할 일
● 어휘가 겹치는 정도만 재고 있습니다	모델 미설정. 연관 생각·군집·검색의 "의미상 유사"는 실제로는 어휘 유사입니다	임베딩 모델을 설정
● 설정한 모델이 쓰이지 않고 있습니다	모델은 설정됐지만 벡터가 로컬에서 나왔습니다. 게이트웨이 무응답이거나 모델 이름 오타	주소·모델명 확인
● 의미 기반으로 동작합니다	표현이 달라도 같은 뜻을 알아봅니다	—

결과는 백엔드별로 10분 캐시되며, 설정을 바꾼 뒤에는 **다시 측정**을 누르세요.

오프라인 임베딩 서버 띄우기

제대로 된 임베딩 모델을 umm 이미지에 넣으려면 추론 런타임을 링크해야 하고, 그러면 umm을 단일 이식 바이너리로 만드는 `CGO_ENABLED=0` 정적 빌드가 깨집니다. 대신 옆에 세우면 정적 빌드를 유지하면서도 완전 오프라인으로 돌릴 수 있습니다. 모델은 한 번만 받으면 이후에는 볼륨만 있으면 됩니다.

```
docker compose --profile embeddings up -d
```

컨테이너가 뜨면서 모델을 스스로 받습니다(`UMM_EMBEDDING_MODEL` 로 변경 가능, 기본 `bge-m3`). 볼륨에 이미 있으면 다시 받지 않고, healthcheck는 서버 응답이 아니라 **모델이 실제로 있는지**를 확인합니다.

그다음 AI Gateway에서 **자동으로 찾기**를 누르면 주소와 모델 목록이 나옵니다. 모델을 고르고 **연결 테스트**로 확인한 뒤 저장하고, **임베딩 품질 측정**에서 ●가 뜨는지 보면 끝입니다.

자동 찾기가 조사하는 주소는 바이너리에 고정되어 있습니다(`embeddings:11434`, `embeddings:8080`, `host.docker.internal:11434`, `127.0.0.1:11434`). 요청에서 주소를 받지 않습니다. 목록의 * 표시는 **모델 이름으로 짐작한 것**이며, 실제로 임베딩하는지는 연결 테스트가 확인합니다.

폐쇄망이라면 모델을 받아 둔 볼륨이나 `ollama/ollama` 이미지를 함께 반입하면 됩니다.

후보 모델 측정 결과

같은 데이터로 갠 값입니다. 임의의 OpenAI 호환 게이트웨이드 쓸 수 있습니다.

모델	크기	차원	판별력	쌍별 정확도	주제 분리	최근접 동일 주제	umm 중단 테스트
내장 로컬 (문자 n-gram)	0	192	-0.250	4.2%	-0.023	18.8%	X
bge-m3	1.2GB	1024	+0.079	83.3%	+0.105	75.0%	✓
bona/bge-m3-korean	1.2GB	1024	+0.146	81.2%	+0.101	68.8%	✓
paraphrase-multilingual	562MB	768	+0.115	85.4%	+0.223	87.5%	X

앞의 네 지표와 마지막 열이 서로 다른 순서를 매깁니다. 이건 측정 오류가 아니라 실제로 알아 둘 값입니다.

paraphrase-multilingual 은 라벨 데이터 네 지표에서 모두 1위이고 크기도 절반 이하지만, umm의 중단 클러스터링 테스트(보안 3 + 자전거 3)에서 실패합니다. **인증 토큰 만료 시간을 24시간으로 정했다** 의 최근접 이웃으로 **라이딩이 끝나면 스트레칭을 꼭 한다** (0.32)를 고르기 때문입니다 — 같은 주제인 **세션 쿠키 ...** (0.23)보다 높게. 한 문장이 틀렸을 뿐인데 6개 메모 공간에서는 두 주제가 한 덩어리로 합쳐집니다.

즉 집계 점수가 높다고 특정 워크스페이스에서 더 나은 군집이 나오는 것은 아닙니다. 모델마다 틀리는 문장이 다르고, 메모 수가 적을수록 한 번의 실수가 결과를 지배합니다.

- 기본 권장은 **bge-m3** 입니다. 라벨 지표는 중간이지만 umm이 실제로 단언하는 중단 동작을 통과합니다.
- 한국어 비중이 높고 무관한 생각이 섞이는 쪽이 더 신경 쓰이면 판별력이 가장 높은 **bona/bge-m3-korean** .
- 디스크나 메모리가 빠듯하면 **paraphrase-multilingual** (562MB)도 실용적입니다. 다만 위 한계를 알고 쓰시고, 설정 후 실제 공간에서 연관 생각과 군집을 눈으로 확인하세요.

무엇을 고르든 **임베딩 품질 측정**에서 ●가 뜨는지, 그리고 실제 공간에서 군집이 납득되는지 두 가지를 함께 보는 것이 가장 확실합니다.

지표가 뜻하는 것:

- **판별력 / 쌍별 정확도** — 뜻이 같은 문장쌍과 단어만 겹치는 문장쌍을 구별하는 능력.
- **주제 분리 / 최근접 동일 주제** — 라벨된 4개 주제 16문장에서, 같은 주제끼리 더 가까운지와 각 문장의 가장 가까운 이웃이 같은 주제인지. 연관 생각과 군집이 실제로 하는 일이 이쪽입니다.

직접 후보를 재려면:

```
UMM_EMBEDDING_TEST_URL=http://127.0.0.1:11434 \
UMM_EMBEDDING_TEST_MODEL=<모델명> \
go test ./internal/intelligence -run Gateway -v
```

중단 동작까지 확인하려면 (PostgreSQL 필요):

```
POSTGRES_DSN=... UMM_EMBEDDING_TEST_URL=http://127.0.0.1:11434 \
UMM_EMBEDDING_TEST_MODEL=<모델명> \
go test ./internal/store -run ClustersAndRelated -v
```

모델을 바꾸면 차원과 fingerprint가 달라져 기존 생각이 열릴 때마다 점진적으로 다시 임베딩됩니다. 유사도 판정 기준은 v0.9.0부터 백엔드 분포에 맞춰 자동 조정되므로 임계값을 손볼 필요는 없습니다.

4-2. 유사도 기준 (/admin/intelligence)

연관 생각·군집·연결 추천이 얼마나 과감할지를 정합니다. 거의 전부 분포 상대적입니다 — 백엔드마다 "가깝다"의 뜻이 다르므로, 이 공간의 유사도 분포에서 평균보다 몇 표준편차 위인지로 판단합니다. 모델을 바꿔도 손볼 필요가 없는 이유입니다.

항목	기본값	의미
<code>related_band</code>	0.6 σ	연관 생각으로 보여 줄 기준
<code>strong_band</code>	0.9 σ	강한 연관
<code>cluster_band</code>	1.1 σ	군집으로 묶을 기준
<code>autolink_band</code>	1.1 σ	연결 추천 기준
<code>autolink_min_notes</code>	6	이보다 적으면 추천하지 않습니다
<code>autolink_max_per_run</code>	12	한 번에 제안할 최대 개수
<code>semantic_accuracy_bar</code>	0.65	이 아래면 "의미를 못 쥌다"고 판단해 의미 기능을 끕니다
<code>semantic_purity_bar</code>	0.60	같은 기준의 최근접 동일 주제
<code>duplicate_similarity</code>	0.92 (절대값)	근접 중복 판정
<code>quality_cache_minutes</code>	10	품질 측정 캐시

`duplicate_similarity` 만 절대값인 것은 의도적입니다. 측정해 보면 실제 중복은 두 모델에서 0.943~0.986에 모이고 그다음 등급은 0.681에서 끊깁니다. 그리고 상대 기준은 여기서 실패합니다 — 중복이 많은 공간은 자기 분포가 통째로 올라가서, 중복이 가장 많을 때 중복이 특별해 보이지 않게 됩니다.

규모에 따른 경계

중복 검사는 모든 쌍을 비교하므로 제공으로 자랍니다(1,024차원 실측: 1,000개 229ms · 2,000개 916ms · 10,000개 $\approx$ 23초). 한 공간에서 최근 1,000개까지만 비교하고, 그 위로는 오늘의 리뷰가 "*"공간이 커서 최근 생각까지만 비교했습니다"*라고 말해 줍니다. 접어 둔 갈래의 생각은 별도로 최근 창과 교차 비교되므로, 오래전에 거절한 결정을 오늘 다시 써도 잡힙니다.

이 화면의 모든 항목은 의미를 재는 백엔드에서만 효과가 있습니다. 내장 임베딩에서는 연관 생각·군집·중복·연결 추천이 전부 꺼지고, 오늘의 리뷰가 `backend-not-semantic` 으로 건너뛴 사실을 표시합니다.

4-1. 남용 방지 (키·권한 → 남용 방지)

로그인 실패와 요청 폭주로부터 서비스를 보호하는 값들입니다. 저장 즉시 적용되며 재시작이 필요 없습니다.

항목	기본값	의미
로그인 실패 허용 횟수	8	같은 주소에서 이만큼 실패하면 잠깁니다. 계정별 임계값은 자동으로 이 값의 3배입니다
로그인 잠금 시간	15분	잠금이 유지되는 시간
분당 API 요청	600	호출자별 상한
분당 AI 요청	6	AI 생성(<code>/ai/assist</code> , Dream 재생성·발전, 관리자 평가 실행) 전용 상한
하루 AI 생성 한도	80	사용자별 24시간 상한. <code>0</code> 이면 제한하지 않습니다

롤링 배포 중 구버전 관리자 화면이 새 남용 방지 필드를 모른 채 보안 섹션을 저장해도, 서버는 생략된 로그인·API·AI 한도를 PostgreSQL의 최신 확정값에서 transaction으로 병합합니다. 명시한 값은 `0` 을 포함해 그대로 적용하며 `null` 이나 범위 밖 값은 저장 전에 거부하므로, 업그레이드 도중 운영자가 조정한 한도가 기본값으로 조용히 돌아가 지 않습니다.

계정별 임계값을 주소별보다 높게 둔 이유는, 아이디를 아는 사람이 반복 실패를 일으켜 남의 계정을 마음대로 잠글 수 있는 상태를 피하기 위해서입니다. 로그인 성공은 해당 계정의 실패 횟수만 초기화하며, 같은 주소가 다른 계정에 누적인 실패 횟수는 유지되어 정상 계정 로그인으로 주소 제한을 우회할 수 없습니다.

로그인 실패 횟수와 하루 AI 사용량은 데이터베이스에서 관리합니다. 같은 주소·계정의 로컬 로그인은 모든 인스턴스가 공유하는 PostgreSQL 잠금 아래 비밀번호 확인과 실패 기록을 함께 처리하므로 병렬 요청도 설정한 실패 횟수를 초과해 검증되지 않습니다. 대화형 AI, 자동 Dream, 관리자 AI 평가는 실제 Gateway 호출 직전에 PostgreSQL에서 원자적으로 한도 자리를 선점하고 24시간 사용량으로 영속화하므로, 요청 취소·로그 저장 실패·여러 인스턴스의 동시 실행에도 하나의 상한이 적용됩니다. 분당 API 요청 한도는 인스턴스별로 계산되므로, 실효 상한은 `설정값 × 인스턴스 수` 입니다.

허용 API/MCP scope에는 마이그레이션 009가 `ai:assist` 를 자동 추가합니다. 이 scope는 선택한 생각을 외부 Gateway로 전송하고 AI 쿼터를 소비하는 `/ai/assist` 에만 사용됩니다. 읽기 자동화에는 `notes:read` 만, AI 발전 자동화에는 `ai:assist` 만 또는 실제 작업에 필요한 두 scope를 함께 발급해 최소 권한을 유지하세요.

5. RBAC 사용자 관리 및 불변 감사 로그

RBAC 3대 역할 체계

- `admin` (관리자): 모든 공간 조회/수정/삭제, 서비스 설정, 사용자 관리, 감사 로그 열람, Dream 수동 큐 생성
- `team_lead` (팀장): 일반 기능 및 공간 공유 / 외부 내보내기 승인 요청 심사 및 결재 권한
- `user` (일반 사용자): 개인 생각 캔버스 작성, 소유/공유 공간 협업, 개인 키 발급

🛡 불변 감사 로그 (Immutable Audit Trail)

- 관리자가 수행한 모든 설정 변경, 사용자 역할 수정, 키 발급, 승인 처리 내역이 `audit_logs` 테이블에 영구 보존됩니다.
- 각 행위의 시각, 행위자, 작업 구분(Action), 대상 리소스(Resource ID)를 투명하게 추적할 수 있습니다.

6. 헬스체크 및 성능 지표 모니터링

경로	메서드	용도	설명
<code>/healthz</code>	GET	Liveness Probe	프로세스 생존 상태 및 버전 반환
<code>/readyz</code>	GET	Readiness Probe	PostgreSQL 연결 및 쿼리 가능 상태 확인

운영 현황 화면의 실시간 협업 카드는 열려 있는 이벤트 구독 수와 PostgreSQL 수신 상태를 보여 줍니다. 상태가 "폴백 폴링"이면 `LISTEN` 연결이 끊겨 상태 전환 즉시 1초 폴링으로 동작하고 있다는 뜻입니다. 협업은 계속되지만 데이터베이스 부하가 올라가므로 `umm_realtime_listener_up` 지표에 알림을 걸어 두세요. | `/api/v1/metrics` | GET | Prometheus | route별 request count, latency histogram, in-flight, build 정보. 관리자 브라우저 세션 또는 `metrics:read` API 키 전용 || `/mcp` | POST | Model Context Protocol | AI 에이전트 도구 연동 엔드포인트 |

표준 OTLP endpoint 환경변수가 설정된 경우에만 OpenTelemetry HTTP trace exporter가 활성화됩니다. 관리자 운영 현황에는 댓글 수, 온보딩 완료율, 최근 웹훅 실패와 AI 평가 통계도 표시됩니다.

7. Dream AI 평가 회귀

관리자 → AI 평가에서 최소 두 개의 입력 생각, 기대 단어와 금지 단어, Dream 유형을 저장합니다. 실행은 현재 AI Gateway와 prompt version을 그대로 사용하며 grounding, 기대/금지 단어, 구체성, 모델 응답 상태를 0~1 점수와 세부 항목으로 보존합니다. 모델·prompt·Gateway 설정을 바꾸기 전후에 같은 active case를 실행해 회귀를 확인하세요. Gateway 장애도 `error` run으로 남아 평가 이력이 사라지지 않습니다.

8. Master-key 회전

새 키를 `ENCRYPTION_KEY`, 현재 키를 `ENCRYPTION_KEY_PREVIOUS`에 배치한 뒤 재시작합니다. 보안 화면에서 fallback 1개 이상, unreadable 0을 확인하고 현재 키로 회전을 실행합니다. 이 작업은 OIDC/AI secret, 웹훅 secret, 암호화 AI prompt를 한 트랜잭션으로 다시 암호화하며 `enc:` wrapper 도입 전 raw v1 AI prompt도 현재 wrapper·v2 형식으로 정규화합니다. 회전 전에 열어 둔 OIDC·AI Gateway 화면에서 마스킹된 값을 동시에 저장해도 서버가 같은 설정 lock 뒤 최신 암호문을 병합합니다. pending 0을 확인하고 새 백업을 만든 후에만 이전 키 환경변수를 제거합니다.

9. 서명 웹훅 운영

사용자는 개인 설정에서 허용된 `webhooks:write` scope로 subscription을 관리합니다. 대상은 공개 HTTPS 443만 허용합니다. 도메인 변경과 PostgreSQL delivery outbox는 원자적으로 커밋되고, 재시작 시 대기 또는 lease가 만료된 항목을 이어서 처리합니다. 전달 worker는 구독·소유 사용자·이벤트 공간·현재 membership과 정확한 delivery claim을 실제 HTTP 응답까지 잠급니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 확정되면 payload를 보내지 않고, 전송이 먼저 시작되면 변경 transaction은 delivery terminal 상태와 payload 삭제가 확정될 때까지 기다립니다. 수신 시스템은 timestamp와 raw body의 HMAC-SHA256, 허용 시간 창을 검증하고 at-least-once 요청의 delivery UUID를 멍등 처리해야 합니다. 일시 실패는 세 번 재시도하고 연속 10회 실패 subscription은 자동 중지됩니다. 외부 오류는 잘못된 UTF-8을 제거하고 500 byte의 완전한 rune 경계 안에서 기록하므로 다국어 오류도 실패 횟수와 자동 중지를 막지 않습니다. terminal payload는 즉시 제거되고 metadata도 30일 후 정리되므로 운영 지표와 개인 설정의 마지막 오류를 함께 확인하세요.

umm API & MCP 연동 가이드 (API & MCP Guide)

umm은 시스템 통합 및 자동화를 위한 **REST API**와 AI 에이전트(Claude Desktop, Cursor, Custom Agent 등) 연동을 위한 **Model Context Protocol (MCP)** 엔드포인트를 제공합니다.

1. 인증 체계 (Authentication)

모든 API 및 MCP 호출은 개인 설정(</settings>)에서 발급받은 **Bearer API Key** 또는 세션 쿠키를 사용합니다:

```
Authorization: Bearer umm_key_a1b2c3d4_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2. REST API 주요 엔드포인트 (</api/v1>)

공간 (Spaces)

메서드	경로	설명	권한 스코프
GET	/spaces	사용자가 접근 가능한 공간 목록 조회	spaces:read
POST	/spaces	새 공간 생성 (<code>{"name": "새 프로젝트"}</code>)	notes:write
PUT	/spaces/{id}	공간 이름 변경 (<code>{"name": "수정된 이름"}</code>)	notes:write
DELETE	/spaces/{id}	공간 및 내부 메모 영구 삭제	notes:write
GET	/spaces/{id}/members	공간 참여 멤버 및 권한 목록 조회	spaces:read
POST	/spaces/{id}/members	공간에 사용자 초대	notes:write

생각 포스트잇 (Notes & Edges)

메서드	경로	설명	권한 스코프
GET	/spaces/{id}/notes	공간 내 모든 포스트잇과 연결선 조회	notes:read
POST	/spaces/{id}/notes	새 포스트잇 생성 (title , content , color , x , y , width , height)	notes:write
PUT	/notes/{id}	포스트잇 내용/위치/크기 수정	notes:write
DELETE	/notes/{id}	포스트잇 삭제	notes:write
GET	/notes/{id}/history	포스트잇 과거 수정 이력 스냅샷 조회	notes:read
POST	/notes/{id}/restore/{version}	특정 버전으로 포스트잇 복원	notes:write
GET	/notes/{id}/related	의미 유사도 기반 연관 생각 목록 추천	notes:read
GET	/notes/{id}/backlinks	들어오고 나가는 실제 연결선과 상대 생각 조회	notes:read
GET	/search?q= ...	로컬 의미·키워드 하이브리드 검색과 필터 ·cursor	notes:read
GET	/notes/{id}/comments	댓글·멘션 목록	notes:read
POST	/notes/{id}/comments	댓글 작성과 멘션 알림	notes:write
PUT	/comments/{id}/resolve	댓글 해결·재개	notes:write
POST	/spaces/{id}/edges	두 포스트잇 간 연결선 생성 (source , target , relation)	notes:write

공간 범위 하이브리드 검색은 공간과 개별 메모의 AI 제외 상태를 함께 사용해 Canvas와 같은 임베딩 알고리즘을 선택합니다. 메모 하나라도 제외된 공간은 일반 메모까지 완전한 로컬 비교 공간에서 검색해 혼합 vector 때문에 의미 결과 가 빠지지 않으며 검색어도 외부 Gateway로 보내지 않습니다. 원격 검색을 선택한 경우에는 현재 space-membership·활성 note 정책을 검색 결과를 읽을 때까지 고정합니다.

AI Assist

메서드	경로	설명	권한 스코프
POST	/ai/assist	선택한 1~20개 생각을 요약·질문·확장·반대 관점·실행 항목으로 발전	ai:assist
POST	/ai/ask	직접 적어 둔 생각만으로 질문에 답하고 근거를 함께 반환. 근거가 없으면 모델을 부르지 않고 grounded:false	ai:assist
POST	/ai/agent	조회 → 읽기 → 재조회를 거쳐 답하고, 조회 과정(steps)을 함께 반환. 읽기 전용이며 한 번에 6회까지	ai:assist

`/ai/agent`에 바인딩된 도구는 조회 셋뿐입니다. 만들기·고치기·연결하기·합치기 도구는 **아예 붙어 있지 않아** 모델이 부르고 싶어도 부를 것이 없습니다. 한도에 닿으면 도구 없이 한 번 더 물어 지금까지 확인한 것을 남기고 `truncated:true`로 표시합니다. 채팅 모델이 없으면 `412` (임베딩 모델과 별개), Gateway 실패는 `502`로 구분합니다.

`ai:assist`는 선택한 본문을 외부 Gateway로 보내고 AI 쿼터를 소비하는 기능만 따로 허용하는 최소 권한 scope입니다. `notes:read`만 가진 API key는 일반 생각 조회를 할 수 있어도 AI Assist를 호출할 수 없습니다. 서버는 선택한 모든 생각이 현재 접근 가능하고 AI 제외 상태가 아닌지 외부 호출 직전에 다시 확인합니다. 해당 `note·space·membership` 행을 Gateway 응답이 끝날 때까지 잠그므로 공유 회수나 AI 제외가 먼저 확정되면 본문을 보내지 않고 미사용 쿼터를 취소하며, AI lease가 먼저 시작되면 정책 변경은 호출 종료까지 기다립니다. 긴 lease는 일반 요청 pool과 분리된 인스턴스당 최대 2개 PostgreSQL 연결만 사용하므로, 추가 AI 호출이 대기해도 `readiness·인증·Canvas용 pool_max_conns`를 점유하지 않습니다.

🌞 Today 리뷰

메서드	경로	설명	권한 스코프
GET	<code>/today</code>	검토 대상·고립 생각·댓글·온보딩 집계. Dream 항목은 별도 권한이 있을 때만 포함	<code>notes:read</code> (<code>dreams:read</code> for Dream)
POST	<code>/notes/{id}/review</code>	검토 완료, 다시 보기, 고정	<code>notes:write</code>
GET	<code>/onboarding</code>	실제 사용 기반 온보딩 진행률	로그인
POST	<code>/onboarding/complete</code>	온보딩 완료 표시	로그인

⚙️ 임베딩 Gateway 분리

`ai_gateway` 설정의 `embedding_base_url` · `embedding_api_key`로 임베딩만 다른 서버로 보낼 수 있습니다. 비워 두면 `base_url` · `api_key`를 그대로 씁니다.

`embedding_base_url`을 지정하면 `api_key`는 **그쪽으로 보내지 않습니다**. 한 호스트에 발급된 키를 다른 호스트로 넘기지 않기 위해서이며, 인증이 필요 없으면 `embedding_api_key`를 비워 두면 됩니다. 두 키 모두 조회 시 마스킹되고 저장 시 암호화됩니다.

🌿 생각의 갈래와 결정 기록

메서드	경로	설명	권한 스코프
GET	/spaces/{id}/branches	갈래 목록과 생각 → 갈래 매핑(assignments)	notes:read
POST	/spaces/{id}/branches	갈래 만들기(뿌리 생각은 선택)	notes:write
POST	/branches/{id}/resolve	채택 또는 접어 둠. 이유 필수	notes:write
POST	/branches/{id}/reopen	다시 열기(이유는 지웁니다)	notes:write
DELETE	/branches/{id}	갈래만 삭제(생각은 남습니다)	notes:write
PUT	/notes/{id}/branch	생각을 갈래에 넣기/빼기(null 이면 빼기)	notes:write
GET	/turning-points	표시한 것만 시간순으로: 채택·접어 둠·답함·상충	notes:read

resolve 는 이유가 비어 있으면 400 branch-resolution-required 로 거부합니다. 결정만 남고 까닭이 사라지는 것이 이 기능이 막으려는 일이기 때문입니다. 접어 둔 갈래에 속한 생각은 /search 의 noteLines , /ai/ask 의 sources[].branch , 마크다운 내보내기, MCP note_lines 에 모두 표시됩니다.

🌙 Dream 검토함

메서드	경로	설명	권한 스코프
GET	/dreams	출처와 상태를 포함한 개인 Dream 검토함 조회	dreams:read
POST	/dreams/{id}/feedback	노출·채택·숨김 등의 무중복 피드백 기록	dreams:read
POST	/dreams/{id}/accept	후보를 Dream 메모와 출처 연결선으로 확정	dreams:read , notes:write
POST	/dreams/{id}/regenerate	같은 출처에서 중복되지 않는 다른 관점 생성	dreams:read
POST	/dreams/{id}/develop	확장·반대 관점·실행 항목으로 발전	dreams:read
POST	/dreams/{id}/developed-note	발전 결과를 새 메모와 expanded 연결선으로 원자 저장(동일 재시도 무중복)	dreams:read , notes:write

Today의 대기 Dream과 GET /dreams 의 후보·출처는 매 조회마다 Dream이 속한 공간의 현재 owner/member 권한을 확인합니다. 자동 Dream의 원본 선별도 현재 접근 가능한 사용자 작성 생각만 허용하고, 최종 source lease 안에서 자격을 다시 확인합니다. 공간 공유가 회수되면 파생 본문·공간명·원본이 즉시 목록에서 빠지고 보관한 Dream ID로 피드백·숨김·채택·재생성·발전을 요청해도 상태나 개인화를 바꾸지 않습니다. 피드백은 Dream 행과 membership을 같은 transaction에서 잠급니다. 자동 생성·재생성·발전은 AI 제외되지 않은 원본 note, 관련 공간과 현재 membership을 정렬된 순서로 잠그며 Dream 기반 호출은 Dream 행과 공간도 포함한 뒤 그 lease 안에서만 Gateway를 호출합니다. 회수가 먼저 끝나면 외부 호출 전 쿼터 예약을 취소하고 본문을 보내지 않으며, lease가 먼저 시작되면 회수 transaction이 호출 종료 뒤까지 기다립니다. 자동 생성 후보·source·알림과 채택·발전 결과 저장도 각

열린 transaction 안에서 함께 확정합니다. 별도 pool 연결로 권한을 다시 조회하지 않으므로 작은 pool에서도 자신이 보유한 transaction 연결을 기다리지 않습니다.

`GET /notifications` 의 각 항목에는 `resourceType`, `resourceId`, `resourceSpaceId`, `metadata` 가 포함됩니다. Dream 알림은 `/dreams?focus={resourceId}`, 공간 공유 알림은 `/space/{resourceId}`, 댓글·멘션은 `/space/{resourceSpaceId}?note={resourceId}` 로 연결할 수 있습니다. note 알림은 현재 생각의 soft-delete 상태와 실제 공간 접근권한을 목록·unread count 양쪽에서 확인하며, 이전 행에 `resourceSpaceId` 가 없어도 생각이 속한 공간으로 권한을 계산합니다. Dream 알림도 현재 공간 ID를 저장하고 이전 행은 Dream에서 실제 공간을 찾아 공유 회수 뒤 본문을 숨깁니다. unread count와 page 조회는 DB 연결을 겹쳐 잡지 않고 순차 실행합니다.

`pool_max_conns` 1~2에서는 realtime listener도 안전 폴링으로 전환하므로 endpoint가 자신이 필요한 request 연결을 기다리지 않습니다.

개인 설정과 API key 생성·회전·폐기, 세션 관리, 모든 `/admin/*` 작업은 API key가 아니라 로그인한 브라우저 세션에서만 허용됩니다. 제한된 자동화 key가 새 자격 증명을 만들거나 관리자 role을 상속해 권한을 넓힐 수 없습니다. OIDC·AI Gateway 설정에서 마스킹된 secret을 그대로 보내면 서버가 저장 직전의 최신 암호문을 설정별 transaction lock 안에서 병합하며, master-key 회전도 같은 lock을 사용하므로 회전 전에 연 관리 화면의 저장이 새 암호문을 되돌리지 않습니다.

`GET /metrics` 는 별도의 운영 경계입니다. 관리자 브라우저 세션 또는 명시적으로 `metrics:read` 를 가진 API key만 Prometheus text를 조회할 수 있습니다. 일반 사용자 세션에 내부적으로 부여되는 wildcard와 관리자 계정이 발급한 `notes:read` 같은 일반 key는 이 권한으로 승격되지 않으며 `403` 을 반환합니다.

`PUT /preferences` 는 원자적 부분 수정입니다. 바꾸려는 필드만 보내면 되고, 보내지 않은 필드는 요청이 처리되는 시점의 최신 저장값이 유지됩니다. 언어와 테마처럼 서로 다른 설정을 동시에 바꿔도 한 요청이 다른 요청을 되돌리지 않으며, `dream_pause_until: null` 은 일시정지를 명시적으로 해제합니다.

🔑 로그인한 기기

메서드	경로	설명	권한
GET	<code>/sessions</code>	이 계정에 로그인된 브라우저 세션 목록. 요청을 보낸 세션은 <code>current</code> 로 표시	브라우저 세션
DELETE	<code>/sessions/{id}</code>	특정 세션 종료. 현재 세션을 종료하면 쿠키도 함께 지워짐	브라우저 세션
POST	<code>/sessions/revoke-others</code>	이 브라우저를 제외한 모든 로그인 종료	브라우저 세션

세션 토큰은 어떤 응답에도 포함되지 않습니다.

세션의 `userAgent` 는 표시·감사용 정보이며 서버가 유효한 UTF-8의 최대 300 byte로 정규화합니다. 다중 byte 문자는 rune 경계에서만 잘리므로 긴 User-Agent 때문에 올바른 로그인 자체가 실패하지 않습니다.

Canvas 목록·수정 이력·댓글·백링크처럼 본문을 반환하는 조회는 사전 권한 확인 결과를 재사용하지 않고, 콘텐츠를 읽는 PostgreSQL statement 자체에서 현재 공간 owner/member 조건을 확인합니다. 멤버가 제거된 뒤 시작된 조회는 빈 본문을 반환하지 않고 404로 종료됩니다.

댓글 작성은 생각과 공간을 확인한 뒤 호출자의 현재 멤버십 행을 transaction 종료까지 잠급니다. 작성과 권한 회수가 겹치면 먼저 잠금을 얻은 작업이 끝난 뒤 다음 작업이 진행되며, 권한 회수가 먼저 확정된 요청은 댓글·알림·실시간 이벤트·webhook outbox를 만들지 않고 404로 종료됩니다. 댓글 해결·재개·삭제 transaction도 대상 댓글·활성 생각·공간과 비소유자의 현재 membership 권한 행을 commit까지 잠급니다. 멤버 제거 또는 `edit/manage → view` 강등이 먼저 확정되면 mutation을 거부하고, mutation이 먼저 잠금을 얻으면 상태·공간 이벤트·webhook outbox가 원자적으로 확정된 뒤 권한 변경을 진행합니다. 생각이 soft-delete되었거나 공간에서 제거된 뒤 보관한 댓글 ID로 해결·재개·삭제를 요청하면 `comment-mutation-forbidden` 403 Problem Details로 종료해 행·이벤트·webhook을 바꾸지 않습니다. 이 타입은 재시도해도 적용할 수 없는 오프라인 변경임을 뜻합니다. `review_digest=false` 는 `/today` 의 협업 활동만 제외하고 고정·다시 보기 기한·오래된 생각의 기본 검토 목록은 유지합니다.

`/search`, `/notifications`, `/dreams`, `/admin/audit` 는 응답의 `nextCursor` 를 다음 요청의 `cursor` 에 그대로 전달합니다. `cursor` 내부 형식에 의존하지 마세요.

하이브리드 검색은 접근 가능한 전체 메모에서 정확 제목·정확 본문·제목 구문·본문 구문 순으로 키워드 후보를 제한한 뒤 최근 비키워드 후보에 의미 점수를 결합합니다. 따라서 광범위한 단어가 500개 넘는 본문에 포함되거나, 메모가 2,000개를 넘거나, 검색어가 본문 2,000자 이후에 있어도 오래된 정확 결과를 최신 부분 일치 때문에 누락하지 않습니다.

키워드 조건은 `pg_trgm` 인덱스를 사용하며 검색어는 최대 8개 단어까지 반영됩니다(그 이상은 무시되어 결과가 좁아지지 않고 넓어집니다). 한 검색어의 모든 단어는 메모 본문 쪽 또는 공간 이름 쪽 한 곳에서 모두 일치해야 하며, 양 쪽에 나뉘어 걸친 조합은 의미 점수 후보로 넘어갑니다.

요청 한도

상황	응답	problem type
호출자별 API 요청 한도 초과	429 + Retry-After	rate-limited
AI 생성 분당 한도 초과	429 + Retry-After	ai-rate-limited
AI 생성 하루 한도 초과	429 + Retry-After	ai-daily-limit
AI 쿼터 저장소 일시 오류	503 + Retry-After	ai-quota-unavailable
로그인 실패 반복으로 잠김	429 + Retry-After	login-locked

한도 값은 관리자 설정에서 조정합니다. 구버전 관리 클라이언트가 새 남용 방지 필드를 생략한 채 보안 섹션을 PUT 해도 서버는 PostgreSQL transaction lock 아래 최신 저장값에서 생략 필드만 병합합니다. 명시한 0은 하루 AI 무제한으로 적용되고 `null` 은 잘못된 설정으로 거부됩니다. 로컬 로그인은 정렬된 주소·계정 advisory lock 아래 잠금 확인부터 비밀번호 검증과 실패 기록 또는 session 저장까지 같은 PostgreSQL 연결에서 끝내므로, 여러 replica의 병렬 요청도 `login_max_failures` 보다 많은 추측을 통과시키지 않고 작은 pool에서도 추가 연결을 기다리지 않습니다. 일일 AI 한도는 실제 Gateway 호출 전에 PostgreSQL에서 원자적으로 예약한 뒤 24시간 원장으로 영속화하며, 자동 Dream과 관리자 AI 평가도 같은 원장을 사용합니다. 따라서 replica가 여러 개거나 요청이 취소되거나 별도 `ai_calls` 로그 저장이 실패해도 동시에 한도를 넘지 않습니다. 쿼터를 영속화할 수 없으면 비용 보호를 위해 AI 호출을 시작하지 않고 503으로 종료합니다. 429와 503은 재시도 가능한 응답이므로 오프라인 queue도 보존한 뒤 다시 시도합니다.

🔑 서명 웹훅

`/webhooks` 에서 이벤트 subscription을 만들면 secret이 한 번만 반환됩니다. 대상은 공개 HTTPS 443이어야 하며 HMAC 입력은 아래와 같습니다.

```
signed = X-Umm-Timestamp + "." + raw_request_body
X-Umm-Signature-256 = "sha256=" + hex(HMAC-SHA256(secret, signed))
```

도메인 변경과 활성 구독별 PostgreSQL outbox는 같은 트랜잭션에서 확정되며, 프로세스가 재시작되어도 워커가 대기 항목을 이어서 처리합니다. 워커는 정확한 delivery claim과 구독·소유 사용자·이벤트 공간·현재 membership을 실제 HTTP 응답까지 하나의 authorization lease로 잠급니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 확정되면 캡처한 payload를 보내지 않고, 전달이 먼저 시작되면 정책 변경은 terminal 상태와 payload 삭제가 확정될 때까지 기다립니다. 이 lease는 일반 request pool 밖의 인스턴스당 최대 3개 연결을 사용합니다. 전달 시도는 at-least-once 방식이므로 수신 측은 `X-Umm-Delivery` 를 먹등 키로 사용해 중복을 무시하고 timestamp 허용 시간도 함께 확인해야 합니다. terminal payload는 즉시 비워지고 delivery metadata는 30일 보존됩니다. 이벤트에는 `space.updated` , `note.*` , `edge.created` , `comment.*` , `member.*` , `dream.accepted` 가 있습니다.

안전한 재시도와 오류

Canvas의 메모·연결·댓글 생성/수정/삭제 요청에 8~128자의 `Idempotency-Key` 를 보내면 같은 사용자와 키의 성공 응답을 24시간 재생합니다. 서버는 method·경로·query·본문 fingerprint가 같은 요청만 재생하며, 다른 요청에 키를 재사용하면 409입니다. 처리 중 2분 lease는 handler가 살아 있는 동안 갱신되고, 아직 처리 중이면 `Retry-After` 가 포함된 425를 반환합니다. 도메인 변경·SSE·webhook outbox·먹등 성공 응답은 같은 PostgreSQL 트랜잭션으로 확정되므로 성공 응답 기록 실패가 mutation 중복을 만들지 않습니다. handler 실행 전 프로세스가 중단되면 lease 만료 후 같은 요청이 예약을 자동 재획득합니다. 오프라인 클라이언트는 한 논리 작업에 같은 키를 유지하고 `Retry-After` 이후 다시 시도해야 합니다. 장시간 AI 요청과 API key·웹훅 서명 key처럼 비밀을 한 번만 공개하는 요청은 `Idempotency-Key` 를 거부합니다.

오류 본문은 `application/problem+json` 이며 `type` , `title` , `status` , `detail` 을 제공합니다. 기존 클라이언트를 위해 `error` 도 같은 설명을 유지합니다. 접근 가능한 메모의 version 충돌만 409와 `clientVersion` , 최신 서버 `latest` 메모를 반환합니다. 메모가 삭제되었거나 공간 접근 권한을 잃은 경우에는 404로 종료합니다.

브라우저 오프라인 queue는 409 충돌과 408·425·429·5xx처럼 다시 시도할 수 있는 응답만 보존하고 일시 오류는 `Retry-After` 또는 기본 5초 뒤 자동 재시도합니다. 400·404 등 영구적인 client 오류는 사용자에게 사유를 알린 뒤 queue에서 제거해 무한 재시도를 막습니다. 메모를 볼 수는 있지만 공간 권한이 `edit` 에서 `view` 로 낮아진 `note-read-only` , 더는 댓글을 바꿀 수 없는 `comment-mutation-forbidden` 403은 일반 인증/권한 403과 구분해 해당 변경만 제거하고 이후 항목을 계속 동기화합니다. flush 도중 다른 탭이나 새 편집이 추가한 항목은 최신 queue와 ID 단위로 병합하고 탭 간 Web Lock으로 저장을 직렬화해 진행 중이던 snapshot이 덮어쓰지 않습니다. 브라우저가 quota 초과나 사이트 권한으로 `localStorage` 쓰기를 거부하면 `offline-storage-unavailable` 오류로 종료하고 `queued=true` 를 반환하지 않습니다. Canvas autosave는 이 비내구성 결과를 받으면 마지막으로 서버 또는 queue에 보존된 메모로 화면을 복원하되, 실패한 요청 중 시작한 새 편집은 별도 저장 시도까지 보존합니다. 읽기 또는 삭제도 예외 경계 안에서 처리해 인증 bootstrap과 queue count 렌더링을 유지하며, 읽을 수 없거나 손상된 기존 queue를 빈 queue로 덮어쓰지 않습니다.

⚡ 실시간 이벤트 스트림 (SSE)

- 경로: `GET /api/v1/spaces/{spaceID}/events`
- 프로토콜: Server-Sent Events (SSE)
- 이벤트 타입: `space-change`
- 용도: 타 사용자가 캔버스에서 포스트잇을 추가/수정/이동할 때 실시간 브로드캐스트 수신
- 재개: 각 이벤트의 `id` 가 `space_events.sequence` 입니다. 재연결 시 `Last-Event-ID` 헤더 또는 `?after=` 쿼리로 마지막 sequence를 전달하면 그 이후부터 이어받습니다.
- 전달 방식: 서버는 PostgreSQL `LISTEN/NOTIFY` 로 깨어나 즉시 전송합니다. 구독자 수와 무관하게 유휴 상태에서는 데이터베이스를 조회하지 않습니다. 수신 연결 상태가 바뀌면 열린 SSE를 즉시 깨우고 cursor 이후를 다시 조회한 뒤, 단절 중에는 1초 폴링으로 전환하고 복구 시 30초 safety net으로 돌아가므로 클라이언트가 알아차릴 필요는 없습니다.

3. Model Context Protocol (MCP) 엔드포인트

- 엔드포인트: `POST /mcp`
- 인증: `Authorization: Bearer <API_KEY>`
- 프로토콜: JSON-RPC 2.0 (Stateless HTTP POST)

🔧 제공 도구 목록 (Tools)

1. `list_spaces` : 접근 가능한 공간 목록 조회
2. `list_notes` : 지정된 공간의 생각 포스트잇 및 연결선 조회
3. `create_note` : 캔버스에 새로운 생각 포스트잇 생성
4. `connect_notes` : 두 생각 간의 연결선 생성
5. `search_notes` : 지정 공간의 생각 검색
6. `get_related_notes` : 오프라인 유사도 기반 연관 생각 조회
7. `list_clusters` : 생각 군집 조회
8. `list_dreams` : 출처와 검토 상태를 포함한 최근 Dream 조회
9. `list_lines` : 생각의 갈래 목록과 각 갈래의 상태·이유
10. `find_open_questions` : 열린 것으로 표시된 질문 (표시된 것이지 추론이 아닙니다)
11. `find_contradictions` : 상충으로 표시된 쌍
12. `get_connections` : 한 생각에 붙은 연결과 그 출처
13. `capture_thought` : 공간을 정하지 않고 수집함에 담기

`list_notes` · `search_notes` 는 `note_lines` 를 함께 돌려줍니다. `status` 가 `abandoned` 이면 그 생각은 사람이 검토한 뒤 채택하지 않기로 한 갈래에 속하며 `resolution` 에 이유가 옵니다 — 현재 방침처럼 다루면 안 됩니다. 갈래·질문·상충은 모두 사람이 표시하므로, 빈 결과는 표시된 것이 없다는 뜻이지 아무것도 없다는 뜻이 아닙니다.

💬 MCP 호출 예시 (JSON-RPC 2.0)

요청: 생각 포스트잇 생성

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "method": "tools/call",
  "params": {
    "name": "create_note",
    "arguments": {
      "space_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "content": "MCP 엔드포인트를 통해 Cursor나 Claude에서 직접 생각을 붙이고 연결할 수 있습니다.",
      "color": "lavender",
      "x": 400,
      "y": 250
    }
  }
}
```

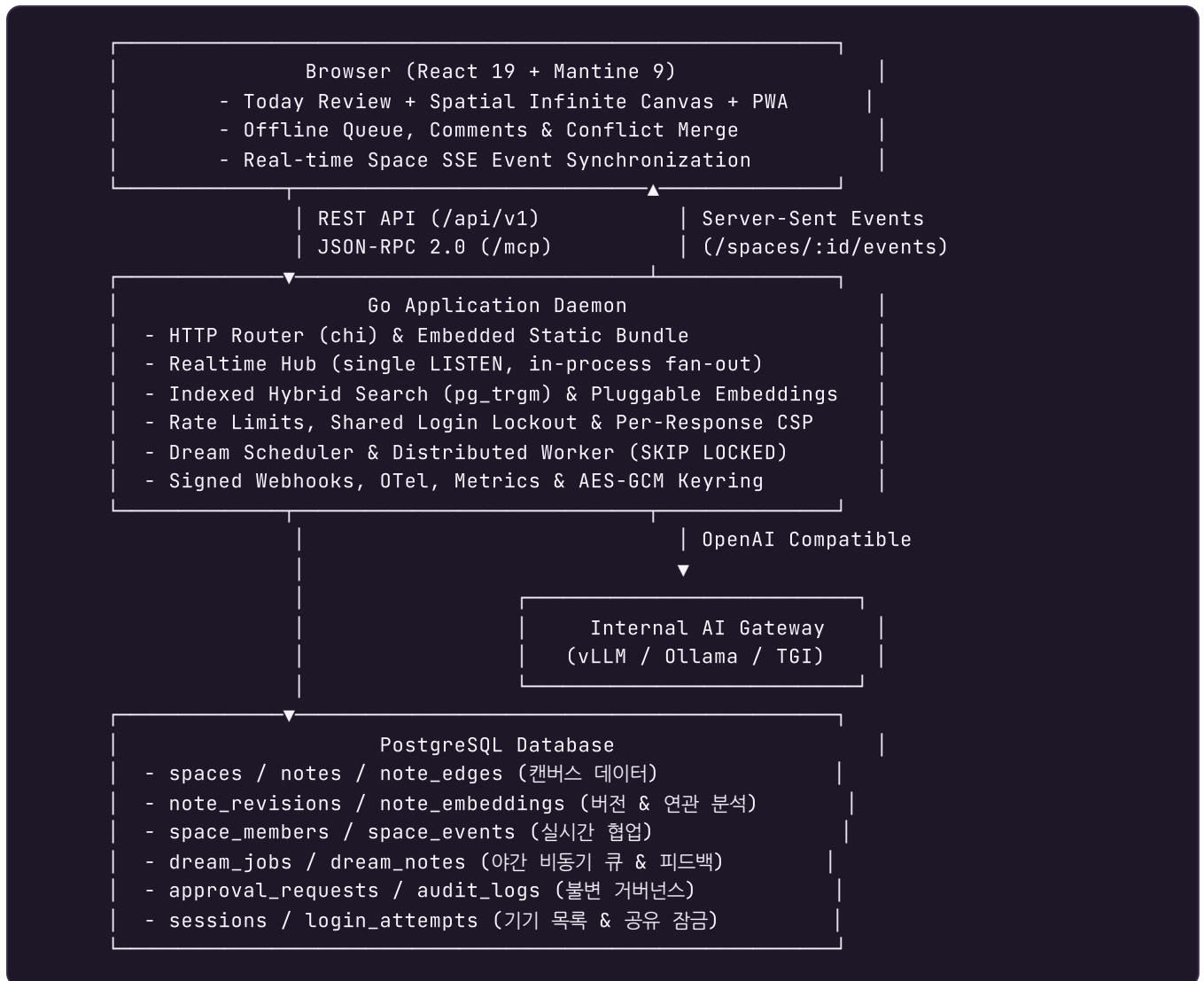
응답

```
{
  "jsonrpc": "2.0",
  "id": "req-101",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "생각 포스트잇이 성공적으로 생성되었습니다. (ID: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d)"
      }
    ]
  }
}
```

umm 실행 아키텍처 및 불변 조건 (Architecture)

umm은 단일 Go 바이너리와 단일 PostgreSQL 인스턴스만으로 오프라인 폐쇄망에서 무중단 운영이 가능하도록 설계된 **Spatial Thought Memory** 엔진입니다.

1. 시스템 아키텍처 다이어그램



2. 데이터 경계 및 도메인 분리

- **Canvas Domain:** spaces, notes, note_edges
- **Intelligence Domain:** note_revisions, note_embeddings
- **Collaboration Domain:** space_members, space_events, notifications, note_comments, comment_mentions

- **Identity & Access:** `users`, `sessions`, `oauth_states`, `teams`, `login_attempts`, `ai_quota_reservations`
- **Personalization & Review:** `user_preferences`, `note_reviews`, `api_keys`
- **Governance & Security:** `approval_requests`, `audit_logs`, `app_settings`
- **Dream & LLM Domain:** `dream_jobs`, `dream_notes`, `dream_sources`, `dream_feedback`, `ai_calls`, `ai_eval_cases`, `ai_eval_runs`
- **Automation & Reliability:** `webhook_subscriptions`, `webhook_deliveries`, `idempotency_records`, `product_events`

3. 핵심 아키텍처 원칙

1. 외부 의존성 없는 의미 연관성 분석 (Offline Semantic Engine)

- 연관 생각(Related Thoughts)과 클러스터링은 기본적으로 **192차원 문자 n-gram feature hashing**과 **cosine similarity**로 계산됩니다. 외부 다운로드나 API 호출이 없어 폐쇄망에서 100% 동작합니다.
- v0.8.0부터 관리자가 AI Gateway에 임베딩 모델을 지정하면 OpenAI 호환 `/v1/embeddings`를 대신 사용합니다. 모든 벡터에는 모델명과 정규화된 Gateway endpoint의 SHA-256 fingerprint로 만든 알고리즘 식별자가 함께 저장되고 **같은 알고리즘끼리만 비교**합니다. 모델 또는 Gateway를 바꾸면 같은 모델명을 유지해도 서로 다른 벡터 공간이 뒤섞이지 않고 점진적으로 다시 임베딩됩니다. API key와 timeout은 벡터 공간을 바꾸지 않으므로 fingerprint 대상에서 제외합니다. 원격 응답과 장애 뒤 비교 집합을 로컬로 통일하는 후속 pass를 저장하는 transaction은 모두 작업 시작 시점의 `ai_gateway` 설정 세대로 현재 설정 행을 shared lock 확인하므로, 두 pass 사이 또는 호출 중 설정이 바뀌어도 이전 결과가 새 Gateway의 벡터를 덮어쓸 수 없습니다. 외부 임베딩 직전에는 같은 설정 행과 대상 note·space의 `ai_excluded` 행을 다시 잠그고 응답 저장이 끝날 때까지 유지합니다. 제외 설정이 먼저 확정되면 로컬 vector로 전환하고, lease가 먼저 시작되면 정책 변경이 외부 호출 종료까지 기다리므로 point-in-time 확인 뒤 캡처한 본문이 새 정책을 넘어 전송되지 않습니다. 콘텐츠 version이 낮은 벡터도 알고리즘이 다르다는 이유만으로 최신 행을 교체하지 못합니다.
- 게이트웨이 호출이 실패하면 로컬 벡터로 내려가되 저장되는 알고리즘 이름은 로컬로 기록됩니다. 실패한 호출이 성공한 것처럼 남는 경로는 없습니다. 공간으로 범위를 제한한 검색은 접근 권한, 공간 제외, 활성 메모의 개별 `ai_excluded`를 먼저 함께 확인합니다. 하나라도 제외되면 Canvas가 정규화한 것과 같은 로컬 비교 공간을 선택하고 저장된 원격 vector가 없는 후보도 응답 본문에서 로컬 임베딩해 의미 검색을 유지합니다. 원격 검색을 선택한 경우에는 space·membership·활성 note 행을 query embedding과 검색 row 소비가 끝날 때까지 잠급니다. 제외 또는 접근 회수가 먼저 확정되면 검색어도 원격 Gateway로 보내지 않습니다. 이 임베딩 정책 lease는 Dream·AI Assist와 같은 인스턴스당 2-slot AI 전용 연결을 사용해 느린 Gateway가 request pool을 점유하지 않습니다. `enc:` Gateway API key를 복호화할 수 없으면 ciphertext를 credential로 사용하지 않고 provider 전체를 로컬로 닫습니다.

1-1. 푸시 기반 실시간 협업 (Realtime Hub)

- `space_events`의 AFTER INSERT 트리거가 `pg_notify`를 호출합니다. PostgreSQL은 알림을 커밋 시점에 전달하므로, 롤백된 변경이 다른 사람에게 보이는 상태는 만들어질 수 없습니다.
- 인스턴스마다 전용 연결 하나가 `LISTEN umm_space_events`를 유지하고, 도착한 알림을 프로세스 안의 구독자에게 팬아웃합니다. 구독자 수와 무관하게 데이터베이스 부하는 일정합니다. 단 `pool_max_conns`가 1 또는 2이

면 전용 리스너를 시작하지 않고 기존 1초 안전 폴링을 사용해 request pool의 모든 연결을 readiness·인증·transaction 요청에 남깁니다. 상한 3부터 listener를 시작하면서 request용 두 연결을 보존합니다. 목록과 unread count를 함께 반환하는 알림 endpoint는 count와 rows를 순차 실행해 request 연결 하나만 사용하고, 짧은 Dream 채택 transaction도 같은 연결에서 권한을 확인합니다. 외부 호출 동안 유지하는 access lease는 request pool과 분리해 AI 최대 2개, 웹훅 최대 3개로 각각 제한하므로 인스턴스의 최악 연결 상한은 `pool_max_conns + 5` 입니다.

- 알림은 페이로드를 신뢰하지 않습니다. 깨어난 리더는 자신이 마지막으로 보낸 sequence 이후를 다시 조회하므로, 알림이 합쳐지거나 유실되어도 이벤트를 건너뛰지 않습니다.
- 리스너가 끊기면 지수 백오프로 재연결합니다. 연결 상태 전환 자체가 모든 SSE 구독자를 coalesced 신호로 즉시 깨우므로, 리더는 기존 30초 safety-net deadline을 기다리지 않고 cursor를 따라잡은 뒤 타이머를 1초 폴링으로 재설정합니다. 재연결 신호에서는 다시 30초 안전망으로 돌아가며 협업이 멈추는 구간이 없습니다.

1-1-1. 백엔드에 독립적인 유사도 판정

- 연관 생각·군집·검색 라벨·Dream 페어 선택은 더 이상 고정 cosine 컷오프를 쓰지 않습니다. 각 후보 집합의 관측 분포에서 **평균 위 표준편차 단위**로 기준을 잡습니다(`intelligence.SimilarityScale`).
- 이유는 측정으로 확인됐습니다. 로컬 문자 n-gram은 무관한 쌍을 0.18 부근에, 강한 일치를 0.34 부근에 놓지만, 문장 임베딩 모델은 무관한 쌍조차 0.36 위에 놓습니다. 같은 상수가 한쪽에서는 합리적이고 다른 쪽에서는 워크스페이스 전체를 통과시킵니다.
- Dream의 bridge 향도 같은 이유로 상대화했습니다. 고정 정점 0.35는 유사도 0.70 이상에서 정확히 0을 반환하므로, 임베딩 모델을 붙이면 가중치 55%의 주 신호가 통째로 사라졌습니다.
- 표본이 4개 미만이거나 분포에 퍼짐이 없으면 예전 상수로 물려칩니다. 메모가 두세 개뿐인 공간의 동작은 바뀌지 않습니다.
- 데이터셋과 채점은 `internal/intelligence/quality.go` 에 있고, CI 리포트와 관리자 화면이 **같은 코드**를 씁니다. `quality_test.go` 는 그 값을 하한으로 고정해 나빠지면 실패시킵니다. 후보 모델은 `UMM_EMBEDDING_TEST_URL` / `UMM_EMBEDDING_TEST_MODEL` 로 같은 기준에서 비교합니다.

0-8a. 기억에 묻기: 근거 검색과 답변

- 답변은 두 단계이고 **검색이 먼저 분리되어 있습니다**. 무엇을 읽는지가 답을 근거 있게 만들 수 있는지를 결정하고, LLM 없이 검증 가능한 유일한 절반이기 때문입니다.
- 검색만 하지 않고 **그래프를 한 걸음 따라갑니다**. 질문의 답은 질문을 던진 생각 안이 아니라 옆에 적혀 있는 경우가 많습니다(실측: 답 노트가 질문과 단어를 하나도 공유하지 않아 연결로만 도달).
- `ai_excluded` 는 세 경로 모두에서 막습니다 — 노트 단위, 공간 단위, 그리고 이웃 탐색. 검색에서 막고 이웃에서 끌어오면 더 긴 경로로 같은 유출입니다. umm은 이 결함을 v0.8.0에서 실제로 냈고, 검색 계층이 그게 맞히기 가장 쉬운 자리입니다.
- 제외된 것은 **개수를 보고합니다**. 기대보다 적은 자료로 만든 답이 완전해 보이면 안 되고, 내용을 드러내지 않고 알릴 방법은 숫자뿐입니다.
- 근거가 없으면 **모델을 호출하지 않습니다**. 근거 없는 답이 가장 그럴듯하고 가장 틀리는 경우가 그때입니다.
- 모델 미설정(412)과 게이트웨이 실패(502)를 구분합니다**. 앞의 것은 관리자가 1분이면 고치고 뒤의 것은 아닙니다.
- 답변 로직이 `internal/dream` 에 있는 이유는 게이트웨이 클라이언트·할당량·호출 로그가 거기 있기 때문입니다. 패키지 이름은 역사적인 것이고 실제로는 AI 계층입니다.

0-8b. 조력자: 살펴본 뒤 답하기

- 한 번의 검색으로 답하는 것과 **찾아보고 읽고 다시 찾아보는 것**을 나눴습니다. 뒤의 것은 다음에 무엇을 볼지 모델이 정하므로 결과가 좋아질 수도, 엉뚱해질 수도 있습니다. 그래서 두 모드가 나란히 있고 사람이 고릅니다.
- **읽기 전용을 플러그가 아니라 도구 목록으로 구현했습니다.** 쓰기 도구를 붙여 두고 실행 직전에 권한을 확인하는 대신, 애초에 바인딩하지 않습니다. 모델이 부르고 싶어도 부를 것이 없고, 틀리려면 누군가 의도적으로 도구를 추가해야 합니다. 통합 테스트는 Go의 도구 목록이 아니라 **게이트웨이로 실제 전송된 JSON**을 검사합니다.
- 조회는 전부 `/ai/ask` 와 같은 **store** 경로를 씁니다. 접근 규칙과 AI 제외를 다시 서술하지 않아야 어긋나지 않습니다.
- **마지막 턴은 도구 없이 보냅니다.** 도구를 쥐여 준 채 "그만 찾고 답하라"고 하면 모델은 도구를 한 번 더 부르고 사람은 빈 답을 받습니다. 만들면서 실제로 그랬습니다.
- **한 턴은 요청한 사람의 할당량에서 하나 차감합니다.** 이 자리도 처음엔 틀렸습니다 — 빈 사용자에게 차감했고, 빈 사용자는 할당량 검사를 건너뛰므로 한 요청에 공짜 호출 6번이었습니다. 회귀 테스트가 실제 차감 건수를 셉니다.
- 조회 과정을 그대로 돌려줍니다. 살펴본 끝에 나온 답이 추측보다 나은 근거는 **어디를 봤는지 볼 수 있다는 것**뿐입니다.

0-8c. 갈래: 접어 둔 생각이 현재처럼 읽히지 않게

- 문제는 어수선했음이 아니라 **이미 버린 선택지를 다시 집어드는 것**입니다. 검색 결과에서 접어 둔 생각과 현재 생각은 생김새가 같습니다.
- **표시는 사람이 합니다.** 한 달 동안 아무것도 안 적힌 갈래를 umm이 알아서 접지 않습니다. 침묵은 결정이 아니고, 그렇게 처리하면 잠시 멈춰 둔 일이 조용히 묻힙니다. `상충` · `질문` 과 같은 규칙입니다.
- **정리에는 이유가 필수입니다.** 스키마의 CHECK 제약과 API 양쪽에서 막습니다. 이유 없는 결정 기록은 이 기능이 막으려는 망각이 한 단계 뒤로 미뤄진 것입니다.
- 표시는 **두 조회 경로 모두에 붙입니다.** 직접 일치한 생각과 연결을 따라 도달한 생각 — 뒤쪽이 더 중요합니다. 아무도 그걸 보겠다고 고르지 않았기 때문입니다.
- **표시 문구가 뜻을 결정합니다.** 처음 쓴 `[보류된 갈래]` 를 실제 모델은 "아직 결정되지 않음"으로 읽어, 사용자가 버리기로 **결정한 것**을 미결로 보고했습니다. 지금은 채택하지 않기로 했다는 사실과 **그 이유**를 함께 씁니다. 테스트는 문구가 아니라 "보류"라는 단어 자체를 금지합니다.
- **Note 에 컬럼을 붙이지 않았습니다.** 노트를 읽는 경로가 17군데이고, 한 군데를 빠뜨리면 v0.11.0에서 실제로 낸 결함(`Backlinks` 가 `origin` · `confidence` 없이 엣지를 반환)이 그대로 재현됩니다. 갈래 정보는 필요한 곳에 따로 붙입니다.

0-8d. 렌즈: 한 갈래만 보기

- **흐리게 두되 숨기지 않습니다.** 사라진 생각은 지워진 생각으로 읽히고, 캔버스가 실제보다 단출해 보입니다. 흐리게 둔 개수를 항상 함께 말합니다.
- **의미가 아니라 구조로 좁힙니다.** 유사도 기반 렌즈는 뒤에 있는 임베딩만큼만 좋고, 기본 백엔드는 글자 n-gram이라 단어가 겹치는 것을 주제라고 부릅니다. 갈래와 연결은 사람이 그은 것이라 렌즈는 그 사람이 만든 구조만큼 정확하고, 백엔드가 약해도 조용히 나빠지지 않습니다.
- **흐림은 카드가 아니라 노드 바깥 요소에 겁니다.** `.postit` 은 `animation: note-land ... both` 를 갖고 있고, 애니메이션의 마지막 키프레임은 캐스케이드에서 인라인 스타일을 이깁니다. 카드에 `opacity` 를 걸면 DOM에

는 값이 보이지만 **화면에는 한 번도 그려지지 않습니다**. 실제로 그렇게 만들었고, 인라인 속성을 읽는 검증이 그걸 통과시켰습니다.

- 그래서 e2e는 `toHaveCSS` 로 계산된 스타일을 봅니다. 속성을 읽는 검사는 그리지도 않는 효과를 통과시킵니다.
- 그래프 탐색은 `web/src/lens.ts` 로 분리했습니다 — 렌즈에서 틀릴 수 있는 유일한 부분입니다. 연결은 **양방향**으로 따라갑니다. 화살표만 따라가면 캔버스가 바로 옆에 보여 주는 백링크가 빠집니다.

0-8e. 결정 기록

- 여섯 달 뒤의 질문은 "무엇을 적었나"가 아니라 *****어떻게 하기로 했고 왜인가*****입니다. **왜**가 가장 먼저 사라지므로 이유는 결정과 같은 화면에 둡니다.
- **표시한 것만 들어갑니다**. 활동이 몰린 주를 전환점이라 부르지 않고, 조용한 갈래를 접힌 것으로 처리하지 않습니다. 결정과 추측이 섞인 기록은 기록이 없는 것보다 나쁩니다 — 필요할 때 어느 쪽인지 구분할 수 없습니다.
- **진행 중인 갈래는 제외합니다**. 정해진 것이 없고, 넣으면 결정 기록이 할 일 목록이 됩니다.
- **정렬은 SQL에서 합니다**. 종류별로 최근 N개를 가져와 Go에서 합치면 혼한 종류가 드문 종류의 오래된 항목을 밀어내고, 그 해에는 아무것도 결정하지 않은 것처럼 읽힙니다.
- 비어 있을 때 "표시된 것이 없다는 뜻이지 아무 일도 없었다는 뜻은 아닙니다"라고 말합니다 — `find_open_questions` 와 같은 규칙입니다.

0-8f. 접어 둔 결정이 되 돌아오는 자리

- 결정 기록은 **찾아봐야** 보입니다. 정작 필요한 순간은 **다시 쓰고 있을 때**이므로, 근접 중복 한쪽이 접어 둔 갈래에 있으면 오늘의 리뷰가 결정과 **이유를 함께** 내놓습니다. "그거 안 하기로 했잖아"만 남으면 왜인지 모른 채 다른 이유로 다시 거절하거나 아예 거절하지 않게 됩니다.
- **여기서만 "하나로 합치기"를 뺍니다**. 합치면 먼저 적은 쪽이 남고 그쪽이 접어 둔 생각입니다. 한 번 누르면 지금 쓰는 생각이 이미 거절한 갈래로 조용히 접혀 들어가고, 거절된 쪽이 남습니다. 버튼 하나로 할 일이 아닙니다.
- **정확히 한쪽만** 접어 둔 갈래일 때만 표시합니다. 양쪽 다면 그냥 중복 두 개이고, 결정이 둘 다를 덮고 있습니다.
- 이 기능은 umm에서 **유일한 절대 임계값**(근접 중복 0.92)에 기대므로, 의미를 재는 백엔드에서만 동작하고 기본 임베딩에서는 오늘의 리뷰가 이미 그렇듯 "재지 않았음"으로 건너웁니다.

0-9c. 질문과 답

- `notes.kind` 는 요청 본문에서 **검증 없이** 들어오고 있었습니다 — v0.11.0 이전의 `relation` 과 같은 결함이고, 5000자 문자열도 저장됐습니다. 아무것도 읽어 오지 않아 아무도 몰랐습니다. 이제 `thought·question·idea` 어휘이고, 어휘 밖은 조용히 바꾸지 않고 거부합니다.
- **만들기와 수정 두 경로 모두 검증합니다**. 한쪽만 막으면 다른 쪽이 우회로로 남고, 그게 검증되지 않은 필드가 고쳐진 뒤에도 살아남는 방식입니다.
- `answers` 가 질문을 답습니다. `supports` · `refines` 는 가깝지만 달지 않습니다 — 그중 하나를 답으로 치면 논쟁만 한 노트가 질문을 해결한 것처럼 보입니다.
- **질문도 답도 표시하는 것이지 추론하는 것이 아닙니다**. 물음표로 끝나는 문장이 질문이 아닌 경우도, 물음표 없는 질문도 많습니다. 빈 결과는 "다 답했다"가 아니라 "표시된 게 없다"이고, 0일 때 화면에 숫자를 띄우지 않습니다.
- `attempts` 는 질문을 가리키지만 답은 아닌 연결 수입니다. 아무도 건드리지 않은 질문과 한참 논쟁하고도 결론이 안 난 질문은 다른 상황입니다.

0-9b. 기록된 상충

- `contradicts` 연결은 v0.11.0부터 만들 수 있었지만 아무것도 읽어 오지 않았습니다. `Contradictions` 가 그것을 돌려주고, 브리핑·API·MCP `find_contradictions` 가 소비합니다.
- **탐지가 아니라 기록입니다.** umm은 두 노트를 읽고 모순이라고 판단하지 않습니다. 그래서 빈 결과는 "없음"이 아니라 "아무도 표시하지 않음"이고, **0일 때 화면에 숫자를 띄우지 않습니다** — "상충 0"은 워크스페이스에 모순이 없다는 뜻으로 읽힙니다.
- 상충에는 **방향**이 있습니다(source가 target을 반박). 나오는 길에 뒤바뀌면 누가 무엇을 반박하는지가 뒤집히므로 테스트로 고정합니다.
- 삭제된 생각과 접근할 수 없는 공간은 제외합니다.

0-9a. 생각 합치기

- 노트를 참조하는 테이블이 **여덟 개**라, 합치기는 캐스케이드에 맡기면 조용히 데이터를 잃습니다. 각각을 명시적으로 처리합니다: 연결·덧글·Dream 인용은 따라가고, 거절 기록은 짝이 바뀌었으므로 삭제하고, **복습 일정과 리비전은 남습니다** — 일정은 그 사람이 본 카드의 회상 타이밍이고 리비전은 사라질 행의 역사이므로, 옮기면 살아남은 생각에게 일어나지 않은 일을 기록하게 됩니다.
- 남는 쪽의 **임베딩도 지웁니다.** 내용이 바뀌었으므로 저장된 벡터는 더 이상 없는 문장을 설명합니다.
- **합쳐진 문구는 호출자가 넘깁니다.** umm은 두 노트가 거의 같다는 것은 알아도 어느 표현을 남길지는 모르고, 이어 붙이면 같은 말을 두 번 하는 노트가 됩니다.
- 자기 자신·없는 노트·빈 내용·쓰기 권한 없는 공간·**다른 공간**을 거부합니다. 마지막은 연결이 양 끝을 같은 공간에 요구하기 때문입니다.
- 같은 쌍을 반대 방향에서 합치는 두 요청이 교착하지 않도록 두 행을 **고정된 순서**로 잠급니다.

0-9. 아침 브리핑과 중복 탐지

- 브리핑은 **umm이 실제로 만들어 낸 것만** 셉니다. 모순이나 "중요도 상승" 항목이 없는 이유는 탐지하지 않기 때문입니다 — 세지 않은 항목 옆의 `0` 은 "없다"로 읽히지 "아무도 안 봤다"로 읽히지 않습니다.
- 그래서 응답에 `skipped` 가 있습니다. 빈 `duplicates` 는 백엔드에 따라 뜻이 다르고, `quiet` 는 **보고할 것도 건너된 것도 없을 때만** 참입니다.
- 중복 판정은 umm에서 **유일한 절대 코사인 임계값**입니다. 다른 모든 기준이 상대인 이유는 백엔드마다 "가깝다"가 다르기 때문인데, 거의 같은 글은 어떤 임베딩 공간에서도 맨 위에 오고 두 모델이 같은 지점에 둡니다(측정: bge-m3 0.943+, paraphrase-multilingual 0.954+, 그다음 등급 최대 0.681/0.581).
- 상대 기준은 **여기서 오히려 틀립니다.** 중복이 많은 공간은 자기 분포가 올라가, 중복이 가장 많을 때 유난해 보이지 않게 됩니다.
- 내장 알고리즘은 이 판정에서도 관문에 걸립니다. 거의 같은 글 0.505–0.889와 어휘 함정 0.449–0.572의 **범위가 겹쳐** 분리되지 않습니다.

1-0. 수집과 정리

- 수집함은 소유자당 하나인 **평범한 공간**입니다(`spaces.is_inbox` , 부분 유니크 인덱스). 검색·임베딩·Dream·연결이 전부 그대로 동작하므로 하위 계층이 두 번째 종류의 저장소를 알 필요가 없습니다. 삭제는 거부합니다 — 지우면 다음 수집이 조용히 새로 만들면서 이전 것을 잃습니다.

- `/capture` 는 공간 id를 받지 않습니다. 그게 요점입니다. 재시도 키 허용 목록에 포함되어 오프라인에서도 담기고 재시도가 사본을 만들지 않습니다.
- `MoveNote` 는 출발지와 목적지 **양쪽의 쓰기 권한**을 각각 확인합니다. 연결은 공간에 속하고 양 끝이 그 안에 있어야 하므로 옮기는 노트는 연결을 가져갈 수 없고, 삭제한 개수를 반환해 호출자가 **미리** 알릴 수 있게 합니다.
- `SuggestSpaces` 는 임베딩이 의미를 판별한다고 측정됐을 때만 `basis=meaning` 을 주장하고, 아니면 `basis=recent` 로 물러섭니다. 어휘 중복을 이해인 것처럼 내놓는 것보다 한계를 밝히는 편이 낫습니다.
- `loadEmbeddings` 는 저장된 벡터만 읽습니다. 벡터는 공간 조회 시 만들어지므로, 비교 대상 노트의 공간을 조회하지 않으면 벡터가 없고 모든 코사인인 0이 됩니다 — 정렬처럼 보이지만 정렬이 아닙니다. 후보는 공간별로 나눠 부르지 않고 **한 번에** 모아 해석합니다. 호출이 나뉘면 각기 다른 저장 알고리즘이 선택되어 순위를 매기려는 공간들 사이에서 점수가 비교 불가능해집니다.

0-8. 의미 줌: 멀리서 볼 때 보이는 것

- 0.45 줌 아래에서는 노트 대신 **뭉친 것들**을 그립니다. 기존 줌 범위(0.15~2.2)에서 포스트잇 글자가 읽히지 않게 되는 지점입니다. 계층은 두 단계입니다 — 군집 위에 주제, 주제 위에 지식 섬을 만들 데이터가 없고, 없는 계층을 지어내면 근거 없는 구조를 사실처럼 보여 주게 됩니다.
- **노트 25개 미만이면 전환하지 않습니다.** 요약은 정보 과부하를 푸는 기능이고, 과부하가 없는 캔버스에서는 같은 것의 더 나쁜 뷰입니다(6개짜리 공간이 `fitView` 때문에 스스로를 요약하던 실측 문제).
- **판정할 수 없으면 배치로 묶습니다.** 낮은 줌에서 군집은 노트를 *대체*하므로, 어휘로 묶인 군집을 그렇게 보여 주면 워크스페이스에 대해 자신 있게 틀린 그림을 줍니다. umm은 공간 도구이고 노트 위치는 사람이 결정한 데이터이므로, 배치 기준 군집은 지어낸 구조가 아니라 실재하는 구조를 보고합니다. `ThoughtCluster.Basis` 가 어느 쪽인지 알고, 화면도 실선/점선으로 구분합니다 — 다른 주장이기 때문입니다.
- 배치 군집은 **단일 연결**입니다. 한 줄로 늘어놓은 생각은 쌍의 사슬이 아니라 하나의 흐름이고, 사람들은 실제로 그렇게 배치합니다. 거리 기준 120px은 절대값이고 그래도 됩니다 — 유사도와 달리 캔버스에는 사람이 작업하는 고정된 척도가 있습니다(기본 노트 260×180).
- 덩어리는 **멤버들이 차지한 땅을 그대로 덮습니다.** 줌아웃이 레이아웃을 재배치하면 umm에서 가장 중요한 사용자 데이터를 읽습니다. 어느 덩어리에도 속하지 않은 노트는 그대로 남습니다 — 숨기면 워크스페이스가 실제보다 정돈된 것처럼 보입니다.
- 라벨 크기는 화면 픽셀이 아니라 **캔버스 단위**입니다. 이 뷰는 0.45 아래에서만 나타나므로 쓰인 크기의 절반 이하로 그려집니다.

1-1-0. 판정 기준은 설정이고, 바닥은 설정이 아닙니다

- 유사도 문턱은 전부 `app_settings` 의 `intelligence` 영역에서 옵니다(`store.IntelligenceSettings`). 기본값은 측정해서 정한 상수 그대로라 바꾸지 않으면 동작이 같습니다.
- 기준은 표준편차 단위이므로 임베딩 백엔드를 바꿔도 같은 뜻을 유지합니다. 저장된 값이 범위를 벗어나면 **그 항목만** 기본값으로 되돌아갑니다 — 잘못된 설정 하나가 전체를 멈추지 않습니다.
- 저장 시에는 조용히 보정하지 않고 **거부**합니다. 표준편차 칸에 40을 넣은 관리자에게는 그게 표준편차가 아니라고 말해야지, 화면과 다르게 동작해서는 안 됩니다.
- `Semantic` 판정에서 `Discrimination > 0` 은 **설정이 아닙니다.** 관문의 두 하한은 낮출 수 있지만, 어휘가 뜻을 앞서는 백엔드는 어떤 값으로도 통과하지 못합니다. 관문 전체가 그 상태를 막으려고 존재합니다.
- 품질 리포트는 캐시되는데 판정은 기준에 의존하므로, 캐시된 리포트를 **다시 재지 않고 다시 판정**합니다(`QualityReport.WithBars`). 측정값은 기준과 무관하므로 정확하고 무료이며, 설정 변경을 보지 못한 다른 인스

턴스도 같은 숫자에서 같은 결론에 도달합니다.

- 설정 캐시는 30초입니다. 캔버스 로드와 검색마다 읽히므로 매번 DB를 왕복하면 설정 테이블이 핫 패스에 올라옵니다.

1-1-1a. 게이트웨이 자동 탐색

- 제대로 된 임베딩을 붙이는 데 남아 있던 마찰은 **모델 이름을 모른다**는 것이었습니다. `GET /admin/ai-gateway/discover` 가 알려진 주소에 OpenAI 호환 `/v1/models` 를 물어 응답한 게이트웨이와 모델 목록을 돌려줍니다.
- 조사 주소는 바이너리에 고정되어 있고 요청에서 읽지 않습니다. 읽는다면 이 화면이 서버가 닿을 수 있는 모든 곳을 조사하는 수단이 되고, umm 자신의 사이드카를 찾는 데는 그런 것이 필요 없습니다.
- 후보 응답은 신뢰하지 않습니다. 그 포트에 다른 것이 떠 있을 수 있으므로 본문 크기를 제한하고, HTML·오류·잘못된 JSON에서 모델이 나오지 않는지 테스트로 고정합니다.
- `likelyEmbedding` 은 이름 기반 힌트입니다. 게이트웨이는 서빙하는 모든 모델을 나열하고 무엇이 임베딩인지 응답에 없습니다. 채팅 모델 사이에서 후보를 위로 올릴 뿐이고, 판정은 `/admin/ai-gateway/test` 와 품질 측정이 합니다.
- 후보는 선언 순서대로 보고합니다. 가장 빨리 응답한 호스트가 아니라 umm이 문서에 적어 둔 사이드카가 먼저 와야 합니다.

1-1-2. 임베딩 품질을 운영자에게 노출

- 설정한 임베딩 모델이 실제로 쓰이고 있는지는 제품 밖에서 알 방법이 없었습니다. 게이트웨이가 죽거나 모델명에 오타가 나면 umm은 조용히 로컬 임베딩으로 폴백하고, 화면은 그대로 "연관 생각"과 "의미상 유사"를 말합니다.
- `GET /api/v1/admin/embedding-quality` 가 설정된 백엔드로 라벨 데이터를 직접 임베딩해 판별력·쌍별 정확도·주제 분리·최근접 동일 주제와 함께 세 가지 상태를 구분해 반환합니다: 미설정(`semantic=false`), 설정했으나 폴백 중(`fellBack=true`), 정상(`semantic=true`).
- provider는 `EmbeddingProvider` 가 아니라 설정에서 직접 해석합니다. 회로 차단기가 열려 있는 동안 로컬로 치환된 값을 보여 주면, 장애를 조사하러 온 운영자에게 정상처럼 보이기 때문입니다.
- 결과는 백엔드 identity 기준 10분 캐시입니다. 설정 화면을 열 때마다 22쌍(44문장)을 운영자의 추론 서버로 다시 보내지 않기 위해서입니다.
- 문장쌍만으로는 부족합니다. 쌍 데이터는 "이 둘이 같은 뜻인가"를 묻지만 연관 생각과 군집은 "이 공간에서 무엇이 무엇과 묶이는가"를 묻습니다. 라벨된 4개 주제 16문장으로 **최근접 이웃이 같은 주제인 비율**을 함께 재며, `semantic` 판정에 포함합니다. 실제로 쌍 지표 1위 모델이 이 항목과 종단 군집 테스트에서 떨어졌습니다.
- 지표는 **모델 선택의 순위표가 아닙니다**. 집계 점수가 높아도 특정 워크스페이스에서 더 나은 군집이 나온다는 보장은 없습니다. 모델마다 틀리는 문장이 다르고, 메모가 적을수록 한 번의 실수가 결과를 지배합니다. 후보 비교는 `docs/admin-guide.md` 에 있습니다.
- 임베딩 모델을 umm 이미지에 번들하지 않는 이유는 `CGO_ENABLED=0` 정적 단일 바이너리입니다. 대신 `compose.yaml` 의 `embeddings` 프로파일로 OpenAI 호환 추론 서버를 옆에 세웁니다. 폐쇄망에서도 모델을 한 번만 반입하면 됩니다.

1-1-3. 뜻과 출처를 나눠 기록하는 연결

- `note_edges.relation` 은 요청 본문에서 온 자유 텍스트였고 출처 칸이 없었습니다. 실측 결과 일반 사용자가 `relation='dreamed'` 로 **Dream**이 발견했다고 주장하는 엷지를 만들 수 있었고, 5000자 문자열도 저장돼 캔버

스가 라벨로 렌더링했습니다.

- 모델링 오류는 하나였습니다. 한 칸이 뜻과 만든 주체를 함께 담고 있었습니다. `relation` 은 검사된 어휘 (`related·supports·contradicts·refines·expands·follows`)로 뜻만 담고, `origin` (`manual·agent·dream·development·import·auto`)이 주체를 담습니다.
- `origin` 은 요청 본문에서 읽지 않습니다. 저장소의 단일 쓰기 경로 `createEdge` 가 호출하는 코드에서 인자로 받으므로, 핸들러가 나중에 JSON을 그대로 바인딩해도 주체를 고를 수 없습니다. 웹은 `manual` , MCP는 `agent` 입니다.
- 어휘 밖의 값은 `related` 로 조용히 바꾸지 않고 400과 `allowedRelations` 로 거부합니다. 사용자가 설명하지 않은 연결을 기록하고 실수를 감추지 않기 위해서입니다.
- `confidence` 는 `origin='auto'` 일 때만 값을 갖도록 제약이 걸려 있습니다. 점수 없는 추측은 순위를 매길 수 없고, 사람이 그 선의 점수는 아무도 만들지 않은 숫자입니다.
- `UNIQUE(source,target)` 을 `UNIQUE(source,target,relation)` 으로 바꿔 한 쌍에 여러 뜻이 공존합니다. 기계 추천이 사람이 그 선을 덮지 못합니다. 역방향 탐색용 `(target_note_id, relation)` 인덱스를 추가했습니다 — 이전에는 전체 스캔이었습니다.
- 엣지를 읽는 모든 경로가 `origin` 과 `confidence` 를 실어야 합니다. v0.11.0에서 `Backlinks` 하나를 빠뜨렸고 아무것도 실패하지 않았습니다 — 필드가 빈 값으로 도착했을 뿐이라 화면에는 출처 없는 연결이 나왔습니다. OpenAPI는 처음부터 `Edge.origin` 을 필수로 요구하고 있었으므로 명세가 맞고 구현이 틀렸는데 확인하는 것이 없던 경우입니다. 지금은 테스트가 고정합니다.
- MCP의 `get_connections` 가 이 그래프를 에이전트에게 열어 줍니다. `get_related_notes` 가 임베딩으로 비슷해 보이는 것을 찾는 데 반해, 이쪽은 실제로 기록된 연결을 방향·뜻·출처와 함께 돌려주므로 에이전트가 자기가 쓴 부분을 식별할 수 있습니다.
- 마이그레이션 010의 데이터 이동 경로는 `scripts/graph-migration-drill.sh` 가 실제 데이터로 앞뒤로 굴러 검증합니다. `migrate-dry-run.sh` 는 빈 스키마에서 돌기 때문에 이 부분을 건드리지 않습니다.
- 백필의 한계는 남습니다. 릴리스 전에 위조된 `dreamed` 엣지는 진짜와 구별할 정보가 기록된 적이 없어 `origin='dream'` 으로 함께 옮겨집니다.

1-1-4. 자동 연결과 그 관문

- `SuggestLinks` 는 공간의 모든 짝을 채점해 그 공간 자신의 분포에서 두드러지는 것들을 `origin='auto'` 엣지로 기록합니다. 사라지는 목록이 아니라 실제 엣지인 이유는, 지금 답하지 않으면 없어지는 제안은 없느니만 못하기 때문입니다.
- 백엔드가 의미를 판별한다고 측정되지 않으면 실행을 거부합니다. `MeasureEmbeddingQuality(...).Semantic` 이 관문입니다. 기본 로컬 알고리즘은 뜻보다 어휘를 높게 보므로(쌍별 정확도 4.2%), 그 위에서 제안하면 단어가 겹치는 무관한 생각들을 이어 놓게 됩니다.
- `outcome` 은 `suggested / no-candidates / backend-not-semantic / too-few-notes` 네 가지입니다. 조용한 결과에도 종류가 있고, "찾지 못했다"와 "판단을 거부했다"는 다른 대응을 부릅니다. 거부 경로에서도 `edges` 는 항상 배열입니다.
- `confidence` 는 확률이 아니라 그 공간의 평균에서 몇 표준편차 위인지를 0.5~0.99로 옮긴 값입니다. 화면에도 "두드러진 정도"로 적습니다. 측정이 뒷받침하는 주장은 그것뿐입니다.
- 거절은 `link_dismissals` 에 남습니다. 자동 연결은 이미 연결된 짝을 건너뛰므로, 추천을 지우면 건너뛴 근거까지 사라져 다음 실행에서 그대로 다시 올라왔습니다(실측). 짝은 방향과 무관하게 정규화해 저장합니다. 사람이 그 연결을 지울 때는 거절을 기록하지 않습니다 — 나중에 다시 제안받고 싶을 수 있습니다.

- 한 번 실행에 최대 12개입니다. 제안이 쌓이면 사람은 전부 무시합니다.

1.2. 인덱스를 타는 하이브리드 검색

- 키워드 조건은 `단어마다 하나의 ILIKE` 를 AND로 묶은 형태로 생성됩니다. 각 조건이 `pg_trgm` GIN 인덱스를 타고 PostgreSQL이 비트맵을 교집합합니다.
- 인덱스는 `(title || ' ' || content)` 표현식 위에 만들어집니다. 코드의 표현식과 한 글자라도 달라지면 조용히 순차 스캔으로 되돌아가므로, 두 값이 일치하는지 검사하는 테스트를 함께 둡니다.
- 의미 점수 후보는 최근 문서로 상한을 둡니다. 큰 워크스페이스에서 한 번의 검색이 전체 스캔이 되지 않게 하기 위해서입니다.

2. 낙관적 동시성 제어 (Optimistic Concurrency Control)

- 각 생각 노드는 단조 증가하는 `version` 번호를 가집니다.
- 다중 사용자가 동시에 작업하더라도 버전 충돌을 감지하고 409 Problem Details에 최신 서버 메모를 포함합니다. 브라우저는 내 변경과 서버 변경을 나란히 보여 주고 선택·병합한 뒤 새 버전으로 저장합니다.
- 댓글 해결·재개·삭제 transaction은 대상 댓글을 update lock으로, 활성 생각·공간과 비소유자의 현재 membership 권한을 shared lock으로 고정된 다음 mutation과 `space_events` ·webhook outbox를 함께 commit 합니다. 멤버 제거와 `edit/manage → view` 강등은 이 transaction 뒤에서 직렬화되므로 point-in-time 권한 확인으로 협업 상태를 뒤늦게 바꾸지 못합니다.
- 오프라인 변경은 PWA local queue에 먹통 키와 함께 보관됩니다. 재연결 시 안전한 순서로 replay하고 같은 메모의 연속 PUT은 마지막 상태로 합칩니다. 모든 browser storage 읽기·쓰기·삭제는 공용 예외 경계 안에서 수행하며, quota 또는 권한 오류가 나면 mutation을 저장했다고 보고하지 않습니다. Canvas는 메모별 마지막 서버 응답 또는 성공적으로 기록된 queue 상태를 별도로 유지하고, 비내구성 autosave가 실패했으며 그 뒤 새 편집도 없다면 그 상태로 즉시 복원합니다. 요청 중 시작한 더 최신 편집은 객체 세대로 구분해 먼저 되돌리지 않고 자신의 직렬화된 저장 결과로 확정합니다. 인증된 queue owner는 메모리에도 유지해 저장소 접근이 차단되어도 앱 초기화와 상태 조회를 계속하고, 읽을 수 없거나 손상된 기존 queue는 빈 배열로 덮어쓰지 않습니다. 인증 복구 가능성이 있는 일반 401/403은 queue를 멈추지만, 공간에서 제거되었거나 생각이 삭제되어 댓글 해결·삭제를 더는 적용할 수 없는 경우 서버가 `comment-mutation-forbidden` 403을 반환하므로 그 항목만 사유를 알리고 제거한 뒤 이후의 독립 변경을 계속 처리합니다. 서비스 워커의 / 앱 셀 개시는 성공한 `text/html` 탐색 응답만 갱신하므로 manifest·asset JSON·SVG를 주소창에서 직접 열어도 오프라인 셀이 비-HTML 응답으로 바뀌지 않습니다. 8자 content hash가 붙은 `assets/` bundle만 1년 immutable이고, manifest·service worker·아이콘·asset manifest 같은 고정 URL은 `no-cache` 로 매 배포 재검증합니다.

3. 분산 큐 & DB 트랜잭션 (SKIP LOCKED)

- 외부 메시지 브로커(RabbitMQ, Redis 등) 없이 PostgreSQL의 `FOR UPDATE SKIP LOCKED` 를 활용하여 Dream 백그라운드 작업을 분산 워커 간 중복 없이 안전하게 임대(Lease) 처리합니다.
- 생성된 Dream은 `dream_notes.note_id IS NULL` 인 검토 후보로 먼저 저장됩니다. Today·검토함·상세와 source query는 Dream 소유권뿐 아니라 현재 공간 owner/member 권한을 함께 확인하므로 공유가 회수되면 파생 본문·공간명·원본도 응답에서 빠집니다. AI Assist와 자동 Dream 생성은 선택·선별된 원본 note 행, 원본 공간 행과 membership를 잠그며 재생성·발전은 Dream 행과 Dream 공간까지 같은 lease에 포함합니다. 공간 ID를 정렬된 순서로 잠근 채 외부 Gateway 호출 종료까지 유지하므로, 공유 회수가 먼저 확정되면 쿼터 예약을 호출 전에 취소하고 AI lease가 먼저 시작되면 회수가 호출 종료까지 기다립니다. 긴 lease는 request pool을 빌리지 않는 인스턴스당 2-slot PostgreSQL 용량으로 제한되어 세 번째 호출은 연결을 점유하지 않고 대기합니다. 자동 생성에서

품질을 통과한 Dream·source·알림도 이 transaction에서 함께 확정되어 point-in-time 조회 뒤 권한 변경으로 본문이나 부분 후보가 남지 않습니다. 채택 요청은 같은 방식으로 현재 공간·편집 membership을 잠근 뒤 메모와 dreamed 연결선을 한 번만 생성합니다. 채택 후 발전 결과도 같은 권한·Dream 행 잠금 아래 새 메모와 expanded 연결선을 원자적으로 만들며, 동일 본문 재시도는 기존 결과를 반환합니다.

- Dream 노출·숨김·선호 피드백은 목록 응답 시점이 아니라 브라우저 IntersectionObserver에서 카드가 50% 이상 보인 시점에 기록됩니다. 서버는 Dream 행과 현재 공간 membership을 같은 transaction에서 잠근 뒤 상태와 개인화 점수를 함께 확정하므로 권한 회수 뒤 보관한 ID로 피드백을 만들 수 없습니다. 알림의 resourceType/resourceId는 Dream 카드 또는 공유 공간 딥링크로 해석됩니다.
- 공간·메모의 ai_excluded 정책은 Scheduler 자격 계산과 AI Assist·Dream Gateway 직전의 최종 source lease에서 모두 다시 적용되어, 설정 변경 이후의 신규 AI 처리에서 제외됩니다.

4. Daily review와 하이브리드 탐색

- /today는 14일 이상 검토하지 않은 생각, 사용자가 지정한 다시 보기, 연결선이 없는 생각, 대기 Dream, 최근 댓글을 현재 권한과 삭제 상태 범위 안에서 집계합니다. 알림 목록과 unread count도 note 알림의 현재 생각 행과 Dream 알림의 현재 Dream 행을 join해 실제 공간 접근권한을 같은 기준으로 적용하며, legacy 알림의 비어 있는 resource_space_id를 신뢰 경계로 사용하지 않습니다. note_reviews가 공유 메모의 검토·고정 상태를 사용자별로 분리하고, review_digest는 협업 활동 query만 제어해 기본 검토 신호를 제거하지 않습니다.
- 검색은 완전 로컬 192차원 feature-hash cosine 점수와 제목·본문·공간 키워드 점수를 결합합니다. 응답용 본문 일부가 잘려도 전체 본문의 키워드 일치 플래그를 점수에 보존하며, 공간·종류·수정 시각 필터와 opaque cursor를 지원합니다.
- 백링크는 실제 note_edges의 incoming/outgoing 방향을 반환하며 연관 생각은 의미 유사성을 보완합니다.

5. 자동화·관측성·공급망

- 도메인 변경, PostgreSQL SSE log, 활성 구독별 webhook_deliveries outbox는 같은 트랜잭션에서 커밋됩니다. 세 워커가 가용 dispatch slot을 확보한 뒤 FOR UPDATE SKIP LOCKED로 대기 또는 lease가 만료된 처리 항목을 선점하므로 프로세스 재시작과 순간 부하 뒤에도 전달을 이어갑니다. 각 워커는 claimed_at 세대가 정확히 일치하는 delivery와 subscription·owner·space·membership을 잠그고 실제 HTTP 응답, terminal delivery 전환, payload 삭제, subscription counter 갱신까지 같은 transaction에서 확정합니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 끝나면 외부 전송을 시작하지 않고, 전달 lease가 먼저 시작되면 정책 변경이 그 종료까지 기다리므로 point-in-time 확인 뒤 캡처한 payload가 새 정책을 넘어 나가지 않습니다. HMAC 서명, SSRF 재검증, 제한 재시도와 자동 중지를 적용하며, at-least-once 전달 시도의 중복은 delivery UUID로 수신 측이 제거합니다. terminal metadata는 30일 보존합니다. 외부 오류는 잘못된 UTF-8을 제거하고 byte 상한 안의 완전한 rune 경계에서 잘라 PostgreSQL 상태 전환과 실패 횟수 갱신을 안정적으로 끝냅니다.
- HTTP 계층은 low-cardinality route pattern으로 Prometheus count-latency를 기록합니다. /api/v1/metrics는 관리자 브라우저 세션 또는 명시적인 metrics:read API 키만 허용하고, 일반 세션의 wildcard와 관리자 소유 일반 키는 운영 지표 권한으로 해석하지 않습니다. 표준 OTLP 환경변수가 있을 때만 OpenTelemetry exporter를 초기화합니다.
- 태그 파이프라인은 offline image archive와 SPDX SBOM을 만들고 checksum 및 GitHub artifact attestation을 발급합니다.

6. 남용 방지와 수평 확장의 경계

- 로그인 실패 횟수와 AI 하루 사용량은 PostgreSQL에서 셉니다. 비밀번호 확인부터 실패 기록 또는 session commit까지 주소·계정별 advisory transaction lock 안에서 처리하고 모든 DB 작업은 같은 연결을 사용하므로, 인스턴스가 여러 대이거나 pool 상한이 1이어도 병렬 추측이 설정한 실패 상한을 넘어가지 않습니다.
- 일반 API 요청 한도는 인스턴스별 인메모리 토큰 버킷입니다. 요청마다 데이터베이스를 왕복하면 막으려는 부하보다 비용이 커지기 때문이며, 그 결과 실패 상한은 $\text{설정값} \times \text{인스턴스 수}$ 가 됩니다. 전역으로 지켜져야 하는 한도는 앞의 두 가지이고, 그것은 데이터베이스에서 셉니다.
- 보안 섹션의 whole-object PUT은 설정별 PostgreSQL advisory transaction lock을 얻은 뒤 최신 행을 읽고, 구버전 payload가 생략한 다섯 남용 방지 필드만 병합합니다. OIDC `client_secret` 과 AI Gateway `api_key` 의 마스킹 저장도 같은 방식으로 비밀 필드를 생략해 최신 암호문을 transaction 안에서 병합하며, master-key 회전은 같은 두 lock을 정해진 순서로 잡고 재암호화합니다. 동시 관리자 저장과 롤링 업그레이드·키 회전이 겹쳐도 먼저 확인한 한도나 새 암호문을 stale snapshot으로 지우지 않으며 명시한 0은 omission과 구분합니다.
- 계정별 로그인 잠금 임계값은 주소별의 3배입니다. 아이디를 아는 사람이 남의 계정을 임의로 잠글 수 있는 상태를 피하기 위한 의도적인 비대칭입니다.
- 만료된 세션·OAuth state·재시도 기록·로그인 실패 기록은 15분마다 정리됩니다. 모든 인스턴스가 동시에 실행해도 안전합니다.
- `scripts/multi-instance-smoke.sh` 가 이 세 가지(교차 인스턴스 이벤트 전달, 교차 인스턴스 맥등 재시도, 공유 로그인 잠금)를 CI에서 실제로 검증합니다.